基于热质耦合退化扩散方程与 ALE 动边界方法的圆柱药材热风烘干模型

摘　要

本文以圆柱药材的一维径向热质传递为对象，建立由常物性解耦、变物性耦合、长时程首达判定到收缩动边界逐级扩展的统一模型。控制方程采用第三类边界条件，以节点中心有限体积法和后向 Euler 格式离散，并结合 Kirchhoff 界面系数、Picard 迭代、括区二分及 ALE 映射完成求解；通过解析解、网格与时间步收敛、守恒残差和反事实对照构成验证链。

针对问题一，附录二物性均为常数，且温度、含水率不进入对方方程及其系数，故热湿解耦是物性形式的推论而非人为简化。计算得到 1800 s 时中心与表面温度分别为 33.5756 °C 和 36.7857 °C，中心含水率仍为 2.5500 kg/kg。$Le = 34.1975$、$Fo_m = 0.02222$ 表明热扩散约比质扩散快 34 倍，水分迁移主要局限于约 0.298 cm 的表层；Duhamel–Bessel 独立解的最大温度偏差仅为 2.98×10^{-4} K。

针对问题二，密度、比热和导热系数随含水率变化，扩散系数又通过 Arrhenius 因子依赖温度，从而形成双向系数耦合。本文采用固定旧时层、仅更新系数的 Picard 迭代，固定两轮与收敛至 10^{-10} 的结果相对差仅为 7.51×10^{-11}。求得 3 h 时中心温度为 49.8495 °C，体积平均含水率为 1.3825 kg/kg；全程比热和导热系数分别下降 47.09% 和 46.26%，说明常物性近似不适用于该工况。

针对问题三，以离散全场最大含水率首次达到 0.15 kg/kg 作为烘干终点，经括区二分得到**烘干时长** 57.4745 **h**（2.3948 **d**）。若误用体积平均含水率判定，仅得 36.0167 h，将提前 21.4578 h，即低估 37.3%。干燥过程无恒速段，内部扩散阻力占比由 54.29% 上升至 95.24%，表明后期由内部扩散控制，提高外部传质能力的边际收益逐渐下降。

针对问题四，收缩 Péclet 数 $Pe_{\mathrm{shr}} = 6.7104 > 1$，故采用 ALE 映射 $x = r/R(t)$ 将收缩区域转化为固定计算域，并以保单调三次插值描述实测半径。计算得到**烘干时长** 51.0888 **h**（2.1287 **d**），终点半径为 1.2000 cm。尽管附录四扩散系数仅为附录三的 0.1856 倍，收缩仍将几何尺度因子放大至 2.7871 倍；在相同物性下冻结半径重算，烘干时长增至 129.8468 h，相对 ALE 结果变化 60.65%，说明几何收缩是本模型中加快干燥的主导机制。

独立 ANSYS 有限元复核显示，各工况温湿场趋势与本文结果一致，问题三终点时间差约 0.052%；灵敏度分析表明，传质系数 h_m 是主要影响因素，而问题三 93.04% 的时长依赖空气条件外推，构成首要不确定性来源。本文揭示了终点判据、内部扩散阻力与几何收缩对干燥时间的共同作用。

关键字： 热质耦合　退化扩散　有限体积法　ALE 动边界　首达时刻　不确定性分析

一、 问题重述与问题分析

1.1 问题背景与待解决的任务

药材烘干是中药材产地加工的关键工序，干燥不足会残留水分导致霉变，干燥过度又会破坏有效成分，因此烘干时长的确定直接决定成品质量与能耗[1]。生产上常用的做法是凭经验设定固定烘干时间，但同一批药材的几何尺寸、初始含水率与热风工况一旦变化，经验时长就不再可靠[2]。要把烘干时长从经验量变成可计算量，必须回到药材内部传热与传质的机理本身。

题目给出的对象是一段长 25 cm、半径 2 cm 的圆柱形药材，初始温度 28 °C、初始含水率 2.55 kg/kg （以干基计，即每千克干物质所含水分质量）。药材被送入热风烘房，先经历约 30 分钟的预热升温，随后进入长时间的恒温脱水阶段。热风的温度与含水率由附件 1 逐点实测给出，覆盖 $0 \le t \le 14400$ s（241 个等间距点，间隔 60 s）；药材半径随脱水而收缩，其实测历程由附件 2 给出，覆盖 $0 \le t \le 259200$ s。药材表面与热风之间的换热与传质分别由对流换热系数 $h = 25$ W/(m^2·K) 与对流传质系数 $h_m = 8 \times 10^{-7}$ m/s 刻画。当药材内部含水率降至 0.15 kg/kg 以下时视为烘干合格。

在此设定下需依次回答四个问题：其一，在附录 2 给出的常物性条件下，求预热阶段 30 分钟内药材内部的温度场与含水率场；其二，改用附录 3 给出的随含水率变化的物性公式，求 3 小时内的温度场与含水率场，且模型须能延伸到整个烘干过程；其三，沿用附录 3 的设定并固定几何，求含水率降至合格所需的时间；其四，改用附录 4 的物性公式并计入附件 2 实测的半径收缩，重新求烘干时长与全过程含水率场。

1.2 四问的依赖关系与总体求解思路

四个问题并不是四个彼此独立的题目，而是同一套控制方程在四组设定下的四次求解。贯穿全题的机理只有两条：药材内部的热量以导热方式向中心传递，水分以扩散方式向表面迁移；两者在表面通过对流与热风交换。真正在四问之间发生变化的是物性的形式、求解区域是否运动、以及终止条件是固定时刻还是待求时刻。据此本文把公共机理前置为一章统一的控制方程与离散方案，四个问题章只写各自的增量。这一依赖关系与总体求解路径如图 1 所示。

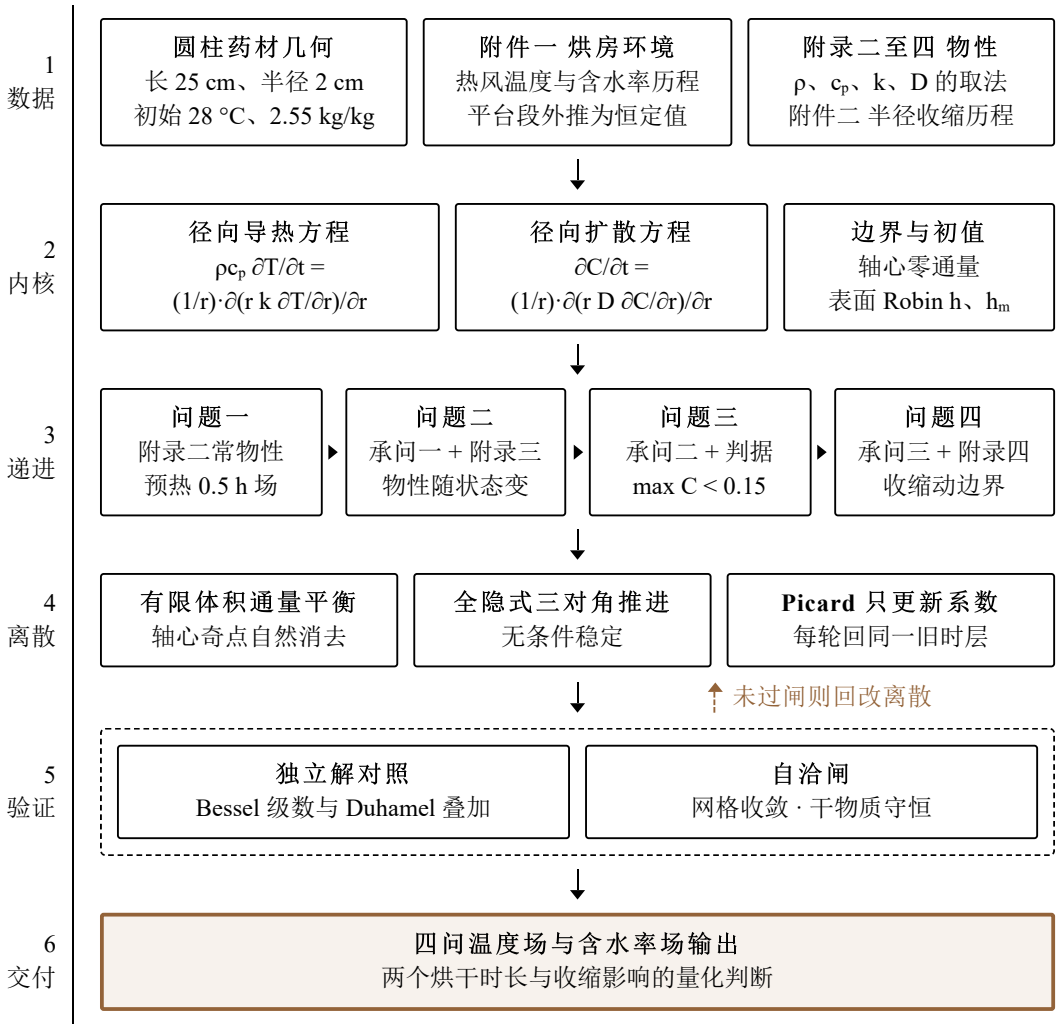


图 1　整体技术路线图

1.3 热风驱动条件的两阶段特征

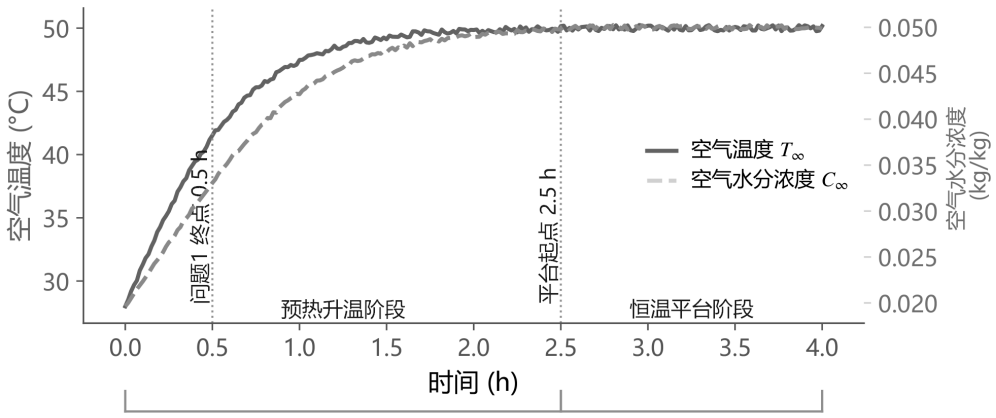


图 2　烘房温度与含水率实测边界条件

四问共用附件 1 的热风驱动。实测曲线显示，热风由初温升至约 50 °C 后进入平台，含水率也稳定在约 0.05 kg/kg（图 2）；问题一的 30 min 位于升温段，问题三、四的数十小时则主要位于平台段。由于附件 1 仅覆盖 14400 s，平台后的条件需作常值延续，并在第 二、 节说明、在第 十一、 节量化。

1.4 问题一的分析

问题一中附录 2 的物性使两场不交换状态量，故温度场与含水率场可以独立求解；含水率方程的退化扩散和轴对称几何则由节点中心有限体积法与全隐式推进处理。常物性线性导热解同时作为数值内核的解析对照，详见第 十、 节。

1.5 问题二、三的分析

问题二改用附录 3 后，物性随含水率变化，扩散系数还含温度的 Arrhenius 因子，因而形成双向耦合的非线性系统；时间步内只更新系数并保持旧时层不变，且求解器须能延伸到三小时以外。问题三不改变方程，只把固定时长输出改为全域最大含水率首次低于阈值的首达判据。

1.6 问题四的分析

问题四同时引入附录 4 物性与附件 2 的半径收缩：前者降低扩散能力，后者缩短迁移路径，净效应须由计算判定。由于收缩 Péclet 数表明动边界不可忽略，本文用 $x = r/R(t)$ 的 ALE 映射固定求解域，显式核对网格运动项与固体对流项的抵消；半径数据采用保单调插值，并将其与密度守恒关系的不一致作为模型限制，主模型仍使用实测半径。

二、 模型假设

题面已经给出几何、初始状态、物性、阈值及表面输运系数；以下六条是为闭合模型而补充的现实简化，并同时说明其理由和影响方向。

（1）**能量方程不含蒸发潜热汇。**题面未给出汽化潜热及相变分配比例，故无法定量闭合该项；本文直接用热风条件驱动固相温度。该处理会使温度、扩散系数偏高，进而使烘干时长偏短，方向性影响在第 十、 节讨论。

（2）**两个表面输运系数在四问中沿用同一取值。**题面只在附录 2 的位置给出 $h = 25$ W/(m^2·K) 与 $h_m = 8 \times 10^{-7}$ m/s，后续附录未重新给值。这两个量刻画的是药材表面边界层内的输运能力，取决于热风流速与烘房结构，而不是药材本身的材料属性，因此在同一烘房、同一风速条件下四问共用是合理的。

（3）**附件 1 覆盖时段之外的热风条件取平台常值。**对 $t > 14400$ s 取 $T_\infty = 50.0$ °C、$C_\infty = 0.05$ kg/kg。平台段已进入稳定状态，故不按升温段趋势外推；该假设覆盖问题三、四的大部分时长，其影响由第 十一、 节量化。

（4）**水分方程写成含水率的守恒形式。**本文以 $\partial_t C = \nabla\cdot(D\nabla C)$ 为主模型，而不采用 $\partial_t(\rho_d C) = \nabla\cdot(\rho_d D\nabla C)$。前者与题面给出的含水率物性和合格判据直接一致，后者仅在 ρ_d 空间均匀时等价；两种形式的差异见第 十一、 节和问题四的质量口径对照。

（5）**收缩为仿射变形。**问题四中药材各处按同一比例向轴心收缩，即半径为 r 处固体的径向速度为 $v_s = r\dot{R}(t)/R(t)$。题面只给出外表面半径的实测历程，内部变形分布未知，仿射假设是与该信息量相容的最简设定，且它保证了收缩过程中不出现内部空腔或物质重叠。

（6）**降为一维径向问题。**药材轴向尺度远大于径向，且扩散时间与长度平方成正比，故忽略轴向和周向输运。该处理使脱水偏慢、给出的烘干时长偏大，与假设（1）的方向相反，但抵消幅度未知，本文不据此声称结果无偏。

三、 符号说明

全文反复使用的核心符号记于下表，少次出现的派生量在首次出现处就地说明。

符号	含义	单位	取值或来源
r	径向坐标，自轴心量起	m	$0 \le r \le R(t)$
t	时间	s	求解自 $t = 0$ 起
$T(r,t)$	药材内部温度	°C	待求，初值 28.0000
$C(r,t)$	药材干基含水率	kg/kg	待求，初值 2.5500
R_0	初始半径	m	0.02
L	药材长度	m	0.25
$R(t)$	当前半径	m	问题四由附件 2 给出
$\dot{R}(t)$	边界移动速度	m/s	由 $R(t)$ 求导
x	无量纲径向坐标 $r/R(t)$	—	$0 \le x \le 1$
$\rho(C)$	湿基密度	kg/m^3	附录 2/3/4 公式
ρ_d	干物质密度 $\rho/(1+C)$	kg/m^3	派生量
$c_p(C)$	比热容	J/(kg·K)	附录 2/3/4 公式
$k(C)$	导热系数	W/(m·K)	附录 2/3/4 公式
$D(C,T)$	水分扩散系数	m^2/s	附录 2/3/4 公式
α	热扩散率 $k/(\rho c_p)$	m^2/s	派生量
h	对流换热系数	W/(m^2·K)	25

续表 1：全文核心符号说明

符号	含义	单位	取值或来源
h_m	对流传质系数	m/s	8×10^{-7}
$T_\infty(t)$	热风温度	°C	附件 1 实测
$C_\infty(t)$	热风含水率	kg/kg	附件 1 实测
C_{th}	合格含水率阈值	kg/kg	0.15
t_{end}	烘干结束时刻	s 或 h	问题三、四待求
$\Phi(C)$	扩散势 $\int_0^C D\,\mathrm{d}s$	m²/s	界面系数用
N, Δt	径向控制体数与时间步长	—, s	200，见各问

四、 热质耦合退化扩散控制方程的统一建立

4.1 几何降维与控制体划分

药材为长 $L=0.25$ m、半径 $R_0=0.02$ m 的圆柱，热风从各方向均匀吹过，故温度与含水率关于轴线轴对称、且在远离端面的主体段沿轴向近似均匀。按假设（6）只保留径向坐标，求解域退化为线段 $r\in[0,R(t)]$，但柱坐标下的体积元仍为 $2\pi rL\,\mathrm{d}r$，因此径向位置对物质量的贡献带有权重 r，这一点在离散时不可忽略（图 3）。

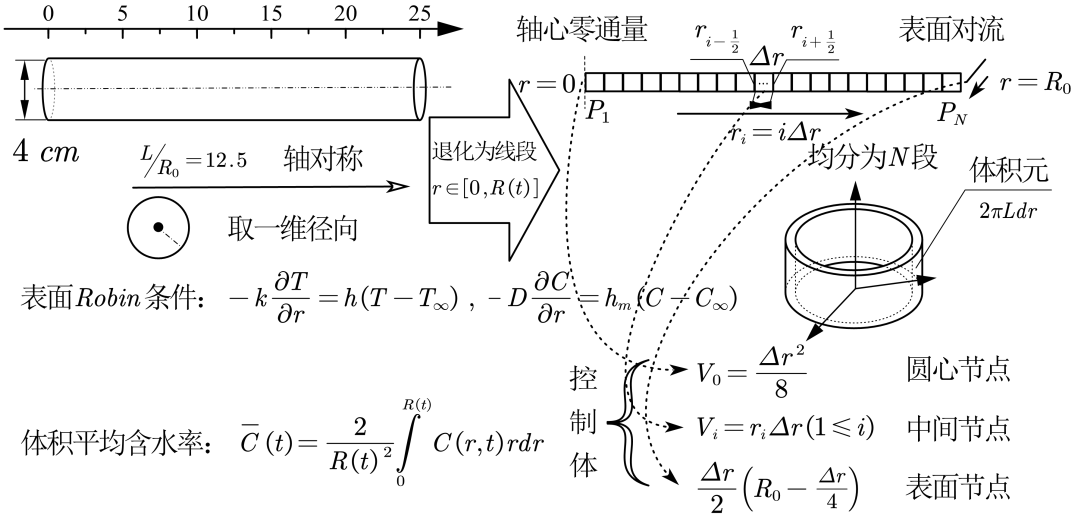


图 3　圆柱几何与径向控制体划分

本文采用图 3 所示的节点有限体积布局：半径等分为 N 段，节点取 $r_i=i\Delta r$（$i=0,1,\ldots,N$），界面取相邻节点中点。轴心内侧界面权重 $r_{-1/2}=0$，通量自然为零，从几何上满足对称条件并避免柱坐标 $1/r$ 奇性 [3]；末节点落在物理表面，交付的径向

位置均可直接读取。三类控制体权重为 $V_0 = \Delta r^2/8$、$V_i = r_i \Delta r$ $(1 \le i \le N-1)$、$V_N = (\Delta r/2)(R_0 - \Delta r/4)$，其和严格为 $R_0^2/2$。体积平均含水率统一定义为

$$\bar{C}(t) = \frac{2}{R(t)^2} \int_0^{R(t)} C(r,t)\, r \, \mathrm{d}r, \tag{1}$$

4.2 控制方程与边界条件

药材内部同时进行两个输运过程：热量由表面向中心导入，水分由内部向表面迁出。前者服从傅里叶导热定律，后者服从 Fick 扩散定律 [4]。在一维柱坐标系下有两条方程：

$$\begin{cases} \rho(C)\, c_p(C)\, \dfrac{\partial T}{\partial t} = \dfrac{1}{r}\dfrac{\partial}{\partial r}\left(r\, k(C)\, \dfrac{\partial T}{\partial r} \right), & (2) \\[2ex] \dfrac{\partial C}{\partial t} = \dfrac{1}{r}\dfrac{\partial}{\partial r}\left(r\, D(C,T)\, \dfrac{\partial C}{\partial r} \right). & (3) \end{cases}$$

式 (2) 中的物性是含水率的函数，式 (3) 中的扩散系数同时依赖含水率与温度，这两处依赖构成了温度场与水分场之间的双向耦合：含水率通过 ρ, c_p, k 改变升温的时间尺度，温度通过 D 中的 Arrhenius 因子改变脱水的速率。耦合路径如图 4。

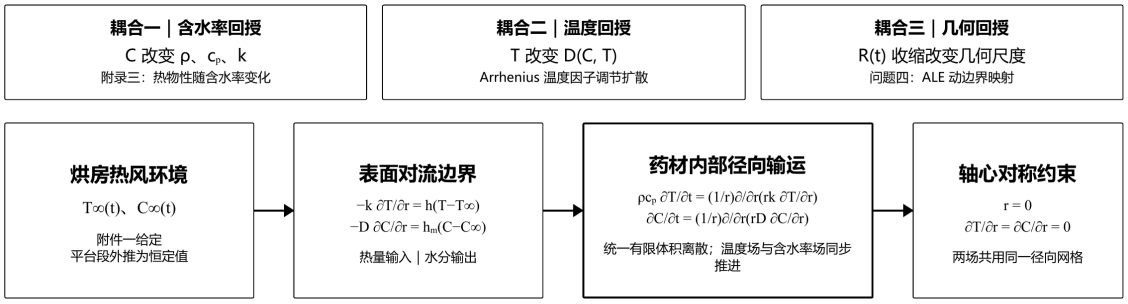


图 4　热质耦合机理框架

热风经边界层驱动药材内部，在轴心取对称条件、表面取第三类（Robin）条件：

$$\begin{cases} \left.\dfrac{\partial T}{\partial r}\right|_{r=0} = 0, \qquad -k(C)\left.\dfrac{\partial T}{\partial r}\right|_{r=R} = h\big[T(R,t) - T_\infty(t)\big], & (4) \\[2ex] \left.\dfrac{\partial C}{\partial r}\right|_{r=0} = 0, \qquad -D(C,T)\left.\dfrac{\partial C}{\partial r}\right|_{r=R} = h_m\big[C(R,t) - C_\infty(t)\big], & (5) \end{cases}$$

初始条件为 $T(r,0) = 28\ ^\circ\mathrm{C}$、$C(r,0) = 2.55\ \mathrm{kg/kg}$。两式右端分别由固气温差和含水率差驱动；$h_m = 8 \times 10^{-7}\ \mathrm{m/s}$ 有限，故表面含水率不能瞬时等于热风值，必须保留传质阻力。

4.3 三组物性设定与统一方程族

三个附录为同一方程组给出三组 ρ, c_p, k, D 闭合关系，四问分别取用其中之一。密度、比热与导热系数中的第二、三式以湿基含水量 $C/(1+C)$ 为自变量，而非干基含水

率 C；在 $C_0 = 2.55$ 时二者分别为 0.7183 与 2.55，故该映射被固化在物性函数内部。扩散系数为：

$$D = \begin{cases} 7.0 \times 10^{-9} \exp\left(-\dfrac{0.89}{C}\right), & \text{附录 2,} \\ 2.4 \times 10^{-3} \exp\left(-\dfrac{0.45}{C}\right) \exp\left(-\dfrac{3850}{T_K}\right), & \text{附录 3,} \\ 4.2 \times 10^{-4} \exp\left(-\dfrac{0.30}{C}\right) \exp\left(-\dfrac{3850}{T_K}\right), & \text{附录 4.} \end{cases} \tag{6}$$

单位为 m^2/s；附录 3、4 的 Arrhenius 项以 $T_K = T + 273.15$ 计算，28 °C 至 50 °C 间该因子增大 2.3878 倍。

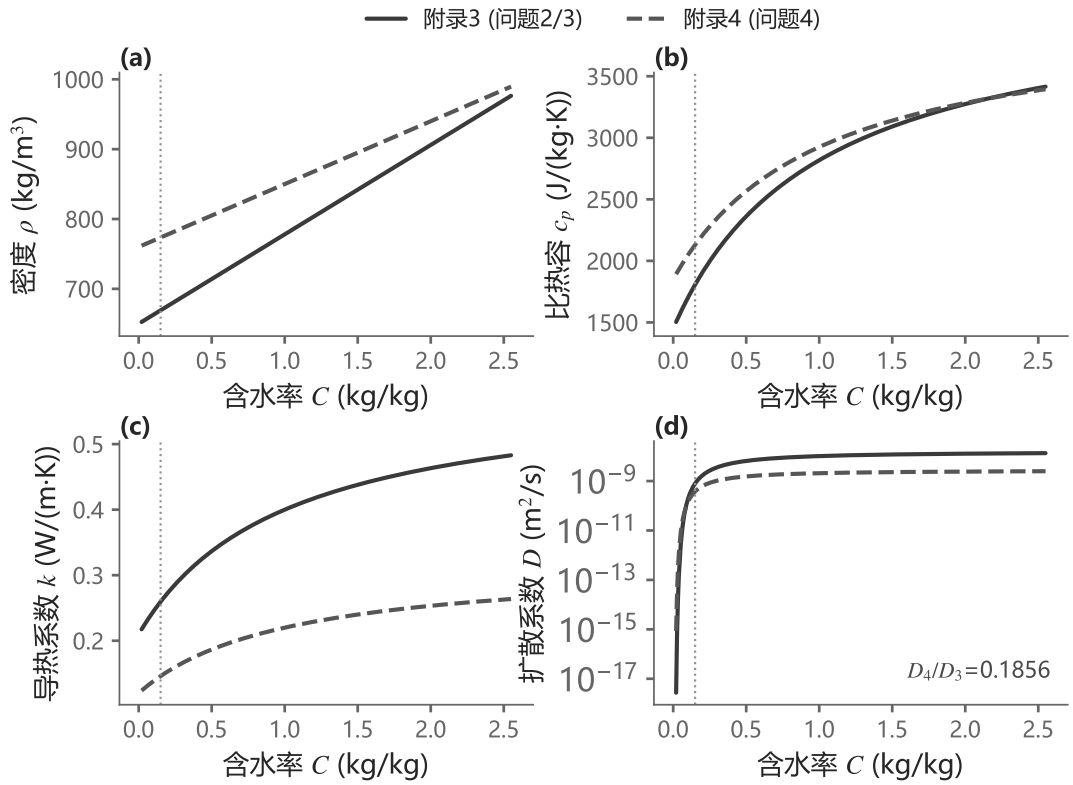


图 5　三组附录物性随含水率的变化

图 5 表明附录 3 的 c_p、k 随含水率显著变化，故问题二起必须采用变系数方程；附录 4 的比热接近附录 3，而导热系数仅为其 55%。

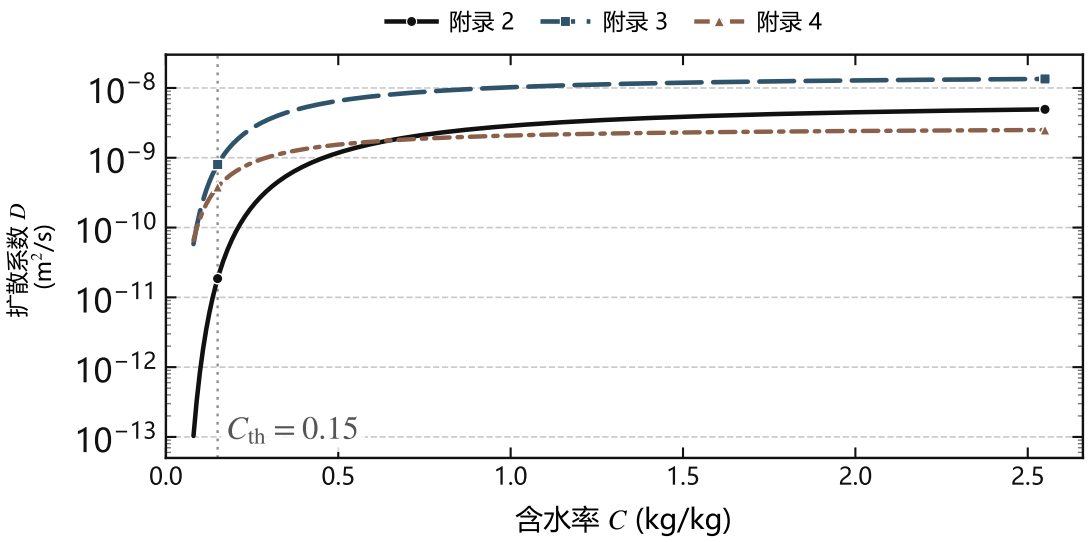


图 6　三组扩散系数的退化特征

因子 $e^{-a/C}$ 使 D 在 $C \to 0^{+}$ 时趋于零，故式 (3) 是退化抛物型（图 6）。C 从 2.55 降至 0.15 时，附录 2、3、4 的 D 分别衰减 266.2175、16.84 和 6.57 倍，说明后期内部输运必然减慢，不能将 D 视为常数。附录 4 的低扩散能力与收缩的几何增强相互竞争，其终点净效应由第 九、 节计算判定。

4.4 无量纲分析与主导机制识别

在离散之前先用无量纲数判断哪些机制会主导解的形态。以 R_0 为长度尺度，方程 (2)、(3) 与边界条件 (4)、(5) 给出四个特征量：

$$Bi_T = \frac{hR_0}{k}, \qquad Bi_m = \frac{h_m R_0}{D}, \qquad Fo_m = \frac{Dt}{R_0^2}, \qquad Le = \frac{\alpha}{D}, \qquad \alpha = \frac{k}{\rho c_p}. \tag{7}$$

Bi 衡量表面与内部阻力之比，Fo 为无量纲时间，Le 为热质扩散速率比。附录 2 下 $Bi_T = 1.3889 > 0.1$，故温度不能采用集中参数近似；$Le = 34.1975$ 表明热扩散快于质扩散。问题一在 1800 s 时 $Fo_m = 0.02222$，对应渗透深度约 0.30 cm（半径的 15%），故中心含水率近似不变。Bi_m 则由 3.2404 升至 862.7，表明控制环节由初期的表面传质与内部扩散并重转为后期的内部扩散主导，该迁移由第 十一、 节定量确认。

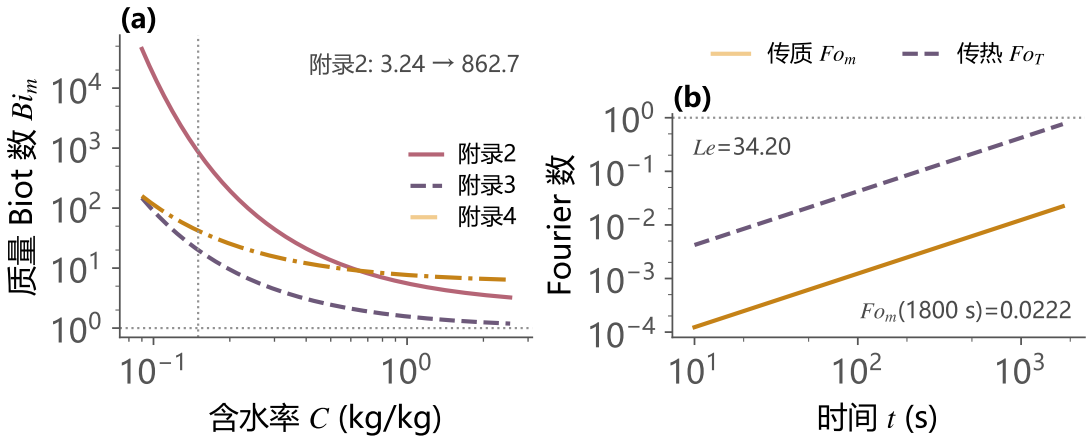


图 7　关键无量纲数的变化范围

图 7 给出上述量随状态的连续变化：Bi_m 随 C 降低急剧上升，Fo_m 在问题一窗口内接近零，形成“表面先干、内部滞后”。因此两场共用时间步推进，步长与表面网格主要按水分方程的非线性和薄层梯度确定，具体离散见下一节。

五、 退化扩散方程的统一有限体积离散与首达时刻检测

四个子问题的控制方程同属“轴对称非线性退化抛物方程组附加、Robin 边界”，差别只在于是否耦合温度、取哪一组物性、半径是否随时间变化、以及终止条件是给定时长还是含水率首次达标。因此本文只建立一套离散内核，四问共用；这样做第十、节的验证一次完成即对四问同时有效，且四问结果之间的差异必定来自物性、几何和终止条件，而非来自不同的主循环实现。

5.1 通量形式的空间离散

在轴对称条件下，体积因子为 $2\pi Lr$。为简化书写，下面省略这一公共因子，把 r 直接吸收到控制体体积和界面面积中。对第 i 个控制体在 $[r_{i-1/2}, r_{i+1/2}]$ 上积分方程 (3)，利用散度结构把体积积分化为界面通量之差，得到守恒型离散：

$$V_i \frac{C_i^{n+1} - C_i^n}{\Delta t} = A_{i+1/2} \frac{D_{i+1/2}}{\Delta r} \left(C_{i+1}^{n+1} - C_i^{n+1}\right) - A_{i-1/2} \frac{D_{i-1/2}}{\Delta r} \left(C_i^{n+1} - C_{i-1}^{n+1}\right), \tag{8}$$

其中界面面积权 $A_{i+1/2} = r_{i+1/2} = (i + \frac{1}{2})\Delta r$，体积权 V_i 按上述定义。式 (8) 在 $i = 0$ 处右端第二项因 $A_{-1/2} = 0$ 自然消失，对称条件由几何而非由额外编码保证；在 $i = N$ 处右端第一项替换为表面对流通量 $-A_N h_m (C_N^{n+1} - C_\infty^{n+1})$，$A_N = R$。能量方程的离散结构与式 (8) 完全相同，只需把 $V_i/\Delta t$ 换成 $\rho c_p V_i/\Delta t$、把 $D_{i\pm1/2}$ 换成 $k_{i\pm1/2}$、把表面导度换成 h。因此，在给定本轮 D、k 和边界系数后，两套方程均为三对角带状线性系统，单步线性求解复杂度为 $O(N)$；非线性只通过外层 Picard 迭代更新这些系数。

求解网格取 $N=200$，即 $\Delta r=1.0\times10^{-4}\,\mathrm{m}=0.01\,\mathrm{cm}$，因此固定半径问题共有 201 个节点。之所以不直接采用交付要求的 0.1 cm，是为了让求解网格与输出网格解耦：0.1 cm 恰为 0.01 cm 的 10 倍，问题一、二输出时取 $i=0,10,\ldots,200$ 的节点值，全过程不做空间插值。问题三、四在 ALE 坐标中使用同一组参考节点，表面值读取末节点；固定物理半径的输出先映射到当前参考坐标，再取最邻近的 ALE 节点，超出当前域的单元留空。体积权之和是否严格等于 $R^2/2$ 在程序中以 10^{-14} 相对误差核对，这是几何权自洽性的零成本判据。

表面控制体的处理需要单独说明，因为这里同时存在“节点位于物理表面”和“节点位于控制体中心”两种常见约定。本文采用前一种节点布局：末节点 $r_N=R$ 就是物理表面，Robin 条件直接施加在该节点上。另一种布局是把半个控制体的内部扩散阻力 $(\Delta r/2)/D$ 与表面对流阻力 $1/h_m$ 串联，取等效导度 $U_{\mathrm{eff}}=[(\Delta r/2)/D+1/h_m]^{-1}$。该串联形式适用于节点位于控制体几何中心、表面在节点之外半格的布局；在本文采用的布局下它会把已经属于表面节点的那半格阻力重复计入。第 十、 节的常环境解析对照给出了判决性证据：串联形式的表面温度误差随网格线性减小（$N=200$ 与 $N=400$ 分别为 1.99×10^{-2} K 与 1.02×10^{-2} K，比值约 2），说明它引入了一项 $O(\Delta r)$ 的一致偏差；而节点形式的误差与网格无关（同两档均为 4.94×10^{-4} K 与 4.91×10^{-4} K），仅由级数截断决定。因此主方案取节点形式，串联形式作为形式选择的对照保留在第 十一、 节。

5.2 界面扩散系数的 Kirchhoff 积分平均

界面系数 $D_{i+1/2}$ 的取法在本题中不仅是精度问题，还会改变退化尾部的排湿速率。由式 (6)，D 含因子 $e^{-a/C}$，在 $C\to0^+$ 时快速趋于零：取附录 3 的 $a=0.45$，该因子在 $C=0.05$ 处比 $C=0.15$ 处小 403.4288 倍。相邻节点跨越这一量级落差时，用算术平均会被大值一侧支配从而高估通量，用调和平均会被小值一侧支配从而低估通量；在有限网格和大梯度下，这两种简单平均都不能可靠代表连续通量。本文采用 Kirchhoff 变换的差商形式 [5]：定义势函数

$$\Phi(C)=\int_0^C D(s)\,\mathrm{d}s,\qquad D_{i+1/2}=\begin{cases}\dfrac{\Phi(C_{i+1})-\Phi(C_i)}{C_{i+1}-C_i}, & C_{i+1}\neq C_i,\\[2mm] D(C_i), & C_{i+1}=C_i.\end{cases}\tag{9}$$

这样取值使离散通量 $-D_{i+1/2}(C_{i+1}-C_i)/\Delta r=-[\Phi(C_{i+1})-\Phi(C_i)]/\Delta r$ 成为 $-\partial_r\Phi$ 的中心差分；在系数足够光滑时，该离散对 Φ 具有相应的二阶空间精度。Φ 有闭式，由分部积分并引入指数积分 $E_1(z)=\int_z^\infty t^{-1}e^{-t}\,\mathrm{d}t$ 得

$$\int_0^C e^{-a/s}\,\mathrm{d}s=C\,e^{-a/C}-a\,E_1\!\Big(\frac{a}{C}\Big),\qquad \Phi(C)=D_0 f_T\Big[C\,e^{-a/C}-a\,E_1\!\Big(\frac{a}{C}\Big)\Big],\tag{10}$$

其中 f_T 是与 C 无关的温度因子（附录 3、附录 4 中 $f_T=e^{-3850/T_K}$，附录 2 中 $f_T\equiv1$ 已并入 D_0）。f_T 与 C 无关正是闭式 (10) 成立的条件，故界面处取 f_T 于 $T_{i+1/2}=$

$(T_i + T_{i+1})/2$，不进入积分。式 (10) 已用数值积分独立核对；该形式避免退化尾部的界面通量被简单平均系统性偏置。

5.3 全隐式时间推进与只作用于系数的 Picard 迭代

退化扩散使方程在含水率低的区域刚性很强：附录 3 参数下场内 D 的最大最小比在问题三后期达到 3.6009，问题四达到 4.6057；显式格式的稳定步长会受到最大扩散系数和最小网格尺度的共同限制，难以满足本题的长时程积分需求。因此时间离散取全隐式后向 Euler，如式 (8) 所示，右端所有空间项都取在新时层。

非线性由 Picard 迭代处理，关键在于迭代只更新系数、不改变时间项的旧值。第 m 轮用 $C^{(m)}, T^{(m)}$ 计算 $D_{i+1/2}, k_{i+1/2}, \rho c_p$，然后求解

$$\mathcal{L}\big[D(C^{(m)}, T^{(m)})\big]\, C^{(m+1)} = \frac{V_i}{\Delta t} C^{n} + b^{n+1}, \tag{11}$$

其中 $\mathcal{L}$ 是由式 (8) 装配的三对角算子，b^{n+1} 收集新时层的边界驱动项 [6]。式 (11) 的时间项始终使用旧时层 C^n，迭代只更新系数，不额外推进物理时间。收敛判据取 $\max_i |C_i^{(m+1)} - C_i^{(m)}| < 10^{-10}$ 且 $\max_i |T_i^{(m+1)} - T_i^{(m)}| < 10^{-8}$ K，上限 50 轮；实际计算中四问的最大迭代轮数分别为 $4, 4, 5, 4$，说明系数的非线性并不强，刚性主要来自空间算子而非迭代格式。

固定两轮与收敛迭代的复算结果相对差为 7.51×10^{-11}（问题二体积平均含水率），问题三终点时刻差为 0；装配残差在四问均不超过 1.22×10^{-11}，故主结果不受 Picard 轮数选择影响。

5.4 求解步长、输出步长与驱动数据插值

问题一、二取 $\Delta t = 0.25$ s，问题三、四取 $\Delta t = 2$ s；输出间隔分别为 1 s 与 60 s，均为求解步长的整数倍，故固定域结果直接从求解层读取。问题四的固定物理半径列先映射至 ALE 坐标并取最近参考节点，超出当前域则留空。更细步长下，问题一温度与问题三终点时刻分别仅变化 0.0000 K 和 0.0003 h，说明主步长已足够。

附件 1 的 $T_\infty(t)$ 与 $C_\infty(t)$ 采样间隔为 60 s，求解时采用分段线性插值以避免过冲；改用保单调三次插值时，问题一温度仅变化 4.85×10^{-4} K，不影响结论。

5.5 烘干终点的首达时刻检测

问题三、四的终点定义为全域含水率首次达标的时刻，而非固定积分时间后的状态：

$$g(t) = \max_{0 \le r \le R(t)} C(r,t) - C_{\text{th}}, \qquad t_{\text{end}} = \inf\{t > 0 :\ g(t) < 0\}, \qquad C_{\text{th}} = 0.15\ \text{kg/kg}. \tag{12}$$

程序逐步记录最大值位置 $r_{\max}(t)$，不预设其位于中心；考虑到空气含水率变化可能引起回吸，$g(t)$ 也不预设单调。若相邻时间层满足 $g(t^{n-1}) \ge 0 > g(t^n)$，则先获得长度

不超过 Δt 的括区，再从 t^{n-1} 的状态出发，以 $\Delta t/16$ 局部重积分并二分至括区小于 0.2 s。候选终点后继续积分至 $1.05t_{\text{end}}$ 并检查 $g<0$，以排除回吸；最终问题三、四的首达时刻分别为 57.4745 h 和 51.0888 h。若至 120 h 仍未达标，则报告未达标，不作外推。

六、 问题一：附录 2 常物性下解耦热质输运场的求解

6.1 解耦性是推论而非简化

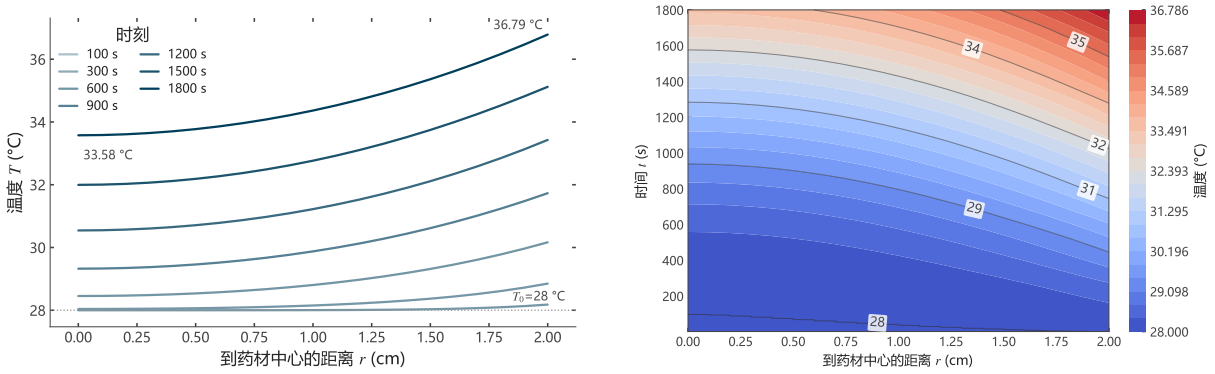
问题一要求在附录 2 物性设定下求解 $t\in[0,1800]$ s 内的温度场与含水率场。此时 ρ、c_p、k 为常数，扩散系数 D 仅依赖含水率而不依赖温度，故式 (2) 与式 (3) 不交换状态量，两场可以独立求解。本文以交换温度场与含水率场的求解顺序后逐点结果一致作为解耦判据。又因 $Bi_T=1.3889>0.1$，且热扩散时标 $R_0^2/\alpha=2368.89$ s 与本问时长同量级，药材内部存在不可忽略的径向温差，不能采用集中参数模型。

6.2 温度场：预热段的分布特征

表 2　30 分钟内药材的温度 (°C)

时间/s	到药材中心的距离/cm				
	0	0.5	1	1.5	2
100	28.0001	28.0004	28.0041	28.0328	28.1802
300	28.0410	28.0637	28.1516	28.3683	28.8491
600	28.4537	28.5364	28.8043	29.3162	30.1654
900	29.3248	29.4587	29.8759	30.6165	31.7306
1200	30.5432	30.7102	31.2227	32.1129	33.4279
1500	31.9961	32.1871	32.7664	33.7465	35.1204
1800	33.5756	33.7723	34.3644	35.3624	36.7857

表 2 和图 8 表明，$t=1800$ s 时中心温度为 33.5756 °C、表面温度为 36.7857 °C，中心–表面温差为 3.21 K，表面仍低于热风 4.73 K。温度场始终位于 $[28.0000,36.7857]$ °C 内并保持单调，说明预热阶段尚未达到径向热平衡。



(a) 温度径向剖面族　　**(b) 温度场等高线**

图 8　问题一温度场与径向剖面

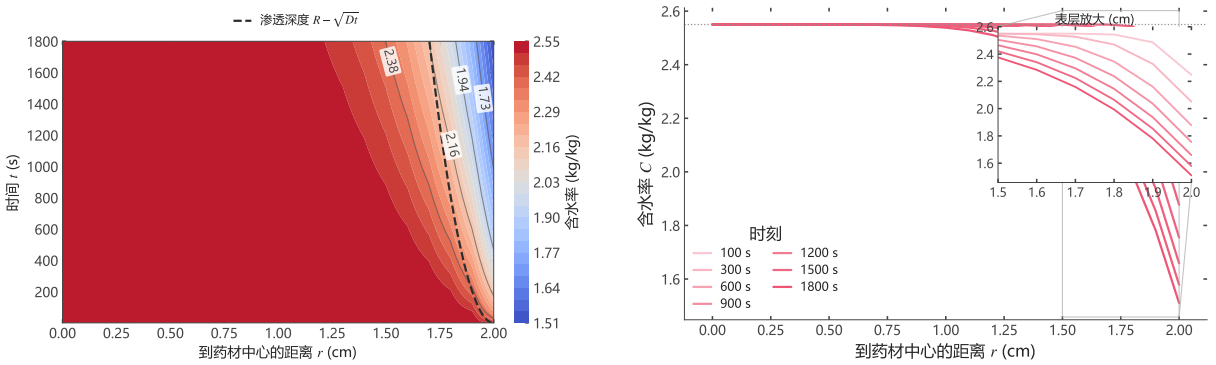
6.3 含水率场：时间尺度分离的定量证据

表 3　30 分钟内药材的水分浓度 (kg/kg)

时间/s	到药材中心的距离/cm				
	0	0.5	1	1.5	2
100	2.5500	2.5500	2.5500	2.5500	2.2474
300	2.5500	2.5500	2.5500	2.5492	2.0510
600	2.5500	2.5500	2.5500	2.5353	1.8771
900	2.5500	2.5500	2.5497	2.5045	1.7548
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5788
1800	2.5500	2.5497	2.5383	2.3755	1.5103

表 3 显示，$t = 1800$ s 时表面含水率已降至 1.5103 kg/kg，中心仍为 2.55 kg/kg，体积平均含水率为 2.2935 kg/kg。此时 $Fo_m = 0.02222$，对应渗透深度约 0.298 cm，仅占半径的 15%；因此表层已明显脱水而中心几乎未受影响。

图 9 进一步表明，含水率变化集中于表层，而温度已在整个截面内显著响应。其根源在于 $Le = \alpha/D(C_0) = 34.1975$，即热扩散速度约为水分扩散速度的 34 倍。含水率场始终单调有界，表层梯度随 D 的退化而逐渐减弱；图中的 $r = R_0 - \sqrt{D(C_0)t}$ 仅作为脱水前沿的扩散标度估计。



(a) 含水率场与渗透深度　　**(b) 含水率剖面与表层放大**

图 9　问题一含水率场与剖面

数值可信度由两类独立检验支撑：时变边界下的 Duhamel 叠加解与数值温度场在 $r = 0, 1, 2$ cm 处的最大偏差为 2.9785×10^{-4} K；网格与时间步加密均呈收敛。更完整的解析对照、收敛序列与误差分离见第 十、 节。

同条件一、二与三维数值对照见图 18，可检验空间维度与离散实现的一致性。

因此，问题一既给出了预热阶段的基准解，也验证了统一离散内核对时变边界和径向输运的处理。问题二将在保持该内核、边界条件和驱动数据不变的基础上，仅引入附录 3 的变物性，由此检验温度场与含水率场由解耦转为双向耦合后的演化。

七、 问题二：附录 3 物性下双向耦合模型的建立与求解

7.1 本问的待求量、定解条件与相对上一问的增量

问题一已验证统一离散内核的可靠性，问题二改用附录 3 的物性关系，求 $t \in [0, 10800]$ s 内的温度场与含水率场；初始、边界条件与热风驱动保持不变。相对问题一，ρ、c_p 与 k 随含水率变化，扩散系数 D 同时依赖含水率和温度：C 通过物性影响温度场，T 又经 Arrhenius 因子影响水分扩散，故两场必须在同一时间步内耦合求解。为满足题目对全过程的要求，输出时长 t_{out} 作为可变参数传入同一求解器。

每个时间步内按第 五、 节的 Picard 迭代更新系数。式 (11) 给出水分方程的一轮更新；能量方程在同一轮中同步装配并求解：

$$\mathcal{L}\big[k(C^{(m)})\big]\,T^{(m+1)} = \frac{\rho(C^{(m)})\,c_p(C^{(m)})\,V_i}{\Delta t}\,T^{n} + b_T^{n+1}, \qquad C^{(0)} = C^{n}, \quad T^{(0)} = T^{n}, \tag{13}$$

其中 $\mathcal{L}$ 为式 (8) 的三对角算子，b_T^{n+1} 收集表面 Robin 驱动。每轮迭代由 $C^{(m)}$ 更新 ρc_p 与 k，并以界面温度更新 D 的 Arrhenius 因子；随后同时求解两场，并以 C、T 的迭代增量共同判定收敛。两式的时间项始终取旧时层 C^n, T^n，故迭代只使系数自洽，不额外推进物理时间。

这种耦合不可忽略：当 C 从 2.55 降至 0.15 时，c_p 与 k 分别下降 47.09% 和 46.26%；温度由 28 °C 升至 50 °C 时，Arrhenius 因子增大 2.3878 倍。因此，含水率变化会改变升温过程，温度变化也会反馈改变脱水速率。

7.2 三小时窗口内的耦合演化

表 4　3 小时内药材的温度 (°C)

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
0.5	32.1896	32.3825	32.9663	33.9611	35.4133
1.0	40.3816	40.5536	41.0605	41.8782	42.9977
1.5	45.8466	45.9347	46.1928	46.6049	47.1399
2.0	48.4501	48.4879	48.5983	48.7734	49.0032
2.5	49.4669	49.4791	49.5136	49.5653	49.6609
3.0	49.8495	49.8553	49.8746	49.9101	49.9664

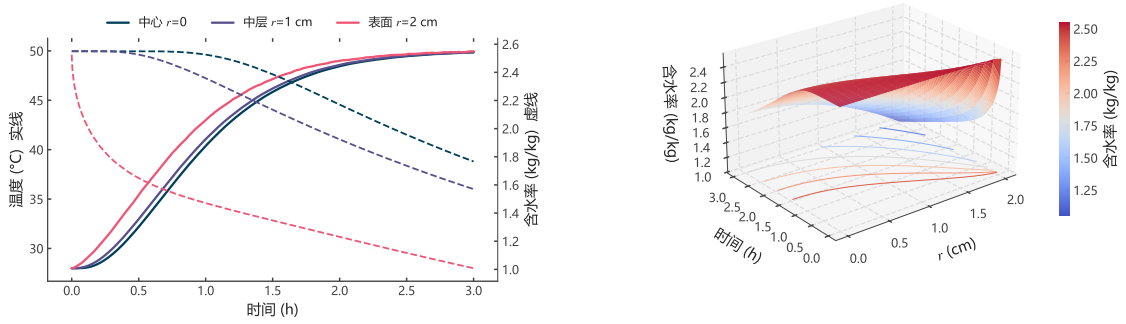
表 5　3 小时内药材的水分浓度 (kg/kg)

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
0.5	2.5499	2.5489	2.5255	2.3257	1.6486
1.0	2.5257	2.4948	2.3578	2.0230	1.4711
1.5	2.3860	2.3256	2.1344	1.8020	1.3476
2.0	2.1708	2.1084	1.9236	1.6259	1.2311
2.5	1.9566	1.9006	1.7360	1.4720	1.1166
3.0	1.7662	1.7165	1.5702	1.3333	1.0081

表 4 显示，$t = 3$ h 时中心与表面温度分别为 49.8495 °C 和 49.9664 °C，温差仅 0.117 K；温度场已在小时量级接近热风平台。

含水率场则仍在演化（表 5）：$t = 3$ h 时中心、表面与体积平均含水率分别为 1.7662、1.0081 和 1.3825 kg/kg，较初值下降 45.8%。温度已近热平衡而水分仍明显不均匀，说明脱水尚未完成，后续过程主要受内部扩散限制。

独立二维有限元复核得到的三小时中心、表面温度及含水率与上述结果保持 10^{-4} 量级一致，详见图 18 与表 9。



(a) 温度与含水率历程　　**(b) 含水率时空曲面**

图 10　问题二温度场与含水率场的耦合演化

图 10 直观给出两场的时间尺度分离：温度约 2 h 后趋于平台，含水率仍沿时间下降、沿半径递减，且最大值位于轴心。表层含水率的早期快速下降及后期减速反映了扩散系数随含水率降低而退化；轴心为最湿点是计算结果，而非终点判据的预设。

7.3 模型可延伸至全过程的构造性证据

三小时只是输出窗口。求解器将 t_{out} 作为输入，并在附件 1 覆盖区间后按假设（3）延续平台边界；问题三仅将停止条件改为式 (12) 的首达判据。以 10800 s 和 207360 s 两个上限计算时，前 10800 s 的温度和含水率逐点一致；固定 2 轮 Picard 与收敛迭代得到的体积平均含水率相对差仅为 7.51×10^{-11}。这表明长程运行没有改变短程解，且迭代只用于系数自洽而非额外时间推进。

由此，同一耦合模型可从三小时输出窗口直接延伸至完整烘干过程；问题三不再改变方程、物性或离散方法，只将固定时长输出改为含水率达标的首达时刻判定。

八、 问题三：固定几何下干燥终点的首达时刻

在问题二已验证长程可延伸性的耦合模型基础上，问题三保持控制方程、附录 3 物性、边界条件与离散内核不变，仅将固定时长积分改为终点首达检测。终点按式 (12) 定

义为全域最大含水率首次低于 0.15 kg/kg 的时刻；若误用体积平均含水率，结果将提前 21.4578 h（37.3%）。求解时记录 $\max_r C$ 及其位置，并在首次达标后续算至 $1.05\,t_{\mathrm{end}}$ 排除回吸。附件 1 覆盖结束后的热风条件按假设（3）取平台常值，该外推占最终时长的 93.04%，其影响在第 十一、 节量化。

8.1 终点时刻与全程含水率场

以 $\Delta t = 2$ s、$N = 200$ 共推进 108627 步，得到 $t_{\mathrm{end}} = 206908.1875$ s $= 57.4745$ h $= 2.3948$ d。终点处 $\max_r C = 0.149999990$，即 $g = -9.61 \times 10^{-9}$，且续算段内无回升；体积平均与表面含水率分别为 0.1242736 和 0.0526579 kg/kg。结果 2.3948 d 落在题面给出的 2–3 d 量级内。表 6 按每 6 h、每 0.5 cm 给出全过程含水率，末行即终点时刻。

表 6　烘干过程的水分浓度 (kg/kg)

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
0	2.5500	2.5500	2.5500	2.5500	2.5500
6	1.0170	0.9882	0.9016	0.7551	0.5328
12	0.4562	0.4432	0.4031	0.3298	0.1642
18	0.2989	0.2916	0.2687	0.2253	0.0845
24	0.2378	0.2327	0.2167	0.1856	0.0665
30	0.2058	0.2018	0.1892	0.1641	0.0598
36	0.1859	0.1825	0.1718	0.1504	0.0566
42	0.1721	0.1691	0.1597	0.1407	0.0549
48	0.1618	0.1592	0.1507	0.1334	0.0537
54	0.1539	0.1514	0.1436	0.1277	0.0530
烘干结束 57.4745	0.1500	0.1477	0.1402	0.1249	0.0527

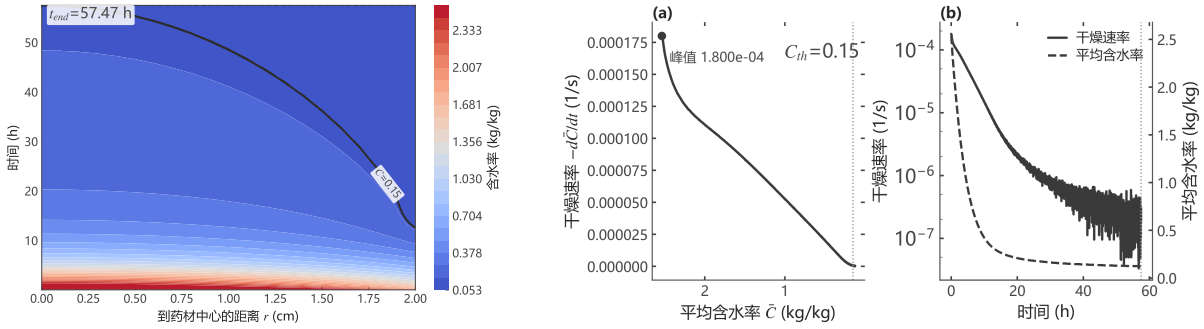
注：判据为 $\max_r C(r,t) < 0.15$ kg/kg，末行时刻即烘干结束时间。

图 11 显示，$C = 0.15$ 等值线由表面向轴心推进，轴心最后达标；后期等值线间距增大，反映扩散系数退化。干燥速率 $-\mathrm{d}\bar{C}/\mathrm{d}t$ 由 1.903×10^{-4} 降至 2.13×10^{-7} s^{-1}，全

程无恒速段。相应地，内部扩散阻力占串联阻力的比例由 54.29% 升至 95.24%，说明后期由内部扩散主导；该结论与第 十一、 节的 h_m 扰动结果一致。

水分方程残差为 1.22×10^{-11}，最大 Picard 迭代轮数为 5；终点时刻随网格加密的末档变化仅为 0.00007 h。更完整的收敛性检验见第 十、 节。

另以独立二维有限元模型作长时程趋势复核；其终点差异及时间步限制见图 18 和表 9，不替换本文首达时刻。



(a) 全程含水率时空分布　　**(b) 干燥速率曲线**

图 11　问题三的含水率演化与干燥速率

九、 问题四：ALE 动边界模型下含收缩的烘干时长

9.1 收缩为什么不能当小扰动处理

问题四要求采用附录 4 的物性并考虑附件 2 给出的实测半径收缩。附件 2 显示半径由 $R(0) = 2$ cm 单调不增地收缩到 $R(259200\ \text{s}) = 1.198$ cm，全程收缩 0.802 cm 即 40.10%，对应面积比 $(R_0/R_{\text{end}})^2 = 2.7871$。判断这一收缩是否必须显式建模，要比较边界移动速度与水分扩散速度：前者的峰值 $|\dot{R}|_{\max} = 8.389 \times 10^{-7}$ m/s 出现在初始阶段，后者的尺度为 $D/R_0 = 1.250 \times 10^{-7}$ m/s（取附录 4 在 $C = 2.55$、$T = 50$ °C 处的 $D = 2.5003 \times 10^{-9}$ m^2/s），二者之比即收缩 Péclet 数

$$Pe_{\text{shr}} = \frac{|\dot{R}|_{\max} R_0}{D} = 6.7104 > 1. \tag{14}$$

式 (14) 表明初期边界收缩比水分扩散快近七倍，动边界不是小修正，也不能用“在固定域上求解后再事后缩放”的近似替代。

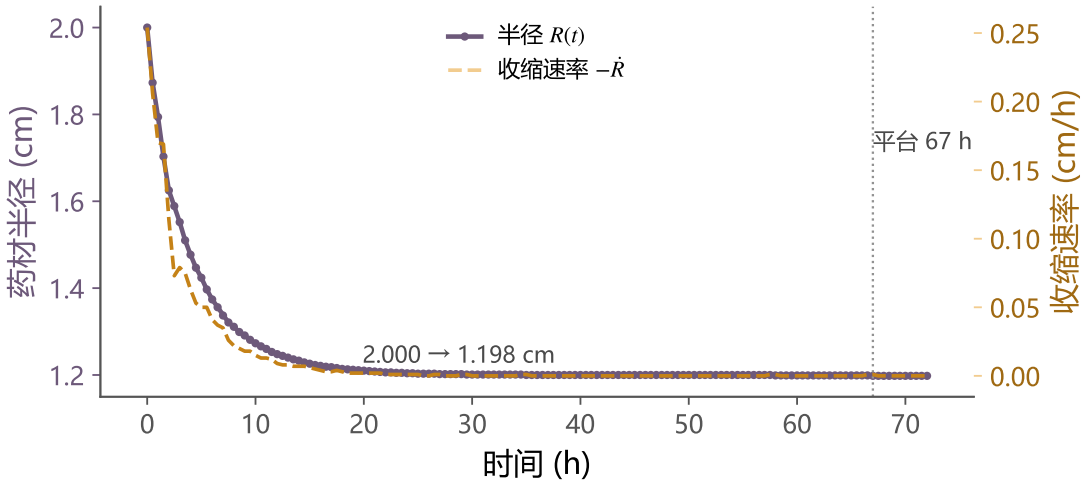


图 12　半径收缩历程与收缩速率

附件 2 的实测半径与插值结果见图 12。数据含多段平台，普通三次样条会在衔接处过冲至 1.1979 cm、低于实测下界 1.1980 cm；故本文采用保单调三次插值，并核对全程半径单调不增且位于 $[1.198, 2.000]$ cm。若终点超出附件覆盖时段，则半径取末值常值延续，避免在已进入平台后作无依据的趋势外推。

9.2 ALE 映射与运动项的恒等抵消

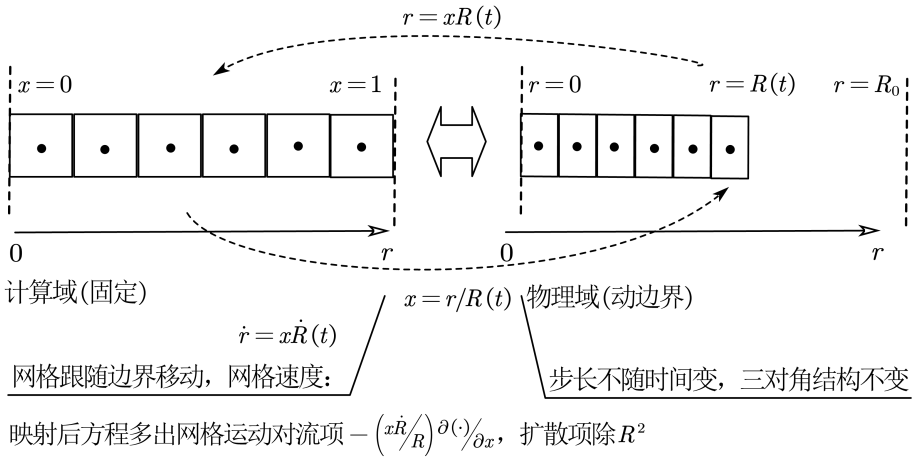


图 13　收缩域到固定域的坐标映射

本文处理动边界的方式是作坐标映射 $x = r/R(t) \in [0, 1]$（图 13），把随时间收缩的区域 $[0, R(t)]$ 变成固定区域，物理表面固定到 $x = 1$，从而可以沿用前文的固定域离散内核 [7]。固定 x 的网格点在物理空间中以速度 $x\dot{R}(t)$ 运动，由链式法则

$$\left.\frac{\partial C}{\partial t}\right|_{x} = \left.\frac{\partial C}{\partial t}\right|_{r} + \frac{x\dot{R}}{R}\frac{\partial C}{\partial x}, \tag{15}$$

式 (15) 右端第二项即网格运动项。同时，收缩意味着固体本身在运动，实验室系下的水量输运方程含固体对流项 $-v_s\partial_r C$，其中按假设（5）$v_s = r\dot{R}/R = x\dot{R}$。此时固定 x 的网格点速度恰好等于该处固体速度，即 x 是物质坐标，两项逐点相消：

$$\underbrace{\frac{x\dot{R}}{R}\frac{\partial C}{\partial x}}_{\text{网格运动}} \underbrace{- v_s\frac{\partial C}{\partial r}}_{\text{固体对流}} = \frac{x\dot{R}}{R}\frac{\partial C}{\partial x} - x\dot{R}\cdot\frac{1}{R}\frac{\partial C}{\partial x} \equiv 0. \tag{16}$$

式 (16) 是恒等式而非近似。两项在实现中分别装配后，其和的最大绝对值为 0；显式核对可避免遗漏固体对流项而造成网格运动与物质运动不一致的符号错误。

在移动控制体上直接作守恒（映射坐标下 $[x, x+\mathrm{d}x]$ 的单位长度体积为 $2\pi R^2 x\,\mathrm{d}x$，仿射收缩与干物质守恒要求其中干物质量不随时间变化，即 $\rho_d(t)R(t)^2 = \rho_{d0}R_0^2$），得到问题四的水分方程与映射后的边界条件

$$\left.\frac{\partial C}{\partial t}\right|_{x} = \frac{1}{R(t)^2}\cdot\frac{1}{x}\frac{\partial}{\partial x}\Big(x\,D\,\frac{\partial C}{\partial x}\Big), \qquad \left.-\frac{D}{R(t)}\frac{\partial C}{\partial x}\right|_{x=1} = h_m\big[C(1,t) - C_\infty(t)\big], \tag{17}$$

能量方程作同一映射，仍保留附录 4 的 ρc_p。映射域上的初边值条件、半径插值和首达判据分别沿用前文设定。式 (17) 中 ρ_d 被消去，故附件 2 与附录 4 的干物质数据不一致不进入主答案；动边界的作用集中为 $1/R(t)^2$，其由 1 增至 2.7871。

9.3 烘干时长与两个反向机制的竞争

以附录 4 物性、$\Delta t = 2$ s、$N = 200$ 共推进 96558 步，得到 $t_{\text{end}} = 183919.5625\ \text{s} = 51.0888\ \text{h} = 2.1287\ \text{d}$。此时半径已收缩到 $R = 1.2000$ cm。终点处 $\max_r C = 0.1499999990$，即 $g = -9.32\times10^{-10}$，$1.05\,t_{\text{end}}$ 内确认无回升。

问题四比问题三的 57.4745 h 短 6.3857 h。这个方向不能靠直觉预判，因为两个机制反向竞争：附录 4 在 $C = 2.55$、$T = 50\ ^\circ$C 处的扩散系数只有附录 3 的 0.1856 倍，即材料本身的输运能力低 5.39 倍，这不利于脱水；而收缩把几何因子放大 2.7871 倍，这有利于脱水。实算结果表明几何增强胜出，但胜出的幅度有限——净效应只有 11.1% 的时间缩短，远小于任一单独机制的量级。这说明用“扩散系数小所以更慢”或“收缩所以更快”的单侧直觉都会给出错误结论，符号必须由求解给出。

收缩的必要性有一个直接对照。把半径冻结在 2 cm 而其余设定不变重新计算，得到 $t_{\text{end}} = 129.8468$ h，相对 ALE 结果变化 60.65%，并且超出了本文预设的 120 h 求解上限。也就是说，若忽略收缩，附录 4 工况下的药材在五天内根本无法干燥到阈值；收缩是烘干能在两天量级完成的必要条件，而非可选修正。

把含水率场画在物理坐标下并叠加边界轨迹（图 14），可以同时看到两个过程：边界向内收缩，达标区域也向内推进。与问题三的图 11a 对比，本问的等值线不仅整体左移（达标更早），其形状也因求解域收缩而被压缩，说明收缩改变的是达标的路径而不只是缩短了路径的长度。边界轨迹与 $C = 0.15$ 等值线在末段趋于平行，对应半径进入平台后几何因子不再变化、脱水回到纯扩散控制的阶段。

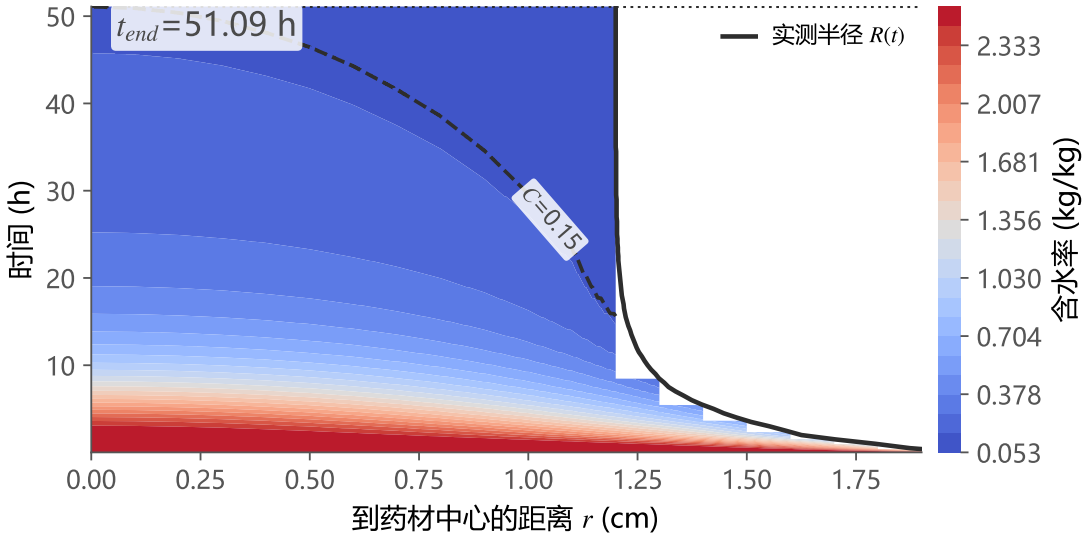


图 14　问题四动边界含水率场

表 7　含收缩的烘干过程水分浓度 (kg/kg)

时间/h	到药材中心的距离/cm				药材表面
	0	0.5	1	1.5	
0	2.5500	2.5500	2.5500	2.5500	2.5500
6	1.7188	1.5359	1.0181	–	0.4205
12	0.7374	0.6546	0.4093	–	0.1669
18	0.4087	0.3689	0.2391	–	0.0892
24	0.2851	0.2611	0.1791	–	0.0673
30	0.2263	0.2095	0.1487	–	0.0595
36	0.1928	0.1798	0.1313	–	0.0560
42	0.1712	0.1605	0.1197	–	0.0541
48	0.1561	0.1469	0.1113	–	0.0530
烘干结束 51.0888	0.1500	0.1413	0.1078	–	0.0526

注：半径由 2.0 cm 收缩至 1.2 cm；超出当前半径的网格点无药材，记 –。

表 7 单设“药材表面”列记录 $C(R(t),t)$；若固定位置满足 $r > R(t)$，则留空以表示该位置已退出药材。$r = 1.5$ cm 列自 3.64 h 起为空，直观反映了收缩。$t = 6$ h 时中心

和表面含水率分别为 1.7188 与 0.4205，比问题三同期更不均匀；终点时中心、$r = 1$ cm 和表面值仍分别为 0.1500、0.1078 和 0.0526。

收缩过程的三维快照复核见图 18：它用于检查收缩几何下的空间形态，不改变一维 ALE 连续积分得到的主结果。

9.4 质量口径对照与题面数据矛盾

为核查附件 2 的收缩数据与附录 4 密度关系，本文用干物质守恒反演半径作形式对照；该对照不替代主模型，也不构成独立求解器。若采用附录 4 密度，预测终点半径为 1.2875 cm，

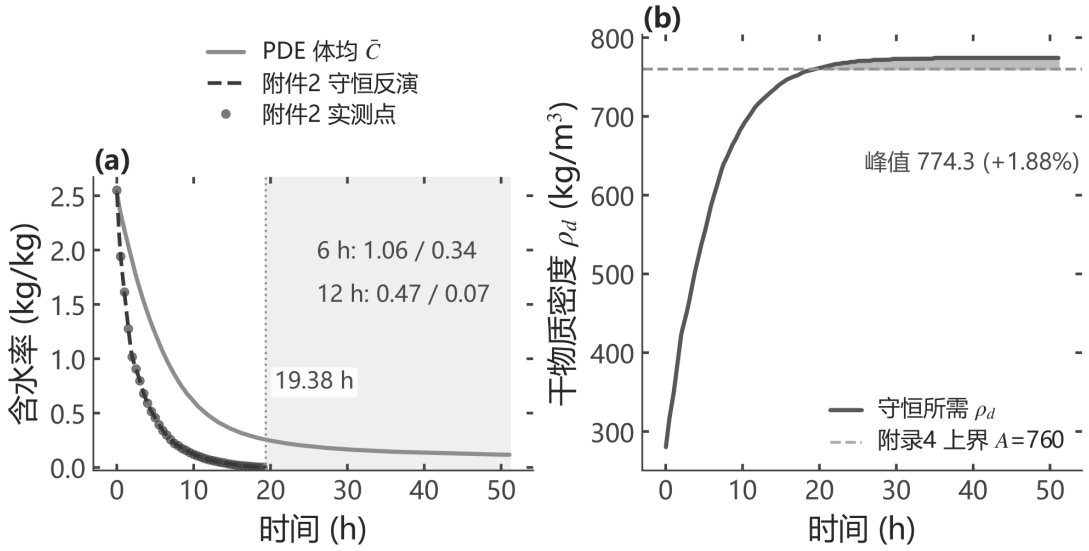


图 15　干物质守恒反演与附录四上界

而附件 2 实测为 1.198 cm，二者相差 7.4696%；反向计算得到的所需干物质密度为 776.8440 kg/m^3，高于附录 4 的 760.0000 kg/m^3 上界，缺口为 2.2163%。因此主答案仍采用 “ALE + 实测 $R(t)$”，该矛盾不进入式 (17) 的 t_{end} 计算链。还需区分半径达到 1.198 cm 的约 67.0 h 与首达干燥终点 51.0888 h；前者是几何数据时刻，后者是含水率判据得到的答案。

十、 四层误差分离的模型检验

本文按误差来源分四层逐层检验：格式实现层用常环境解析解，驱动接入层用时变边界的叠加解，离散误差层用网格与步长的收敛序列，模型误差层给出有方向的误差界。

10.1 格式实现层：常环境 Bessel 特征级数解

本层验证的是离散格式而非物理解，把附录 2 的线性温度方程置于常环境（T_∞ 取常值）下，其解可用 Bessel 特征函数级数精确表示。令 $\theta=(T-T_\infty)/(T_0-T_\infty)$，则

$$\theta(r,t)=\sum_{n=1}^{\infty}C_n J_0\!\left(\lambda_n\frac{r}{R_0}\right)e^{-\lambda_n^2 Fo},\qquad C_n=\frac{2Bi_T}{(\lambda_n^2+Bi_T^2)J_0(\lambda_n)},\tag{18}$$

其中 $F_o=\alpha t/R_0^2$，特征值 λ_n 是 $\lambda_n J_1(\lambda_n)=Bi_T J_0(\lambda_n)$ 的正根。本文以 J_0 零点序列构造括区确保不漏、重根；$Bi_T=1.3889$ 下前五个特征值为 1.4184、4.1664、7.2084、10.3082、13.4271。级数校验取初始条件展开的完备性，但应核对能量型系数 $B_n=4Bi_T^2/[\lambda_n^2(\lambda_n^2+Bi_T^2)]$ 之和而非 C_n 之和：40 项时 $\sum B_n=0.99999$，而 $\sum C_n$ 40 项仅 0.99875、150 项也仅 0.99982，收敛慢得多，据此判截断会误判级数未收敛。

10.2 驱动接入层：Duhamel 叠加

本题的风温随时间变化，对线性方程可用 Duhamel 原理构造独立解析解：

$$T(r,t)=T_0+\int_0^t\frac{\mathrm{d}T_\infty}{\mathrm{d}s}\,\Psi(r,t-s)\,\mathrm{d}s,\tag{19}$$

其中 Ψ 为单位阶跃风温下的响应函数。式 (19) 取 150 项级数、以 $\Delta s=0.05$ s 的梯形公式积分，其结果检验的正是常环境解析解无法覆盖的时变边界驱动路径。

表 8　解析解对照误差

对照解	对照量	数值解	解析解	绝对误差
Bessel 特征级数 (常环境)	中心温度 (°C)	43.9626	43.9631	4.91×10^{-4}
Bessel 特征级数 (常环境)	表面温度 (°C)	46.6383	46.6385	2.70×10^{-4}
Duhamel 叠加 (时变环境)	r=0 cm	33.5756	33.5753	2.98×10^{-4}
Duhamel 叠加 (时变环境)	r=1 cm	34.3644	34.3642	2.78×10^{-4}
Duhamel 叠加 (时变环境)	r=2 cm	36.7857	36.7856	1.76×10^{-4}

注：Bessel 全场最大误差 4.91×10^{-4} K；Duhamel 最大误差 2.98×10^{-4} K。

解析对照结果于表 8。常环境下中心与表面温度的绝对误差与时变情形下三个位置的误差都在 10^{-4} K 量级，比本题温度物理变化范围小五个数量级。

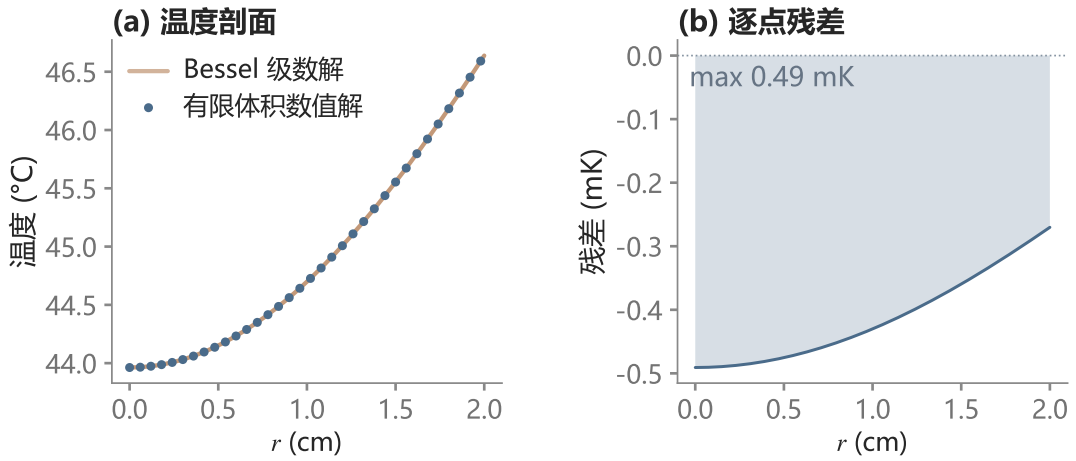


图 16　解析解对照与残差

图 16 中数值解与级数解重合，残差最大约 0.49 mK，沿径向无系统性空间结构、不集中于表面，且随时间不累积，表明表面边界离散与全隐式推进均未引入局部或时间累积偏差。表面离散形式方面最大误差稳定在 0.49 mK，验证时变 Robin 边界接入正确。

10.3 离散误差层：网格与步长收敛

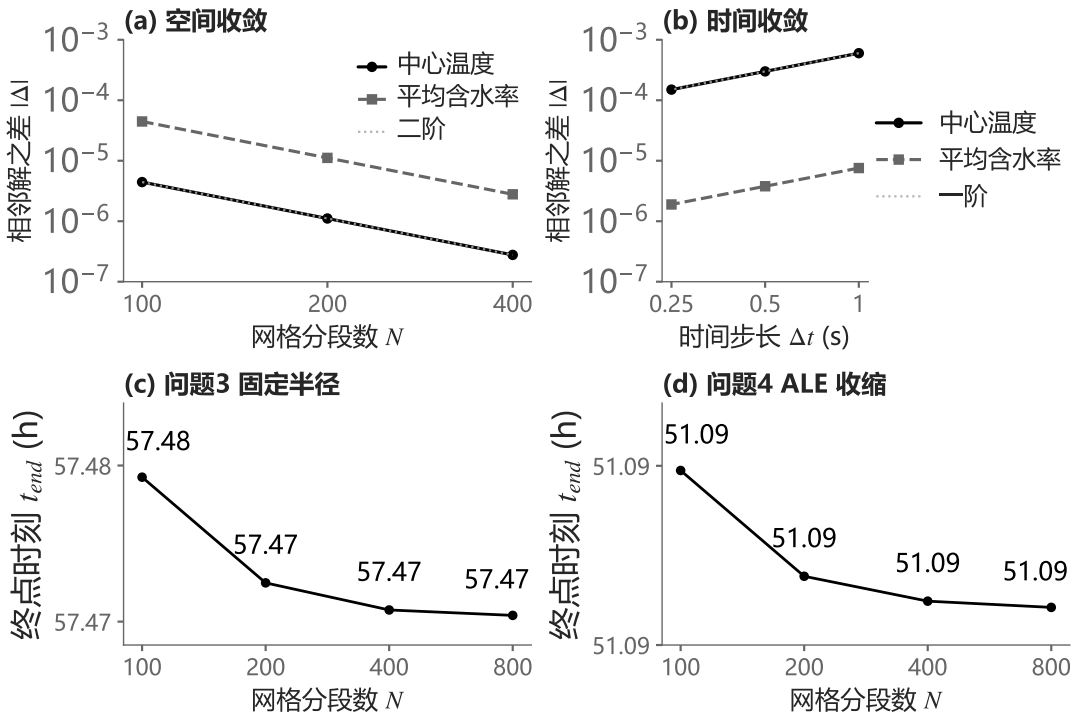


图 17　网格与时间步收敛性检验

四档网格与四档步长上的收敛结果如图 17。两方向收敛趋势斜率分别接近 2 与 1。$N = 200$ 与 $\Delta t = 0.25$ s 组合已落在两曲线平坦段之前说明主结果离散参数处于收敛区

间内部。

10.4 模型误差层：三项有方向的误差界

本文识别出三项可判定方向的模型误差。一是忽略端面散失与轴向不均匀会使脱水路径被高估。二是能量方程不含蒸发潜热汇使固体温度偏高会高估扩散系数。量级核算显示，按体积平均脱水速率折算的潜热功率约 1347 W/m^2，表面显热输入上限为 $h\Delta T_{\max} = 556$ W/m^2，比值 2.42，说明潜热项并非可忽略的小量。三是附件 2 与附录 4 之间的不互洽，其在密度方向上的量度为第 九、 节所定义的干物质密度缺口 2.2163%；该项影响已因 ρ_d 在式 (17) 中被消去而不进入终点时刻。前两项的方向相反，因而部分抵消；但抵消的幅度无法从现有信息确定，故本文只声明真值落在两方向界之间。

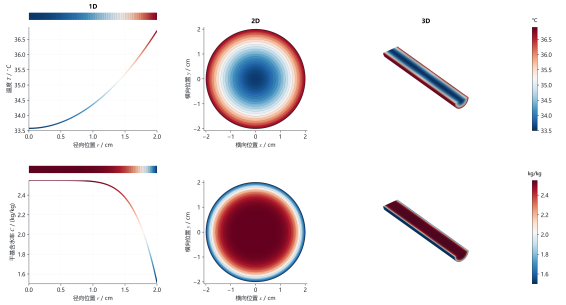
10.5 独立有限元与三维快照复核

前述四层检验分别针对格式、驱动、离散误差与模型边界。为进一步检查不同空间离散实现是否保持同一物理趋势，本文补充四组 ANSYS 计算；它们是数值复核，不是实验验证，也不替代前述解析解与收敛性证据。

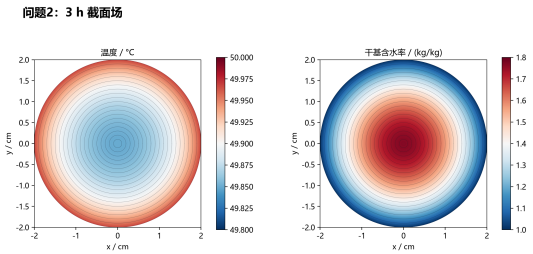
表 9　ANSYS 数值复核摘要

项目	对应工况	主要结果	证据边界
S1	问题一，$t = 1800$ s	一维、二维、三维同条件对照；一维–三维最大差为 0.0044 K（温度）和 0.00535 kg/kg（含水率）	空间维度与实现一致性，不是实验验证
S2	问题二，$t = 3$ h	精细有限元得 $T_c/T_s = 49.8494/49.9664$ °C，$C_c/C_s = 1.76624/1.00814$；与正文结果相差约 10^{-4} 量级	滞后系数双向耦合，不能表述为完全收敛的非线性独立解
S3	问题三，固定半径长时程	首次低于阈值约 57.4445 h，与正文 57.4745 h 相差约 0.052%	$\Delta t = 20$ s、60 s 输出并线性插值，属于条件性复核
S4	问题四，收缩状态快照	0–51.0888 h 七状态中半径约由 2.000 cm 收缩至 1.200 cm，中心保持为高含水区	从一维 ALE 轨迹抽取状态后局部推进，不是连续三维动边界积分

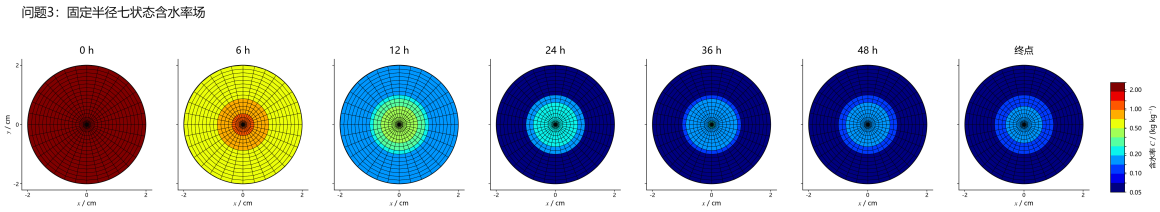
图 18 汇总四组代表性场图。S1 和 S2 主要支持温湿场数值实现的交叉检查；S3 只支持长时程趋势一致性，终点仍以本文的高分辨率首达计算为准；S4 则补充收缩几何下的空间形态信息。由于 S1、S2 的有限元计算仍采用等效传质热类比，且各组计算保留了原始警告记录，本文不把“计算完成”升级为物理真实性或实验精度的结论。



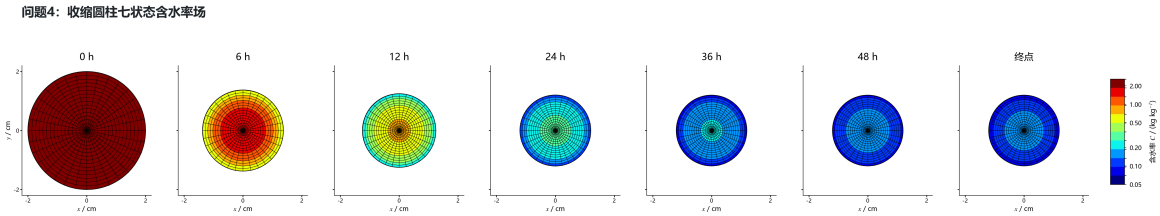
(a) S1：1800 s；左至右为一维、二维、三维



(b) S2：3 h；温度与含水率截面



(c) S3：固定 $R = 2$ cm；七时刻及终点



(d) S4：R 由 2.0 cm 收缩至 1.2 cm；七状态

图 18　四组 ANSYS 数值复核的代表性场图。子图内仅保留必要的坐标、时间和色标；工况说明已写入子图图名。各图结果用于检验数值实现与空间形态，不等同于实验验证。

十一、 关键假设与形式选择的灵敏度分析

灵敏度检验量统一取终点时刻 t_{end}。为控制计算量，扫描采用 $\Delta t = 4$ s，此时基准值问题三、四分别为 57.476233 h、51.090087 h，相对主结果偏移 $+0.0030\%$ 与 $+0.0026\%$，远小于任何被扫参数的效应，故排序不受步长选择影响。

11.1 空气条件外推：本题最大的数据缺口

由第 八、 节，外推段占问题三总时长的 93.04%，因此假设（3）是两问结论的主导假设。扫描取 $T_\infty \in \{49, 50, 51\}$ °C 与 $C_\infty \in \{0.045, 0.05, 0.055\}$ 的九个组合，只改动 $t > 14400$ s 的平台值，实测段始终使用附件 1 的原始数据。九个组合全部达标，问题三、四相对变化跨度分别为 6.7060% 与 6.7127%，单侧最大偏离分别为 3.4383% 与 3.4539%。

两个预期方向都得到证实：风温升高使终点时刻缩短，风的含水率升高使终点时刻延长。前者的量级可以独立核对——由 Arrhenius 因子，$\mathrm{d}\ln D/\mathrm{d}T = 3850/T_K^2$，在 50 °C 处即 3.6868% 每开尔文，与观察到的 ±1 K 对应 6.36% 跨度（单侧约 3.2%）量级一致，说明风温的影响主要通过扩散系数而非通过温差驱动传递。另需指出一条硬边界：当 $C_\infty \geq 0.15$ 时表面驱动力在含水率降到阈值前即消失，终点永不可达。本扫描的上界 0.055 远低于该边界，但这一条件是问题三、四有解的前提，程序在启动时即予核对。

11.2 表面输运系数：表面控制还是内部控制

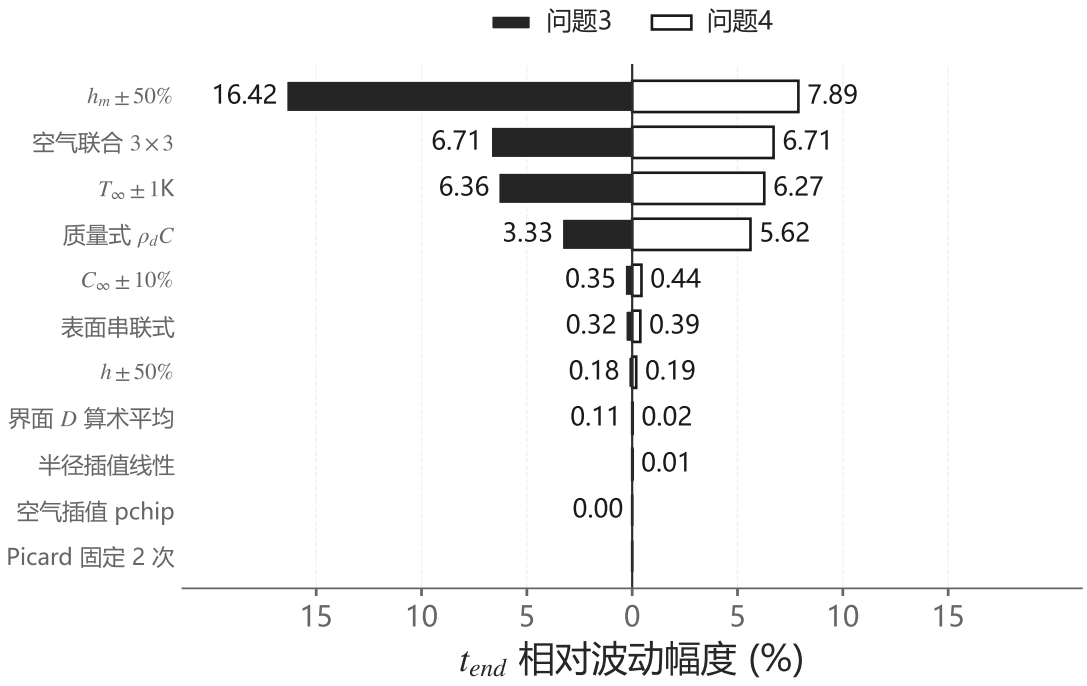


图 19　烘干时长敏感性排序

全部扰动的效应汇总于图 19。可以看出传质系数 h_m 上下浮动的影响是全表跨度首位，是传热系数 h 浮动跨度的近百倍，这直接反映了温度场早已追上热风，传热系数的变化只影响升温快慢而几乎不影响脱水。h_m 虽在跨度上居首位，但其效应远小于纯表面控制的预期。若脱水完全由表面传质限制，h_m 提高 50% 应使终点时刻缩短 33.33%；实测只缩短 3.4503%，表面控制指数仅 0.1035。这在工艺上意味着提高风速的边际效果有限，而提高风温或降低风的含水率更为有效。

11.3 数值与形式选择的稳健性

图 19的下半部分中除水分方程形式一项外都在 0.4% 以下，说明主结果不依赖任何具体的实现选择。同时水分方程形式一项的三个百分点说明 ρ 是常数但 C 沿半径不均匀，ρ_d 在 $C \in [0.15, 2.55]$ 上取值 230.99 至 713.04 $\mathrm{kg/m^3}$，跨度 3.09 倍，远非常数。真正

的等价条件是 ρ_d 在空间上均匀。这一修正后的判断经四组设置验证：当系数取在均匀初值上时，两形式的含水率之差仅为 1.33×10^{-15}；随时间推进，对应的差随非均匀度单调增长且在均匀极限下恒等到机器精度，说明图中的百分数是假设（4）在本题定义下的真实效应量，不是实现误差。

十二、 模型评价与推广

12.1 四问结果的横向对照

表 10　四问结果汇总

问题	物性与几何设定	时长	关键结果	机理归因
问题 1	附录 2 常物性 / 解耦	1800 s	中心 33.58 °C，表面 36.79 °C	预热为主，脱水仅表层
问题 2	附录 3 变物性 / 双向耦合	3 h	平均含水率 1.3825 kg/kg	温度已趋近热风温度
问题 3	附录 3 / 固定半径	57.4745 h	终点最大含水率 0.1500 kg/kg	表面传质与退化扩散共同限速
问题 4	附录 4 / ALE 收缩	51.0888 h	半径收缩至 1.2 cm	收缩缩短行程，抵过 D 减小

注：问题 4 与问题 3 的时长比为 0.8889；冻结半径时问题 4 时长为 129.8468 h，相对 ALE 收缩模型增加 60.65%。

四问的设定、时长与机理归因并列于表 10。四问共用同一套控制方程与同一套离散内核，差别只在物性组、几何是否运动与终止条件，因此表中时长的差异可以直接归因到机理，而不需要担心实现差异带来的混淆。我们还可以发现，问题四与三的时间比为 0.888895：附录 4 扩散系数仅及附录 3 的 0.1856 倍，但收缩把几何因子放大 2.7871 倍，净效应快 11.1%。冻结半径对照相对 ALE 变化 60.65%，说明收缩是烘干在两天量级完成的必要条件。

12.2 模型的优点

第一，终点判据取全域上确界并辅以事件检测：题面要求整根药材达标，判据须为空间上确界，本文逐步记录 $\arg\max_r C$ 核对，实测内部隆起高度仅 4.88×10^{-15}；得终

点时刻后继续积分至 $1.05, t_{\mathrm{end}}$ 确认无回升，排除先达标又回吸的伪首达。

第二，对退化扩散的正确离散。以 Kirchhoff 势差商 (9) 吸收退化性，使离散对 Φ 成标准线性格式；而调和平均在 $C: 0.05 \to 0.03$ 区间低估通量 46.59 倍，恰落在决定终点时刻的一段。

第三，问题一的解耦性作为推论被证明、而非作为简化被假设。附录 2 物性使两方程不交换状态量，本文以求解顺序互换后逐点偏差为 0 作为该推论的离散证据。

第四，ALE 中网格运动项与固体对流项显式保留，其恒等抵消经解析推导与逐点核对确认：两项分别装配后核对其和为 0，且因 ρ_d 在式 (17) 中被完全消去，按第 九、 节的半径预测偏差 7.4696% 和干物质密度缺口 2.2163% 计量，都不进入终点时刻计算链。

第五，四问共用同一求解内核，使四层检验一次完成即全局有效、四问差异必来自机理，此结构亦使模型天然可延伸。

12.3 模型的局限

首要局限是空气条件外推。附件 1 仅覆盖前 14400 s，而问题三、四的终点在 2×10^5 s 量级，93.04% 的时长依赖平台常值假设。其次是能量方程不含蒸发潜热汇。本文以题设风温与传热系数直接驱动固相温度，略去它高估固体温度并经 Arrhenius 因子高估扩散系数，故终点时刻在此项上偏小。与之相反，一维降维忽略端面散失使脱水路径被高估、终点时刻偏大。两项部分抵消，但幅度无法由现有信息确定，故本文只给双向界。第三是表面输运系数取常值。h 与 h_m 四问沿用同一取值，但收缩使表面积减小 40%，边界层厚度与流动状态随之变化；灵敏度检验显示 h_m 在 $\pm 50\%$ 区间造成 16.42% 跨度，此项改进需风速与表面几何的额外信息。第四是收缩形式取仿射。假设（5）令 $v_s = r\dot{R}/R$，使映射坐标成为物质坐标并给出式 (16) 的恒等抵消；本文对不均匀收缩有所考虑，但其详细过程的确定需引入固体变形的本构关系。

12.4 推广方向

本模型可直接推广为三类问题。其一反问题：给定合格判据与设备能力，求最省能耗的风温风湿轨迹，式 (12) 的终点时刻由输出转为约束，决策变量为 $T_\infty(t)$ 与 $C_\infty(t)$ 的分段常值序列。其二几何推广：将一维径向扩为二维轴对称，即可处理短圆柱与端面效应；体积权与 Kirchhoff 界面系数形式不变，仅把界面面积权改为对应面元。其三参数辨识：以实测失重曲线为观测、本模型为正演算子，反演 D 的两个参数，从而摆脱对经验公式的依赖。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于代码调试、术语翻译与校对、语言润色、排版检查、参考文献格式整理，详细使用情况见支撑材料。

参考文献

[1] LI X, WANG Y, MA X, et al. Chinese medicinal materials' drying technologies advancements: principles, energy performance, and influence on the bioactive components[J/OL]. Drying Technology, 2024, 42(12): 1815-1845. DOI: 10.1080/07373937.2024.2394125.

[2] FENG Y, RUAN M, WANG M, et al. Developing an effective pretreatment process for hot-air drying of chinese yam (Dioscorea opposita thunb.) slices: drying characteristics, efficiency, and quality[J/OL]. Food Chemistry, 2025, 493: 146018. DOI: 10.1016/j.foodchem.2025.146018.

[3] MIGNONE A. High-order conservative reconstruction schemes for finite volume methods in cylindrical and spherical coordinates[J/OL]. Journal of Computational Physics, 2014, 270: 784-814. DOI: 10.1016/j.jcp.2014.04.001.

[4] ÇELIK E, PARLAK N, MERAL Z K, et al. Experimental investigation of convective drying behavior of in-shell hazelnuts: moisture diffusivity, transport phenomena, and pressure drop [J/OL]. Heat and Mass Transfer, 2026, 62(8). DOI: 10.1007/s00231-026-03715-3.

[5] ZHANG L, KONG H, ZHENG H. Numerical manifold method for steady-state nonlinear heat conduction using kirchhoff transformation[J/OL]. Science China Technological Sciences, 2023, 67(4): 992-1006. DOI: 10.1007/s11431-022-2389-8.

[6] ZHANG Y, CUI X, YUAN G. Nonlinear iteration acceleration solution for equilibrium radiation diffusion equation[J/OL]. ESAIM: Mathematical Modelling and Numerical Analysis, 2020, 54(5): 1465-1490. DOI: 10.1051/m2an/2019095.

[7] BUSTO S, DUMBSER M, RÍO-MARTÍN L. An arbitrary-lagrangian-eulerian hybrid finite volume/finite element method on moving unstructured meshes for the navier-stokes equations[J/OL]. Applied Mathematics and Computation, 2023, 437: 127539. DOI: 10.1016/j.amc.2022.127539.

附录 A　程序源代码

全部结果由以下 Python 源码计算产生，入口为 `main.py`。模块划分遵循一条原则：三组附录物性各自封装、互不引用，唯一的求解器 `solve_drying` 被四个子问题共用，因此四问结果的差异只可能来自物性组、几何设定与终止条件，不可能来自不同的数值实现。运行环境为 Python 3 与 NumPy、SciPy、pandas、openpyxl。

表 11　源码文件清单

文件	职责	行数
`params.py`	几何、初值、系数与全部计算约定的单一来源	160
`props.py`	三组附录物性与扩散系数，含单位区间核对	141
`utils.py`	唯一求解器、Kirchhoff 势、ALE 装配、首达检测	679
`problem1.py`	问题一：解耦场求解与换序核对	193
`problem2.py`	问题二：双向耦合与长短程一致性核对	195
`problem3.py`	问题三：首达时刻与判据对照	214
`problem4.py`	问题四：ALE 主模型与拉格朗日对照模型	352
`global_q4_analysis.py`	问题四：非均匀收缩下的全局不确定性分析	554
`structural_tests.py`	八项结构性检验（守恒、解析极限、单位探针）	240
`sensitivity_analysis.py`	灵敏度与形式选择扫描	430
`main.py`	入口，按序驱动各问并复核关键判据	244

1.1　参数与计算约定

```

from __future__ import annotations

import os
from pathlib import Path

CODE_DIR = Path(__file__).resolve().parent
# 可用 MODEL_ROOT 指定数据目录。
_root_override = os.environ.get("MODEL_ROOT")
if _root_override:
```

```
   ROOT = Path(_root_override).expanduser().resolve()
else:
   _root_candidates = (CODE_DIR, CODE_DIR.parent)
   ROOT = next(
      (
         candidate
         for candidate in _root_candidates
         if (candidate / "user_data").exists()
         or (candidate / "figures").exists()
      ),
      CODE_DIR,
   )
USER_DATA = ROOT / "user_data"
FIGDIR = ROOT / "figures"
TMPDIR = ROOT / "_tmp"
USER_DATA.mkdir(parents=True, exist_ok=True)
FIGDIR.mkdir(parents=True, exist_ok=True)
TMPDIR.mkdir(parents=True, exist_ok=True)

KELVIN_OFFSET = 273.15
INDEX_HEADER = "时间\\到药材中心的距离"


R0 = 0.02
R0_CM = 2.0
LENGTH_CM = 4.0
HALF_LENGTH_M = LENGTH_CM / 100.0 / 2.0
T0_C = 28.0
C0 = 2.55
C_THRESHOLD = 0.15
DECIMALS = 4
DURATION_DAYS_LOW = 2.0
DURATION_DAYS_HIGH = 3.0


# 附录 2
RHO_APP2 = 820.0
CP_APP2 = 2600.0
K_APP2 = 0.36
H_CONV = 25.0
HM_MASS = 8.0e-7
D0_APP2 = 7.0e-9
A_APP2 = 0.89


RHO_A3, RHO_B3 = 650.0, 128.0
CP_A3, CP_B3 = 1450.0, 2736.0
```

```
K_A3, K_B3 = 0.21, 0.38
D0_APP3 = 2.4e-3
A_APP3 = 0.45
B_ARRH3 = 3850.0


RHO_A4, RHO_B4 = 760.0, 90.0
CP_A4, CP_B4 = 1850.0, 2150.0
K_A4, K_B4 = 0.12, 0.20
D0_APP4 = 4.2e-4
A_APP4 = 0.30
B_ARRH4 = 3850.0


# 输出时刻和空间列
Q1_TABLE_TIMES_S = [100.0, 300.0, 600.0, 900.0, 1200.0, 1500.0, 1800.0]
Q1_TABLE_RADII_CM = [0.0, 0.5, 1.0, 1.5, 2.0]
Q1_SPAN_S = 1800.0
Q1_DT_OUT = 1.0
Q2_TABLE_TIMES_H = [0.5, 1.0, 1.5, 2.0, 2.5, 3.0]
Q2_TABLE_RADII_CM = [0.0, 0.5, 1.0, 1.5, 2.0]
Q2_SPAN_S = 10800.0
Q2_DT_OUT = 1.0
Q3_DT_OUT = 60.0
Q3_TABLE_DT_H = 6.0
Q4_DT_OUT = 60.0
Q4_TABLE_DT_H = 6.0
DR_OUT_CM = 0.1
SHEETS_TC = ["温度", "水分浓度"]
SURFACE_COL_NAME = "药材表面"


N_CELLS = 200
DR_SOLVE = R0 / N_CELLS
OUT_STRIDE_SPACE = int(round((DR_OUT_CM / 100.0) / DR_SOLVE))
# 主计算步长
DT_SOLVE_Q12 = 0.25
DT_SOLVE_Q34 = 2.0
PICARD_TOL_C = 1.0e-10
PICARD_TOL_T = 1.0e-8
PICARD_MAX = 50
N_CONVERGENCE = (100, 200, 400, 800)
DT_CONVERGENCE_Q12 = (1.0, 0.5, 0.25, 0.125)
DT_CONVERGENCE_Q34 = (8.0, 4.0, 2.0, 1.0)
T_MAX_CAP_S = 120.0 * 3600.0
BISECT_TOL_S = 0.2
REBOUND_FACTOR = 1.05
```

```

INCLUDE_LATENT = False
MASS_FORM = "C"
AIR_EXTRAP_MODE = "hold"
AIR_EXTRAP_T_C = 50.0
AIR_EXTRAP_C = 0.05


SENS_T_AIR_C = (49.0, 50.0, 51.0)
SENS_C_AIR = (0.045, 0.05, 0.055)
SENS_H_FACTORS = (0.5, 1.0, 1.5)
SENS_HM_FACTORS = (0.5, 1.0, 1.5)
SENS_DT_SOLVE_S = 4.0
SENS_DT_OUT_S = 3600.0


AIR_XLSX = USER_DATA / "附件1.xlsx"
RADIUS_XLSX = USER_DATA / "附件2.xlsx"
# 输入表头
AIR_COLS = ["时间", "温度", "水分浓度"]
RADIUS_COLS = ["时间", "半径"]
AIR_T_END_S = 14400.0
RADIUS_T_END_S = 259200.0


FACTS = {
   "geometry": {"radius_cm": R0_CM, "length_cm": LENGTH_CM},
   "initial_conditions": {"temperature_C": T0_C, "moisture_kgkg": C0},
   "criteria": {"moisture_threshold_kgkg": C_THRESHOLD, "decimals": DECIMALS},
}
DATA_FACTS = {
   "air_file": AIR_XLSX.name,
   "radius_file": RADIUS_XLSX.name,
   "air_measured_until_s": AIR_T_END_S,
   "radius_measured_until_s": RADIUS_T_END_S,
}

assert abs(R0 - 0.02) < 1.0e-15
assert abs(C_THRESHOLD - 0.15) < 1.0e-15
assert abs(T0_C - 28.0) < 1.0e-12
assert abs(D0_APP2 - 7.0e-9) < 1.0e-24
assert abs(D0_APP3 - 2.4e-3) < 1.0e-18
assert abs(D0_APP4 - 4.2e-4) < 1.0e-19
assert OUT_STRIDE_SPACE == 10
assert abs(DR_SOLVE - 1.0e-4) < 1.0e-18
assert AIR_EXTRAP_C < C_THRESHOLD
```

```
for _out_dt, _solve_dt in (
   (Q1_DT_OUT, DT_SOLVE_Q12),
   (Q2_DT_OUT, DT_SOLVE_Q12),
   (Q3_DT_OUT, DT_SOLVE_Q34),
   (Q4_DT_OUT, DT_SOLVE_Q34),
):
   assert abs(_out_dt / _solve_dt - round(_out_dt / _solve_dt)) < 1.0e-12

__all__ = [
   name for name in dir()
   if name.isupper() or name in ("FACTS", "DATA_FACTS", "ROOT", "USER_DATA", "FIGDIR",
       "TMPDIR")
]
```

1.2　三组附录物性

```

from __future__ import annotations

import numpy as np
from scipy.special import exp1

from params import (A_APP2, A_APP3, A_APP4, B_ARRH3, B_ARRH4, CP_A3, CP_A4, CP_APP2,
               CP_B3, CP_B4, D0_APP2, D0_APP3, D0_APP4, K_A3, K_A4, K_APP2,
               K_B3, K_B4, KELVIN_OFFSET, RHO_A3, RHO_A4, RHO_APP2, RHO_B3, RHO_B4)

T_K_LO, T_K_HI = 250.0, 400.0
ARRH_LO, ARRH_HI = 1e-7, 1e-5


class UnitError(ValueError):
   # 温度单位错时不要继续计算。
   pass


def to_kelvin(T_c):
   return np.asarray(T_c, dtype=float) + KELVIN_OFFSET


def _guard_kelvin(T_K):
   T_K = np.asarray(T_K, dtype=float)
   lo, hi = float(np.min(T_K)), float(np.max(T_K))
   if not (T_K_LO < lo and hi < T_K_HI):
      raise UnitError(
         f"温度实参必须为开尔文且落在 ({T_K_LO},{T_K_HI})，实测 [{lo:.4f},{hi:.4f}]；"
```

```
          "若传入摄氏度，exp(-3850/T) 会退化到 1e-34 量级并使 D 归零"
      )
   return T_K


def arrhenius(T_K, b):
   T_K = _guard_kelvin(T_K)
   f = np.exp(-b / T_K)
   fmin, fmax = float(np.min(f)), float(np.max(f))
   if not (ARRH_LO <= fmin and fmax <= ARRH_HI):
      raise UnitError(f"exp(-{b}/T_K) 量级异常: [{fmin:.6e},{fmax:.6e}] 越出
          [{ARRH_LO:g},{ARRH_HI:g}]")
   return f


# 附录 2
def props_app2():
   return RHO_APP2, CP_APP2, K_APP2


def D_app2(C):
   C = np.asarray(C, dtype=float)
   return D0_APP2 * np.exp(-A_APP2 / np.maximum(C, 1e-300))


# 附录 3
def props_app3(C):
   C = np.asarray(C, dtype=float)
   w = C / (C + 1.0)
   rho = RHO_A3 + RHO_B3 * C
   cp = CP_A3 + CP_B3 * w
   k = K_A3 + K_B3 * w
   return rho, cp, k


def D_app3(C, T_K):
   C = np.asarray(C, dtype=float)
   return D0_APP3 * np.exp(-A_APP3 / np.maximum(C, 1e-300)) * arrhenius(T_K, B_ARRH3)


# 附录 4
def props_app4(C):
   C = np.asarray(C, dtype=float)
   w = C / (C + 1.0)
   rho = RHO_A4 + RHO_B4 * C
   cp = CP_A4 + CP_B4 * w
   k = K_A4 + K_B4 * w
```

```
    return rho, cp, k


def D_app4(C, T_K):
    C = np.asarray(C, dtype=float)
    return D0_APP4 * np.exp(-A_APP4 / np.maximum(C, 1e-300)) * arrhenius(T_K, B_ARRH4)


# Kirchhoff 势
def _phi_shape(C, a):
    C = np.asarray(C, dtype=float)
    out = np.zeros_like(C)
    m = C > 0.0
    if np.any(m):
        z = a / C[m]
        out[m] = C[m] * np.exp(-z) - a * exp1(z)
    return out


def Phi_app2(C):
    return D0_APP2 * _phi_shape(C, A_APP2)


def Phi_app3(C, fT):
    return D0_APP3 * _phi_shape(C, A_APP3) * np.asarray(fT, dtype=float)


def Phi_app4(C, fT):
    return D0_APP4 * _phi_shape(C, A_APP4) * np.asarray(fT, dtype=float)


PROPERTY_SETS = {
    "app2": {"props": props_app2, "D": D_app2, "Phi": Phi_app2, "a": A_APP2, "D0": D0_APP2,
           "b": None, "coupling": "decoupled"},
    "app3": {"props": props_app3, "D": D_app3, "Phi": Phi_app3, "a": A_APP3, "D0": D0_APP3,
           "b": B_ARRH3, "coupling": "two_way"},
    "app4": {"props": props_app4, "D": D_app4, "Phi": Phi_app4, "a": A_APP4, "D0": D0_APP4,
           "b": B_ARRH4, "coupling": "two_way"},
}


if __name__ == "__main__":
    import inspect
    from params import C0, KELVIN_OFFSET as KO
    print("D_app2 signature:", inspect.signature(D_app2))
    assert "T" not in str(inspect.signature(D_app2)), "C-P1-2: D_app2 签名不得含 T"
    print(f"props_app2 = {props_app2()}")
```

```
    print(f"D_app2({C0}) = {float(D_app2(C0)):.6e}")
    r3, c3, k3 = props_app3(C0)
    r4, c4, k4 = props_app4(C0)
    print(f"app3 @C0: rho={float(r3):.4f} cp={float(c3):.4f} k={float(k3):.7f}")
    print(f"app4 @C0: rho={float(r4):.4f} cp={float(c4):.4f} k={float(k4):.7f}")
    TK50 = 50.0 + KO
    d3 = float(D_app3(C0, TK50)); d4 = float(D_app4(C0, TK50))
    print(f"D_app3(C0,323.15)={d3:.6e} D_app4={d4:.6e} ratio={d4 / d3:.6f}")
    print("props self-check OK")
```

1.3 求解器内核

```

from __future__ import annotations

import numpy as np
from scipy.interpolate import PchipInterpolator
from scipy.linalg import solve_banded
from scipy.optimize import brentq
from scipy.special import j0, j1, jn_zeros

import pandas as pd

from params import (AIR_COLS, AIR_EXTRAP_C, AIR_EXTRAP_T_C, AIR_T_END_S, AIR_XLSX,
                BISECT_TOL_S, C0, C_THRESHOLD, DECIMALS, H_CONV, HM_MASS,
                INDEX_HEADER, KELVIN_OFFSET, N_CELLS, PICARD_MAX, PICARD_TOL_C,
                PICARD_TOL_T, R0, RADIUS_COLS, RADIUS_T_END_S, RADIUS_XLSX,
                REBOUND_FACTOR, T0_C, T_MAX_CAP_S)
from props import PROPERTY_SETS, arrhenius, to_kelvin


# 网格权重
def grid_weights(R, N):
    dr = R / N
    r = np.arange(N + 1, dtype=float) * dr
    V = np.empty(N + 1, dtype=float)
    V[0] = dr ** 2 / 8
    V[1:N] = r[1:N] * dr
    V[-1] = (dr / 2) * (R - dr / 4)
    A_in = (np.arange(N, dtype=float) + 0.5) * dr
    return r, dr, V, A_in


def check_geometry_identity(R, N):
    _, _, V, _ = grid_weights(R, N)
```

```
    target = R ** 2 / 2
    return abs(V.sum() - target) / target


def volume_average(C, V):
    return float(np.dot(V, C) / V.sum())


# 界面扩散系数
def D_face(C, pset, T_face_K=None, mode="kirchhoff"):
    Cl, Cr = C[:-1], C[1:]
    if mode == "arithmetic":
        if pset["b"] is None:
            return np.maximum(0.5 * (pset["D"](Cl) + pset["D"](Cr)), 0.0)
        return np.maximum(0.5 * (pset["D"](Cl, T_face_K) + pset["D"](Cr, T_face_K)), 0.0)
    if mode != "kirchhoff":
        raise ValueError(f"未知界面 D 取法 {mode!r}，只允许 'kirchhoff' / 'arithmetic'")
    if pset["b"] is None:
        Phi_l = pset["Phi"](Cl)
        Phi_r = pset["Phi"](Cr)
        D_l = pset["D"](Cl)
    else:
        fT = arrhenius(T_face_K, pset["b"])
        Phi_l = pset["Phi"](Cl, fT)
        Phi_r = pset["Phi"](Cr, fT)
        D_l = pset["D"](Cl, T_face_K)
    dC = Cr - Cl
    out = np.where(np.abs(dC) > 0.0, (Phi_r - Phi_l) / np.where(dC == 0.0, 1.0, dC), D_l)
    return np.maximum(out, 0.0)


def k_face(k_nodes):
    return (k_nodes[:-1] + k_nodes[1:]) * 0.5


def U_eff_series(D_surf, half_thickness, h_ext):
    D_surf = np.maximum(np.asarray(D_surf, dtype=float), 1e-300)
    return 1.0 / (half_thickness / D_surf + 1.0 / h_ext)


def U_surface(Dif_surf, half_thickness, h_ext, form="node"):
    if form == "node":
        return float(h_ext)
    if form == "series":
        return float(U_eff_series(Dif_surf, half_thickness, h_ext))
    raise ValueError(f"未知表面形式 {form!r}，只允许 'node' / 'series'")

```

```
81
82  # 三对角线性系统
83  def assemble_solve(u_old, cap, V, A_in, Dif, dt, U_surf, u_inf, A_surf, source=None):
84      n = u_old.size
85      ab = np.zeros((3, n), dtype=float)
86      diag = V * cap / dt
87      rhs = diag * u_old
88      ab[1, :] = diag
89      coef = A_in * Dif
90      ab[1, :-1] += coef
91      ab[0, 1:] = -coef
92      ab[1, 1:] += coef
93      ab[2, :-1] = -coef
94      ab[1, -1] += A_surf * U_surf
95      rhs[-1] += A_surf * U_surf * u_inf
96      if source is not None:
97          rhs = rhs + V * source
98      return solve_banded((1, 1), ab, rhs)
99
100
101
102 _AIR_CACHE = {}
103
104
105 # 附件 1
106 def load_air(mode="linear"):
107     key = ("air", mode)
108     if key in _AIR_CACHE:
109         return _AIR_CACHE[key]
110     df = pd.read_excel(AIR_XLSX, sheet_name=0)
111     cols = list(df.columns)
112     assert cols == AIR_COLS, f"附件1 列名不符：{cols} != {AIR_COLS}"
113     t = df[AIR_COLS[0]].to_numpy(dtype=float)
114     Ta = df[AIR_COLS[1]].to_numpy(dtype=float)
115     Ca = df[AIR_COLS[2]].to_numpy(dtype=float)
116     assert t.size == 241 and abs(t[-1] - AIR_T_END_S) < 1e-9, "附件1 行数/量程不符"
117     assert np.all(np.diff(t) > 0), "附件1 时间列非严格递增"
118     _AIR_CACHE[key] = (t, Ta, Ca)
119     return t, Ta, Ca
120
121
122 def air_driving(T_plateau=AIR_EXTRAP_T_C, C_plateau=AIR_EXTRAP_C, mode="linear"):
123     t, Ta, Ca = load_air()
124     if mode == "linear":
125         def T_of(tq):
126             tq = np.asarray(tq, dtype=float)
127             return np.where(tq <= AIR_T_END_S, np.interp(np.minimum(tq, AIR_T_END_S), t, Ta),
```

```
            T_plateau)

    def C_of(tq):
        tq = np.asarray(tq, dtype=float)
        return np.where(tq <= AIR_T_END_S, np.interp(np.minimum(tq, AIR_T_END_S), t, Ca),
            C_plateau)
elif mode == "pchip":
    pT, pC = PchipInterpolator(t, Ta), PchipInterpolator(t, Ca)

    def T_of(tq):
        tq = np.asarray(tq, dtype=float)
        return np.where(tq <= AIR_T_END_S, pT(np.minimum(tq, AIR_T_END_S)), T_plateau)

    def C_of(tq):
        tq = np.asarray(tq, dtype=float)
        return np.where(tq <= AIR_T_END_S, pC(np.minimum(tq, AIR_T_END_S)), C_plateau)
elif mode == "const":
    def T_of(tq):
        return np.full_like(np.asarray(tq, dtype=float), T_plateau)

    def C_of(tq):
        return np.full_like(np.asarray(tq, dtype=float), C_plateau)
else:
    raise ValueError(f"未知空气插值模式 {mode}")
return T_of, C_of



def load_radius():
    df = pd.read_excel(RADIUS_XLSX, sheet_name=0)
    cols = list(df.columns)
    assert cols == RADIUS_COLS, f"附件2 列名不符：{cols} != {RADIUS_COLS}"
    t = df[RADIUS_COLS[0]].to_numpy(dtype=float)
    R_cm = df[RADIUS_COLS[1]].to_numpy(dtype=float)
    assert t.size == 145 and abs(t[-1] - RADIUS_T_END_S) < 1e-9, "附件2 行数/量程不符"
    assert np.all(np.diff(R_cm) <= 1e-12), "附件2 半径应单调不增"
    return t, R_cm / 100.0


# 附件 2
def radius_interpolant(mode="pchip"):
    t, R = load_radius()
    if mode == "linear":
        slope = np.diff(R) / np.diff(t)

        class _Lin:
            def __call__(self, tq):
```

```
            return np.interp(tq, t, R)

        def derivative(self):
            def d(tq):
                j = np.clip(np.searchsorted(t, np.asarray(tq, dtype=float),
                                            side="right") - 1, 0, slope.size - 1)
                return slope[j]
            return d
    pch = _Lin()
    dpch = pch.derivative()
elif mode == "pchip":
    pch = PchipInterpolator(t, R)
    dpch = pch.derivative()
else:
    raise ValueError(f"未知半径插值模式 {mode!r}，只允许 'pchip' / 'linear'")
R_end = float(R[-1])

def R_of(tq):
    tq = np.asarray(tq, dtype=float)
    return np.where(tq >= RADIUS_T_END_S, R_end, pch(np.clip(tq, 0.0, RADIUS_T_END_S)))

def Rdot_of(tq):
    tq = np.asarray(tq, dtype=float)
    return np.where(tq >= RADIUS_T_END_S, 0.0, dpch(np.clip(tq, 0.0, RADIUS_T_END_S)))

meta = {"R_start_m": float(R[0]), "R_end_m": R_end,
        "ptp_cm": float((R.max() - R.min()) * 100.0),
        "max_abs_step_cm": float(np.max(np.abs(np.diff(R))) * 100.0),
        "plateau_start_s": float(t[np.argmax(R <= R_end + 1e-15)]),
        "mode": mode}
return R_of, Rdot_of, meta



def _ale_motion_terms(C, r, R, Rdot, dr):
    dCdr = np.empty_like(C)
    dCdr[1:-1] = (C[2:] - C[:-2]) / (2.0 * dr)
    dCdr[0] = (C[1] - C[0]) / dr
    dCdr[-1] = (C[-1] - C[-2]) / dr
    v_mesh = r * Rdot / R
    v_solid = r * Rdot / R
    term_mesh_motion = -v_mesh * dCdr
    term_solid_advection = v_solid * dCdr
    return term_mesh_motion, term_solid_advection, float(np.max(np.abs(v_mesh)))


def _dry_density(C, props_fn, coupling):
```

```
    if coupling == "decoupled":
        rho = np.full_like(np.asarray(C, dtype=float), props_fn()[0])
    else:
        rho = np.asarray(props_fn(C)[0], dtype=float)
    return rho / (1.0 + np.asarray(C, dtype=float))


def make_stepper(pset_name, T_air_of, C_air_of, geometry=("fixed", R0), N=N_CELLS,
                 h=H_CONV, hm=HM_MASS, picard_tol_C=PICARD_TOL_C,
                 picard_tol_T=PICARD_TOL_T, picard_max=PICARD_MAX,
                 mass_form="C", picard_fixed=None, solve_order="C_first",
                 surface_form="node", d_face_mode="kirchhoff"):
    pset = PROPERTY_SETS[pset_name]
    coupling = pset["coupling"]
    props_fn, D_fn = pset["props"], pset["D"]
    geom_mode, geom_arg = geometry[0], geometry[1]
    if geom_mode == "ale":
        R_of, Rdot_of = geom_arg
    else:
        R_const = float(geom_arg)

    def step(C_old, T_old, t_old, dt):
        t_new = t_old + dt
        if geom_mode == "ale":
            R = float(R_of(t_new))
            Rdot = float(Rdot_of(t_new))
        else:
            R, Rdot = R_const, 0.0
        r, dr, V, A_in = grid_weights(R, N)
        A_surf = R
        Ta = float(T_air_of(t_new))
        Ca = float(C_air_of(t_new))

        if geom_mode == "ale":
            term_mesh_motion, term_solid_advection, v_max = _ale_motion_terms(C_old, r, R,
                Rdot, dr)
            motion_src = term_mesh_motion + term_solid_advection
            motion_resid = float(np.linalg.norm(motion_src))
        else:
            motion_src = None
            motion_resid, v_max = 0.0, 0.0

        C_iter = C_old.copy()
        T_iter = T_old.copy()
        n_it, dC_last, dT_last = 0, np.inf, np.inf
        resid_hist = []
        n_target = picard_max if picard_fixed is None else picard_fixed
```

```
    for n_it in range(1, n_target + 1):

        T_face_K = to_kelvin(0.5 * (T_iter[:-1] + T_iter[1:]))
        if coupling == "decoupled":
            Dfc = D_face(C_iter, pset, mode=d_face_mode)
            D_surf = float(D_fn(C_iter[-1]))
            rho_s, cp_s, k_s = props_fn()
            cap_T = np.full_like(T_iter, rho_s * cp_s)
            kfc = np.full(N, k_s)
            k_surf = k_s
        else:
            Dfc = D_face(C_iter, pset, T_face_K, mode=d_face_mode)
            D_surf = float(D_fn(C_iter[-1], to_kelvin(T_iter[-1])))
            rho_n, cp_n, k_n = props_fn(C_iter)
            cap_T = rho_n * cp_n
            kfc = k_face(k_n)
            k_surf = float(k_n[-1])
        U_m = U_surface(D_surf, dr / 2.0, hm, surface_form)
        U_t = U_surface(k_surf, dr / 2.0, h, surface_form)

        if mass_form == "rho_d_C":
            rd = _dry_density(C_iter, props_fn, coupling)
            cap_C = rd
            Dif_C = (rd[:-1] + rd[1:]) * 0.5 * Dfc / dr
            U_m_eff = U_m * float(rd[-1])
        else:
            cap_C = np.ones_like(C_iter)
            Dif_C = Dfc / dr
            U_m_eff = U_m

        def solve_C():



            return assemble_solve(C_old, cap_C, V, A_in, Dif_C, dt, U_m_eff, Ca,
                                  A_surf, source=motion_src)

        def solve_T():
            return assemble_solve(T_old, cap_T, V, A_in, kfc / dr, dt, U_t, Ta, A_surf)

        if solve_order == "C_first":
            C_new = solve_C()
            T_new = solve_T()
        else:
            T_new = solve_T()
            C_new = solve_C()

```

```
            dC_last = float(np.max(np.abs(C_new - C_iter)))
            dT_last = float(np.max(np.abs(T_new - T_iter)))
            resid_hist.append((dC_last, dT_last))
            C_iter, T_iter = C_new, T_new
            if picard_fixed is None and dC_last < picard_tol_C and dT_last < picard_tol_T:
                break

        info = {"n_picard": n_it, "dC": dC_last, "dT": dT_last, "R": R, "Rdot": Rdot,
                "U_m": U_m, "U_t": U_t, "Ta": Ta, "Ca": Ca, "V": V, "dr": dr, "r": r,
                "A_in": A_in, "A_surf": A_surf, "motion_resid": motion_resid,
                "v_mesh_max": v_max, "resid_hist": resid_hist,
                "surface_form": surface_form,
                "converged": (dC_last < picard_tol_C and dT_last < picard_tol_T)}
        return C_iter, T_iter, info

    return step


def solve_drying(pset_name, t_span, stop=("time", None), geometry=("fixed", R0),
                 dt_solve=0.25, dt_out=None, N=N_CELLS, air_mode="linear",
                 T_plateau=AIR_EXTRAP_T_C, C_plateau=AIR_EXTRAP_C, h=H_CONV, hm=HM_MASS,
                 C_init=C0, T_init=T0_C, mass_form="C", picard_fixed=None,
                 solve_order="C_first", record_fields=True, rebound_check=True,
                 surface_form="node", d_face_mode="kirchhoff", t_cap=T_MAX_CAP_S,
                 progress=None):
    T_air_of, C_air_of = air_driving(T_plateau, C_plateau, mode=air_mode)
    step = make_stepper(pset_name, T_air_of, C_air_of, geometry=geometry, N=N, h=h, hm=hm,
                        mass_form=mass_form, picard_fixed=picard_fixed,
                        solve_order=solve_order, surface_form=surface_form,
                        d_face_mode=d_face_mode)
    thr = None if stop[0] == "time" else float(stop[1])


    t_limit = float(t_span) if stop[0] == "time" else min(float(t_span), float(t_cap))
    if dt_out is None:
        dt_out = dt_solve
    stride = int(round(dt_out / dt_solve))
    assert abs(stride * dt_solve - dt_out) < 1e-12, "dt_out 必须被 dt_solve 整除"

    C = np.full(N + 1, float(C_init))
    T = np.full(N + 1, float(T_init))
    t = 0.0
    out_t, out_C, out_T, out_R = [], [], [], []
    hist = {"t": [0.0], "C_max": [float(np.max(C))], "C_bar": [], "T_center":
        [float(T[0])],
            "T_surface": [float(T[-1])], "C_center": [float(C[0])],
            "C_surface": [float(C[-1])], "n_picard": [], "argmax_ok": [],
```

```
        "water_resid": [], "R": [], "D_ratio": []}
    R_init = float(geometry[1][0](0.0)) if geometry[0] == "ale" else float(geometry[1])
    _, _, V0w, _ = grid_weights(R_init, N)
    hist["C_bar"].append(volume_average(C, V0w))
    hist["R"].append(R_init)
    n_pic_max, motion_resid_max, v_mesh_active = 0, 0.0, 0
    n_steps, argmax_all_center = 0, True
    hump_tol = 1e-12
    argmax_center_within_tol, C_hump_excess_max, hump_first_t = True, 0.0, None
    t_end, bracket, g_prev = None, None, float(np.max(C)) - (thr if thr else 0.0)
    state_end = None
    state_prev = (C.copy(), T.copy(), 0.0)
    D_field_ratio_max = 0.0
    water_resid_max = 0.0

    while t < t_limit - 1e-12:
        dt = min(dt_solve, t_limit - t)
        C_prev_arr, T_prev_arr, t_prev = C.copy(), T.copy(), t
        C, T, info = step(C_prev_arr, T_prev_arr, t, dt)
        t += dt
        n_steps += 1
        n_pic_max = max(n_pic_max, info["n_picard"])
        motion_resid_max = max(motion_resid_max, info["motion_resid"])
        if info["v_mesh_max"] > 0.0:
            v_mesh_active += 1

        Vv = info["V"]
        lhs = float(np.dot(Vv, C - C_prev_arr) / dt)
        rhs_flux = -info["A_surf"] * info["U_m"] * (float(C[-1]) - info["Ca"])
        scale = max(abs(lhs), abs(rhs_flux), 1e-300)
        wres = abs(lhs - rhs_flux) / scale
        water_resid_max = max(water_resid_max, wres)
        if np.min(C) > 0:
            D_field_ratio_max = max(D_field_ratio_max, float(np.max(C) / np.min(C)))
        Cmax = float(np.max(C))









        hump = Cmax - float(C[0])
        C_hump_excess_max = max(C_hump_excess_max, hump)
        if int(np.argmax(C)) != 0:
            argmax_all_center = False
            if hump > hump_tol:
                argmax_center_within_tol = False
                if hump_first_t is None:
                    hump_first_t = t
```

```
406
407         if n_steps % stride == 0 and record_fields:
408             out_t.append(t)
409             out_C.append(C.copy())
410             out_T.append(T.copy())
411             out_R.append(info["R"])
412         if n_steps % max(1, stride) == 0:
413             hist["t"].append(t)
414             hist["C_max"].append(Cmax)
415             hist["C_bar"].append(volume_average(C, Vv))
416             hist["T_center"].append(float(T[0]))
417             hist["T_surface"].append(float(T[-1]))
418             hist["C_center"].append(float(C[0]))
419             hist["C_surface"].append(float(C[-1]))
420             hist["n_picard"].append(info["n_picard"])
421             hist["water_resid"].append(wres)
422             hist["R"].append(info["R"])
423
424         if thr is not None and t_end is None:
425             g_now = Cmax - thr
426             if g_prev >= 0.0 > g_now:
427                 bracket = (t_prev, t, g_prev, g_now)
428                 t_end, state_end = _bisect_first_passage(step, C_prev_arr, T_prev_arr,
429                                                         t_prev, dt, thr)
430                 if not rebound_check:
431                     break
432             g_prev = g_now
433         if thr is not None and t_end is not None and rebound_check:
434             if t >= REBOUND_FACTOR * t_end:
435                 break
436         if progress is not None and n_steps % progress == 0:
437             print(f" t={t:9.1f}s maxC={Cmax:.6f} Tc={T[0]:7.3f} npic={info['n_picard']}",
438                   flush=True)
439
440     no_rebound = True
441     if thr is not None and t_end is not None:
442         tail = [c for tt, c in zip(hist["t"], hist["C_max"]) if tt >= t_end]
443         no_rebound = bool(all(c < thr for c in tail)) if tail else True
444
445     return {"t_out": np.asarray(out_t), "C_out": np.asarray(out_C) if out_C else
        np.zeros((0, N + 1)),
446             "T_out": np.asarray(out_T) if out_T else np.zeros((0, N + 1)),
447             "R_out": np.asarray(out_R), "C_final": C, "T_final": T, "t_final": t,
448             "hist": hist, "t_end": t_end, "bracket": bracket, "no_rebound": no_rebound,
449             "C_at_t_end": None if state_end is None else state_end[0],
450             "T_at_t_end": None if state_end is None else state_end[1],
451             "t_at_state_end": None if state_end is None else state_end[2],
```

```
        "n_steps": n_steps, "n_picard_max": n_pic_max,
        "motion_resid_max": motion_resid_max, "v_mesh_active_steps": v_mesh_active,
        "argmax_center_always": argmax_all_center,
        "argmax_center_within_tol": argmax_center_within_tol,
        "C_hump_excess_max": C_hump_excess_max, "hump_tol": hump_tol,
        "hump_first_t_s": hump_first_t, "water_resid_max": water_resid_max,
        "D_field_ratio_max": D_field_ratio_max, "N": N, "dt_solve": dt_solve,
        "dt_out": dt_out, "stride": stride, "surface_form": surface_form}


# 首达时刻的局部二分
def _bisect_first_passage(step, C0_arr, T0_arr, t0, dt, thr):
    lo, hi = 0.0, dt
    sub = max(dt / 16.0, 1e-6)
    state_hi = None
    while hi - lo > BISECT_TOL_S:
        mid = 0.5 * (lo + hi)
        C, T, tt = C0_arr.copy(), T0_arr.copy(), t0
        remaining = mid
        while remaining > 1e-12:
            h_step = min(sub, remaining)
            C, T, _ = step(C, T, tt, h_step)
            tt += h_step
            remaining -= h_step
        if float(np.max(C)) - thr < 0.0:
            hi = mid
            state_hi = (C.copy(), T.copy(), t0 + mid)
        else:
            lo = mid
    return t0 + 0.5 * (lo + hi), state_hi


# 解析解校核
def bessel_eigenvalues(Bi, n_terms=40):
    z0 = jn_zeros(0, n_terms + 2)
    f = lambda x: x * j1(x) - Bi * j0(x)
    roots = []
    lo = 1e-9
    for k in range(n_terms):
        hi = z0[k] - 1e-9
        if f(lo) * f(hi) < 0:
            roots.append(brentq(f, lo, hi, xtol=1e-14, rtol=1e-15))
        lo = z0[k] + 1e-9
    return np.asarray(roots[:n_terms])


def bessel_series_T(r, t, Bi, alpha, R, T0, T_inf, n_terms=40):
```

```
   lam = bessel_eigenvalues(Bi, n_terms)
   Fo = alpha * np.asarray(t, dtype=float) / R ** 2
   rr = np.asarray(r, dtype=float) / R
   Cn = 2.0 * Bi / ((lam ** 2 + Bi ** 2) * j0(lam))
   theta = np.zeros(np.shape(rr), dtype=float)
   for n in range(lam.size):
      theta = theta + Cn[n] * j0(lam[n] * rr) * np.exp(-lam[n] ** 2 * Fo)
   return T_inf + (T0 - T_inf) * theta, lam, Cn


def bessel_Bn(Bi, n_terms=40):
   lam = bessel_eigenvalues(Bi, n_terms)
   Cn = 2.0 * Bi / ((lam ** 2 + Bi ** 2) * j0(lam))
   Bn = 2.0 * Cn * j1(lam) / lam
   return float(np.sum(Bn)), float(np.sum(Cn)), lam



def duhamel_T(r, t_query, Bi, alpha, R, T0, T_air_of, n_terms=40, n_sub=4000):
   lam = bessel_eigenvalues(Bi, n_terms)
   Cn = 2.0 * Bi / ((lam ** 2 + Bi ** 2) * j0(lam))
   rr = np.atleast_1d(np.asarray(r, dtype=float)) / R
   basis = np.array([j0(l * rr) for l in lam])
   out = []
   for tq in np.atleast_1d(np.asarray(t_query, dtype=float)):
      s = np.linspace(0.0, tq, n_sub + 1)
      Ta = np.asarray(T_air_of(s), dtype=float)
      dTa = np.gradient(Ta, s, edge_order=2)

      Fo = alpha * (tq - s) / R ** 2
      decay = np.exp(-np.outer(lam ** 2, Fo))
      theta = np.tensordot(Cn[:, None] * basis, decay, axes=([0], [0]))
      g = 1.0 - theta
      conv = np.trapezoid(g * dTa[None, :], s, axis=1)
      theta_t0 = np.tensordot(Cn[:, None] * basis, np.exp(-lam ** 2 * alpha * tq / R ** 2),
                              axes=([0], [0]))
      out.append(T0 + (Ta[0] - T0) * (1.0 - theta_t0) + conv)
   res = np.asarray(out)
   return res[0] if res.shape[0] == 1 else res



def convergence_ratios(values):
   v = [float(x) for x in values]
   diffs = [v[i + 1] - v[i] for i in range(len(v) - 1)]
   ratio = [(diffs[i] / diffs[i + 1]) if abs(diffs[i + 1]) > 0 else float("inf")
         for i in range(len(diffs) - 1)]
```

```
   rel = [abs(diffs[i]) / max(abs(v[i + 1]), 1e-300) for i in range(len(diffs))]
   return {"values": v, "diffs": diffs, "ratio": ratio, "rel_change": rel}


# 输出列和表格
def output_columns_cm(n_cols=21, dr_cm=0.1):
   return [round(i * dr_cm, 1) for i in range(n_cols)]


def extract_output_nodes(field_rows, stride):
   return np.asarray(field_rows)[:, ::stride]


def write_xlsx(path, sheets, decimals=DECIMALS):
   with pd.ExcelWriter(path, engine="openpyxl") as xw:
      for name, df in sheets.items():
         df.round(decimals).to_excel(xw, sheet_name=name, index=False)
   return path


def make_field_df(times_s, cols_cm, values, index_header=INDEX_HEADER):
   df = pd.DataFrame(values, columns=[str(c) for c in cols_cm])
   df.insert(0, index_header, np.asarray(times_s, dtype=float))
   return df


def fmt(x, decimals=DECIMALS):
   return float(f"{float(x):.{decimals}f}")
```

1.4 问题一

```

from __future__ import annotations

import json
import time

import numpy as np
import pandas as pd

from params import (C0, DECIMALS, DT_SOLVE_Q12, FIGDIR, H_CONV, HM_MASS, KELVIN_OFFSET,
                    N_CELLS, OUT_STRIDE_SPACE, Q1_DT_OUT, Q1_SPAN_S, Q1_TABLE_RADII_CM,
                    Q1_TABLE_TIMES_S, R0, SHEETS_TC, T0_C, TMPDIR, USER_DATA,
                    N_CONVERGENCE, DT_CONVERGENCE_Q12)
from props import D_app2, props_app2
```

```python
import inspect
from utils import (air_driving, convergence_ratios, duhamel_T, extract_output_nodes,
                   grid_weights, make_field_df, output_columns_cm, solve_drying,
                   volume_average, write_xlsx)

TAG = "problem_1"


def run():
    t0_wall = time.time()
    # 先算无量纲数，再推进温度和水分。
    rho, cp, k = props_app2()
    alpha = k / (rho * cp)
    Bi_T = H_CONV * R0 / k
    D_C0 = float(D_app2(C0))
    Bi_m = HM_MASS * R0 / D_C0
    out = {"problem": 1, "property_set": "appendix2", "coupling": "decoupled",
           "rho_kg_m3": rho, "cp_J_kgK": cp, "k_W_mK": k,
           "alpha_m2_s": alpha, "Bi_T": Bi_T, "D_at_C0_m2_s": D_C0, "Bi_m": Bi_m,
           "Fo_m_1800s": D_C0 * 1800.0 / R0 ** 2, "Le_alpha_over_D": alpha / D_C0,
           "diffusion_time_scale_s": R0 ** 2 / alpha}
    print(f"[Q1] alpha={alpha:.6e} Bi_T={Bi_T:.7f} D(C0)={D_C0:.6e} Bi_m={Bi_m:.6f} "
          f"Le={alpha / D_C0:.4f}", flush=True)


    sig = str(inspect.signature(D_app2))
    assert "T" not in sig, f"C-P1-2 违反：D_app2 签名 {sig} 含温度参数"
    out["D_app2_signature"] = sig


    # 主解
    print(f"[Q1] 主求解 N={N_CELLS} dt={DT_SOLVE_Q12} span={Q1_SPAN_S} s", flush=True)
    res = solve_drying("app2", Q1_SPAN_S, stop=("time", Q1_SPAN_S), dt_solve=DT_SOLVE_Q12,
                       dt_out=Q1_DT_OUT, N=N_CELLS, air_mode="linear",
                       C_init=C0, T_init=T0_C, record_fields=True)
    print(f"[Q1] 主求解完成 {res['n_steps']} 步 用时 {time.time() - t0_wall:.1f} s",
          flush=True)
    r_nodes, dr, V, A_in = grid_weights(R0, N_CELLS)
    T_field = extract_output_nodes(res["T_out"], OUT_STRIDE_SPACE)
    C_field = extract_output_nodes(res["C_out"], OUT_STRIDE_SPACE)
    t_out = res["t_out"]
    assert T_field.shape == (1800, 21), f"输出形状不符：{T_field.shape}"
    assert abs(t_out[0] - 1.0) < 1e-12 and abs(t_out[-1] - 1800.0) < 1e-12, "输出时刻不符"

    out["n_steps"] = res["n_steps"]
    out["dt_solve_s"] = DT_SOLVE_Q12
    out["dt_out_s"] = Q1_DT_OUT
```

```
    out["N_cells"] = N_CELLS
    out["surface_form"] = res["surface_form"]
    out["water_resid_max"] = res["water_resid_max"]
    out["n_picard_max"] = res["n_picard_max"]
    out["argmax_center_strict"] = bool(res["argmax_center_always"])
    out["argmax_center_within_tol"] = bool(res["argmax_center_within_tol"])
    out["C_hump_excess_max"] = res["C_hump_excess_max"]


    # 解耦时交换方程顺序，结果应一致。
    res_swap = solve_drying("app2", Q1_SPAN_S, stop=("time", Q1_SPAN_S),
                            dt_solve=DT_SOLVE_Q12, dt_out=Q1_SPAN_S, N=N_CELLS,
                            air_mode="linear", C_init=C0, T_init=T0_C,
                            solve_order="T_first", record_fields=False)
    dT_swap = float(np.max(np.abs(res_swap["T_final"] - res["T_final"])))
    dC_swap = float(np.max(np.abs(res_swap["C_final"] - res["C_final"])))
    out["decoupling_swap_order_max_dT_K"] = dT_swap
    out["decoupling_swap_order_max_dC"] = dC_swap
    assert dT_swap < 1e-12 and dC_swap < 1e-12,\
       f"P1-C1 解耦性失败：换序后 dT={dT_swap:.3e} dC={dC_swap:.3e}"
    print(f"[Q1] 换序一致性 dT={dT_swap:.3e} dC={dC_swap:.3e}", flush=True)


    # 用 Duhamel 解独立核对温度场。
    T_air_of, C_air_of = air_driving()
    idx_check = [0, 100, 200]
    T_duh = duhamel_T(r_nodes[idx_check], 1800.0, Bi_T, alpha, R0, T0_C, T_air_of,
                 n_terms=60, n_sub=7200)
    T_num = res["T_final"][idx_check]
    duh_err = np.abs(np.asarray(T_duh) - T_num)
    out["duhamel_check"] = {
       "r_cm": [0.0, 1.0, 2.0],
       "T_numeric_C": [float(x) for x in T_num],
       "T_duhamel_C": [float(x) for x in T_duh],
       "max_abs_err_K": float(np.max(duh_err))}
    print(f"[Q1] Duhamel 核对 num={[f'{x:.4f}' for x in T_num]} "
         f"duh={[f'{x:.4f}' for x in T_duh]} maxerr={np.max(duh_err):.4e}", flush=True)
    assert float(np.max(duh_err)) < 5e-2, f"Duhamel 核对超差 {np.max(duh_err):.4e} K"


    T18 = T_field[-1]
    C18 = C_field[-1]
    ref_T = {0.0: 33.5766, 1.0: 34.3653, 2.0: 36.7863}
    ref_C = {0.0: 2.5500, 1.0: 2.5383, 2.0: 1.5103}
    col_cm = output_columns_cm(21, 0.1)
    got_T = {c: float(T18[col_cm.index(c)]) for c in ref_T}
    got_C = {c: float(C18[col_cm.index(c)]) for c in ref_C}
```

```
    out["t1800_temperature_C"] = got_T
    out["t1800_moisture_kgkg"] = got_C
    out["t1800_ref_temperature_C"] = ref_T
    out["t1800_ref_moisture_kgkg"] = ref_C
    out["t1800_dev_temperature_K"] = {str(c): abs(got_T[c] - ref_T[c]) for c in ref_T}
    out["t1800_dev_moisture"] = {str(c): abs(got_C[c] - ref_C[c]) for c in ref_C}
    print(f"[Q1] t=1800s T: {got_T}", flush=True)
    print(f"[Q1] t=1800s C: {got_C}", flush=True)


    out["C_center_1800_rounded"] = round(got_C[0.0], DECIMALS)
    out["C_center_unchanged_to_4dp"] = bool(round(got_C[0.0], DECIMALS) == round(C0,
        DECIMALS))
    out["T_air_1800_C"] = float(T_air_of(1800.0))
    out["C_air_1800"] = float(C_air_of(1800.0))
    out["C_bar_1800"] = volume_average(res["C_out"][-1], V)
    out["T_center_history_sample"] = {
       str(int(tt)): float(res["T_out"][int(tt) - 1][0]) for tt in (100, 600, 1200, 1800)}


    # 收敛性
    print("[Q1] 网格收敛 N=100/200/400/800 ...", flush=True)
    gv_T, gv_C = [], []
    for Nc in N_CONVERGENCE:
       rc = solve_drying("app2", Q1_SPAN_S, stop=("time", Q1_SPAN_S), dt_solve=DT_SOLVE_Q12,
                    dt_out=Q1_SPAN_S, N=Nc, air_mode="linear", C_init=C0,
                    T_init=T0_C, record_fields=False)
       _, _, Vc, _ = grid_weights(R0, Nc)
       gv_T.append(float(rc["T_final"][0]))
       gv_C.append(volume_average(rc["C_final"], Vc))
       print(f" N={Nc}: T(0,1800)={gv_T[-1]:.8f} Cbar={gv_C[-1]:.10f}", flush=True)
    print("[Q1] 步长收敛 dt=1/0.5/0.25/0.125 ...", flush=True)
    dv_T, dv_C = [], []
    for dtc in DT_CONVERGENCE_Q12:
       rc = solve_drying("app2", Q1_SPAN_S, stop=("time", Q1_SPAN_S), dt_solve=dtc,
                    dt_out=Q1_SPAN_S, N=N_CELLS, air_mode="linear", C_init=C0,
                    T_init=T0_C, record_fields=False)
       dv_T.append(float(rc["T_final"][0]))
       dv_C.append(volume_average(rc["C_final"], V))
       print(f" dt={dtc}: T(0,1800)={dv_T[-1]:.8f} Cbar={dv_C[-1]:.10f}", flush=True)
    out["convergence"] = {
       "N_list": list(N_CONVERGENCE), "dt_list": list(DT_CONVERGENCE_Q12),
       "grid_T_center": convergence_ratios(gv_T), "grid_C_bar": convergence_ratios(gv_C),
       "dt_T_center": convergence_ratios(dv_T), "dt_C_bar": convergence_ratios(dv_C)}
    for kk in ("grid_T_center", "grid_C_bar", "dt_T_center", "dt_C_bar"):
       rc_ = out["convergence"][kk]["rel_change"][-1]
       assert abs(rc_) < 5e-3, f"{kk} 末档相对变化 {rc_:.3e} 未收敛"
```

```

    tab_ti = [int(round(x)) for x in Q1_TABLE_TIMES_S]
    tab_ci = [col_cm.index(round(x, 1)) for x in Q1_TABLE_RADII_CM]
    tabT = {str(tt): {str(Q1_TABLE_RADII_CM[j]): float(T_field[tt - 1][ci])
                  for j, ci in enumerate(tab_ci)} for tt in tab_ti}
    tabC = {str(tt): {str(Q1_TABLE_RADII_CM[j]): float(C_field[tt - 1][ci])
                  for j, ci in enumerate(tab_ci)} for tt in tab_ti}
    out["paper_table_temperature_C"] = tabT
    out["paper_table_moisture_kgkg"] = tabC

    # 写表并回读检查。
    dfT = make_field_df(t_out, col_cm, T_field)
    dfC = make_field_df(t_out, col_cm, C_field)
    xlsx = USER_DATA / "result1.xlsx"
    write_xlsx(xlsx, {SHEETS_TC[0]: dfT, SHEETS_TC[1]: dfC})
    out["result_file"] = str(xlsx.name)
    out["result_shape"] = [int(dfT.shape[0]), int(dfT.shape[1])]
    print(f"[Q1] 写出 {xlsx.name} sheets={SHEETS_TC} shape={dfT.shape}", flush=True)


    chk = pd.read_excel(xlsx, sheet_name=SHEETS_TC[0])
    worst = 0.0
    for tt in tab_ti:
       for j, ci in enumerate(tab_ci):
          a = float(chk.iloc[tt - 1, ci + 1])
          b = round(tabT[str(tt)][str(Q1_TABLE_RADII_CM[j])], DECIMALS)
          worst = max(worst, abs(a - b))
    out["table_vs_file_max_abs_diff"] = worst
    assert worst == 0.0, f"表格与文件不一致 {worst}"

    out["wall_time_s"] = time.time() - t0_wall
    FIGDIR.mkdir(exist_ok=True)
    with open(FIGDIR / f"{TAG}_results.json", "w", encoding="utf-8") as fh:
       json.dump(out, fh, ensure_ascii=False, indent=2, default=float)
    print(f"[Q1] -> figures/{TAG}_results.json ({out['wall_time_s']:.1f} s)", flush=True)
    return out


if __name__ == "__main__":
    run()
```

1.5 问题二

```

```

```python
from __future__ import annotations

import json
import time

import numpy as np
import pandas as pd

from params import (C0, C_THRESHOLD, DECIMALS, DT_CONVERGENCE_Q12, DT_SOLVE_Q12, FIGDIR,
                    H_CONV, HM_MASS, KELVIN_OFFSET, N_CELLS, N_CONVERGENCE,
                    OUT_STRIDE_SPACE, Q2_DT_OUT, Q2_SPAN_S, Q2_TABLE_RADII_CM,
                    Q2_TABLE_TIMES_H, R0, SHEETS_TC, T0_C, USER_DATA)
from props import D_app2, D_app3, props_app3, arrhenius
from utils import (air_driving, convergence_ratios, extract_output_nodes, grid_weights,
                   make_field_df, output_columns_cm, solve_drying, volume_average,
                   write_xlsx)

TAG = "problem_2"


def run():
    t0_wall = time.time()
    out = {"problem": 2, "property_set": "appendix3", "coupling": "two_way"}


    # 物性和温度因子检查
    r0_, c0_, k0_ = props_app3(C0)
    r_thr, c_thr, k_thr = props_app3(C_THRESHOLD)
    out["props_at_C0"] = {"rho": float(r0_), "cp": float(c0_), "k": float(k0_)}
    out["props_at_Cthr"] = {"rho": float(r_thr), "cp": float(c_thr), "k": float(k_thr)}
    out["cp_drop_pct"] = float((c0_ - c_thr) / c0_ * 100.0)
    out["k_drop_pct"] = float((k0_ - k_thr) / k0_ * 100.0)
    assert abs(float(c0_) - 3415.2958) < 1e-3, f"cp(2.55) 应为 3415.2958，得
        {float(c0_):.4f}"
    assert abs(float(k0_) - 0.4829577) < 1e-6, f"k(2.55) 应为 0.4829577，得
        {float(k0_):.7f}"
    out["capability_checklist_deviation"] = {
       "P2-C1(a)_expects_cp": 3416.6, "corrected_cp": float(c0_),
       "P2-C1(a)_expects_k": 0.4832, "corrected_k": float(k0_),
       "note": "CAPABILITY_CHECKLIST 携带 §0 修正前的上游值；第二项自变量应为 C/(C+1)=51/71"}
    print(f"[Q2] props@C0 rho={float(r0_):.4f} cp={float(c0_):.4f} k={float(k0_):.7f}",
        flush=True)


    f28 = float(arrhenius(T0_C + KELVIN_OFFSET, 3850.0))
    f50 = float(arrhenius(50.0 + KELVIN_OFFSET, 3850.0))
    out["arrhenius"] = {"f_28C": f28, "f_50C": f50, "amplification_28_to_50": f50 / f28,
```

```
                "silent_bug_if_celsius": float(np.exp(-3850.0 / 50.0)),
                "bound_lo": 1e-7, "bound_hi": 1e-5}
assert 1e-7 <= f28 <= 1e-5 and 1e-7 <= f50 <= 1e-5, "Arrhenius 因子越界"
D0_a3 = float(D_app3(C0, 50.0 + KELVIN_OFFSET))
D0_a2 = float(D_app2(C0))
out["D_app3_at_C0_50C"] = D0_a3
out["D_ratio_app3_over_app2"] = D0_a3 / D0_a2
print(f"[Q2] f(28C)={f28:.6e} f(50C)={f50:.6e} 放大 {f50 / f28:.4f} 倍; "
      f"D3/D2={D0_a3 / D0_a2:.4f}", flush=True)


# 主解
print(f"[Q2] 主求解 N={N_CELLS} dt={DT_SOLVE_Q12} span={Q2_SPAN_S} s "
      f"(43200 步，约 1 分钟)", flush=True)
res = solve_drying("app3", Q2_SPAN_S, stop=("time", Q2_SPAN_S), dt_solve=DT_SOLVE_Q12,
                dt_out=Q2_DT_OUT, N=N_CELLS, air_mode="linear",
                C_init=C0, T_init=T0_C, record_fields=True,
                progress=14400)
print(f"[Q2] 主求解完成 {res['n_steps']} 步 Picard max={res['n_picard_max']} "
      f"用时 {time.time() - t0_wall:.1f} s", flush=True)
r_nodes, dr, V, A_in = grid_weights(R0, N_CELLS)
T_field = extract_output_nodes(res["T_out"], OUT_STRIDE_SPACE)
C_field = extract_output_nodes(res["C_out"], OUT_STRIDE_SPACE)
t_out = res["t_out"]
assert T_field.shape == (10800, 21), f"输出形状不符：{T_field.shape}"
out.update({"n_steps": res["n_steps"], "dt_solve_s": DT_SOLVE_Q12,
          "dt_out_s": Q2_DT_OUT, "N_cells": N_CELLS,
          "surface_form": res["surface_form"],
          "water_resid_max": res["water_resid_max"],
          "n_picard_max": res["n_picard_max"],
          "D_field_ratio_max": res["D_field_ratio_max"],
          "argmax_center_strict": bool(res["argmax_center_always"]),
          "argmax_center_within_tol": bool(res["argmax_center_within_tol"]),
          "C_hump_excess_max": res["C_hump_excess_max"]})

col_cm = output_columns_cm(21, 0.1)
ci = {c: col_cm.index(c) for c in (0.0, 0.5, 1.0, 1.5, 2.0)}
out["t10800_temperature_C"] = {str(c): float(T_field[-1][i]) for c, i in ci.items()}
out["t10800_moisture_kgkg"] = {str(c): float(C_field[-1][i]) for c, i in ci.items()}
out["C_bar_10800"] = volume_average(res["C_out"][-1], V)
out["C_bar_0"] = float(C0)
T_air_of, C_air_of = air_driving()
out["T_air_10800_C"] = float(T_air_of(10800.0))
out["C_air_10800"] = float(C_air_of(10800.0))
print(f"[Q2] t=3h T: {out['t10800_temperature_C']}", flush=True)
print(f"[Q2] t=3h C: {out['t10800_moisture_kgkg']} Cbar={out['C_bar_10800']:.6f}",
      flush=True)
```

```
93
94
95  # 长程运行只作一致性检查。
96  print("[Q2] P2-C2 长程对照 t_span=207360 s 的前 10800 s ...", flush=True)
97  res_long = solve_drying("app3", 207360.0, stop=("time", 207360.0),
        dt_solve=DT_SOLVE_Q12,
98                  dt_out=Q2_SPAN_S, N=N_CELLS, air_mode="linear",
99                  C_init=C0, T_init=T0_C, record_fields=True, progress=100000)
100 i10800 = int(np.argmin(np.abs(res_long["t_out"] - Q2_SPAN_S)))
101 assert abs(res_long["t_out"][i10800] - Q2_SPAN_S) < 1e-9, "长程输出缺 10800 s 时层"
102 dC_long = float(np.max(np.abs(res_long["C_out"][i10800] - res["C_out"][-1])))
103 dT_long = float(np.max(np.abs(res_long["T_out"][i10800] - res["T_out"][-1])))
104 out["P2C2_long_vs_short_max_dC"] = dC_long
105 out["P2C2_long_vs_short_max_dT_K"] = dT_long
106 out["P2C2_long_span_s"] = 207360.0
107 assert dC_long < 1e-12 and dT_long < 1e-12,\
108    f"P2-C2 失败：长程改变了短程解 dC={dC_long:.3e} dT={dT_long:.3e}"
109 print(f"[Q2] P2-C2 长短程一致 dC={dC_long:.3e} dT={dT_long:.3e}", flush=True)
110
111
112 # 比较 Picard 收敛和固定两轮。
113 res_p2 = solve_drying("app3", Q2_SPAN_S, stop=("time", Q2_SPAN_S),
        dt_solve=DT_SOLVE_Q12,
114                 dt_out=Q2_SPAN_S, N=N_CELLS, air_mode="linear", C_init=C0,
115                 T_init=T0_C, picard_fixed=2, record_fields=False)
116 _, _, Vp, _ = grid_weights(R0, N_CELLS)
117 cb_conv = volume_average(res["C_out"][-1], V)
118 cb_fix2 = volume_average(res_p2["C_final"], Vp)
119 rel_pic = abs(cb_fix2 - cb_conv) / cb_conv
120 out["PXC3_picard_invariance"] = {
121    "C_bar_converged_1e-10": cb_conv, "C_bar_fixed_2_iters": cb_fix2,
122    "rel_diff": rel_pic,
123    "T_center_converged": float(res["T_out"][-1][0]),
124    "T_center_fixed2": float(res_p2["T_final"][0])}
125 assert rel_pic < 1e-4, f"PX-C3 失败：迭代次数改变了解 rel={rel_pic:.3e}（疑把迭代当推进）"
126 print(f"[Q2] PX-C3 Picard 不变性 rel={rel_pic:.3e}", flush=True)
127
128
129 # 收敛性
130 print("[Q2] 网格收敛 N=100/200/400/800 ...", flush=True)
131 gv_T, gv_C = [], []
132 for Nc in N_CONVERGENCE:
133    rc = solve_drying("app3", Q2_SPAN_S, stop=("time", Q2_SPAN_S), dt_solve=DT_SOLVE_Q12,
134                 dt_out=Q2_SPAN_S, N=Nc, air_mode="linear", C_init=C0,
135                 T_init=T0_C, record_fields=False)
136    _, _, Vc, _ = grid_weights(R0, Nc)
137    gv_T.append(float(rc["T_final"][0]))
```

```
        gv_C.append(volume_average(rc["C_final"], Vc))
        print(f" N={Nc}: T(0,3h)={gv_T[-1]:.8f} Cbar={gv_C[-1]:.10f}", flush=True)
    print("[Q2] 步长收敛 dt=1/0.5/0.25/0.125 ...", flush=True)
    dv_T, dv_C = [], []
    for dtc in DT_CONVERGENCE_Q12:
        rc = solve_drying("app3", Q2_SPAN_S, stop=("time", Q2_SPAN_S), dt_solve=dtc,
                    dt_out=Q2_SPAN_S, N=N_CELLS, air_mode="linear", C_init=C0,
                    T_init=T0_C, record_fields=False)
        dv_T.append(float(rc["T_final"][0]))
        dv_C.append(volume_average(rc["C_final"], V))
        print(f" dt={dtc}: T(0,3h)={dv_T[-1]:.8f} Cbar={dv_C[-1]:.10f}", flush=True)
    out["convergence"] = {
        "N_list": list(N_CONVERGENCE), "dt_list": list(DT_CONVERGENCE_Q12),
        "grid_T_center": convergence_ratios(gv_T), "grid_C_bar": convergence_ratios(gv_C),
        "dt_T_center": convergence_ratios(dv_T), "dt_C_bar": convergence_ratios(dv_C)}
    for kk in ("grid_T_center", "grid_C_bar", "dt_T_center", "dt_C_bar"):
        rc_ = out["convergence"][kk]["rel_change"][-1]
        assert abs(rc_) < 5e-3, f"{kk} 末档相对变化 {rc_:.3e} 未收敛"


    tab_ti = [int(round(x * 3600.0)) for x in Q2_TABLE_TIMES_H]
    tab_ci = [col_cm.index(round(x, 1)) for x in Q2_TABLE_RADII_CM]
    tabT = {str(tt): {str(Q2_TABLE_RADII_CM[j]): float(T_field[tt - 1][c])
                  for j, c in enumerate(tab_ci)} for tt in tab_ti}
    tabC = {str(tt): {str(Q2_TABLE_RADII_CM[j]): float(C_field[tt - 1][c])
                  for j, c in enumerate(tab_ci)} for tt in tab_ti}
    out["paper_table_times_s"] = tab_ti
    out["paper_table_temperature_C"] = tabT
    out["paper_table_moisture_kgkg"] = tabC

    # 写表并回读检查。
    dfT = make_field_df(t_out, col_cm, T_field)
    dfC = make_field_df(t_out, col_cm, C_field)
    xlsx = USER_DATA / "result2.xlsx"
    write_xlsx(xlsx, {SHEETS_TC[0]: dfT, SHEETS_TC[1]: dfC})
    out["result_file"] = str(xlsx.name)
    out["result_shape"] = [int(dfT.shape[0]), int(dfT.shape[1])]
    print(f"[Q2] 写出 {xlsx.name} shape={dfT.shape}", flush=True)

    chk = pd.read_excel(xlsx, sheet_name=SHEETS_TC[1])
    worst = 0.0
    for tt in tab_ti:
        for j, c in enumerate(tab_ci):
            a = float(chk.iloc[tt - 1, c + 1])
            b = round(tabC[str(tt)][str(Q2_TABLE_RADII_CM[j])], DECIMALS)
            worst = max(worst, abs(a - b))
    out["table_vs_file_max_abs_diff"] = worst
```

```
    assert worst == 0.0, f"表格与文件不一致 {worst}"

    out["wall_time_s"] = time.time() - t0_wall
    FIGDIR.mkdir(exist_ok=True)
    with open(FIGDIR / f"{TAG}_results.json", "w", encoding="utf-8") as fh:
        json.dump(out, fh, ensure_ascii=False, indent=2, default=float)
    print(f"[Q2] -> figures/{TAG}_results.json ({out['wall_time_s']:.1f} s)", flush=True)
    return out


if __name__ == "__main__":
    run()
```

1.6 问题三

```

from __future__ import annotations

import json
import time

import numpy as np
import pandas as pd

from params import (AIR_EXTRAP_C, AIR_EXTRAP_T_C, AIR_T_END_S, C0, C_THRESHOLD, DECIMALS,
                    DT_CONVERGENCE_Q34, DT_SOLVE_Q34, DURATION_DAYS_HIGH,
                    DURATION_DAYS_LOW, FIGDIR, N_CELLS, N_CONVERGENCE, OUT_STRIDE_SPACE,
                    Q3_DT_OUT, Q3_TABLE_DT_H, R0, T0_C, T_MAX_CAP_S, USER_DATA)
from utils import (convergence_ratios, extract_output_nodes, grid_weights, make_field_df,
                   output_columns_cm, solve_drying, volume_average, write_xlsx)

TAG = "problem_3"


def run():
    t0_wall = time.time()
    out = {"problem": 3, "property_set": "appendix3", "coupling": "two_way",
           "solver_reused_from": "problem2 (identical solve_drying call, stop=
               first_passage)",
           "C_threshold_kgkg": C_THRESHOLD, "criterion": "max_r C(r,t) < C_thr (R5, 上确界)"}

    # 终点用 max(C) 的首次越阈值时刻。
    print(f"[Q3] 首达求解 N={N_CELLS} dt={DT_SOLVE_Q34} 上限 {T_MAX_CAP_S / 3600:.0f} h",
          flush=True)
    res = solve_drying("app3", T_MAX_CAP_S, stop=("first_passage", C_THRESHOLD),
```

```
                dt_solve=DT_SOLVE_Q34, dt_out=Q3_DT_OUT, N=N_CELLS,
                air_mode="linear", C_init=C0, T_init=T0_C, record_fields=True,
                rebound_check=True, progress=20000)
    assert res["t_end"] is not None,\
       f"上限 {T_MAX_CAP_S}s 内未达标，末态 maxC={float(np.max(res['C_final'])):.6f}"
    t_end_s = float(res["t_end"])
    t_end_h = t_end_s / 3600.0
    print(f"[Q3] t_end = {t_end_s:.4f} s = {t_end_h:.4f} h = {t_end_h / 24:.4f} d "
         f"({res['n_steps']} 步 {time.time() - t0_wall:.1f} s)", flush=True)

    r_nodes, dr, V, A_in = grid_weights(R0, N_CELLS)
    out.update({"t_end_s": t_end_s, "t_end_h": round(t_end_h, DECIMALS),
             "t_end_h_full": t_end_h, "t_end_days": t_end_h / 24.0,
             "bracket_s": [float(res["bracket"][0]), float(res["bracket"][1])],
             "bracket_len_s": float(res["bracket"][1] - res["bracket"][0]),
             "no_rebound_to_1.05t_end": bool(res["no_rebound"]),
             "argmax_center_strict": bool(res["argmax_center_always"]),
             "argmax_center_within_tol": bool(res["argmax_center_within_tol"]),
             "C_hump_excess_max": res["C_hump_excess_max"],
             "hump_tol": res["hump_tol"],
             "hump_first_t_s": res["hump_first_t_s"],
             "final_profile_max_diff": float(np.max(np.diff(res["C_final"]))),
             "n_steps": res["n_steps"], "dt_solve_s": DT_SOLVE_Q34,
             "dt_out_s": Q3_DT_OUT, "N_cells": N_CELLS,
             "surface_form": res["surface_form"],
             "water_resid_max": res["water_resid_max"],
             "n_picard_max": res["n_picard_max"],
             "D_field_ratio_max": res["D_field_ratio_max"]})



    assert res["argmax_center_within_tol"],\
       f"出现真实内部驼峰（高度 {res['C_hump_excess_max']:.3e} > {res['hump_tol']:.0e}）: "\
       "须如实报告剖面非单调，不得改判据"
    assert float(np.max(np.diff(res["C_final"]))) < 0.0, "末态剖面非严格单调递减"
    assert res["no_rebound"], "1.05*t_end 内 g 出现回升：首达时刻需修正"


    # 用二分返回的状态，不取最近输出点。
    hist = res["hist"]
    ht = np.asarray(hist["t"], dtype=float)


    C_end = res["C_at_t_end"]
    T_end = res["T_at_t_end"]
    assert C_end is not None, "二分未返回 t_end 处状态"
    out["at_t_end"] = {"t_s": float(res["t_at_state_end"]),
```

```
                "max_C": float(np.max(C_end)),
                "C_bar": volume_average(C_end, V),
                "C_center": float(C_end[0]), "C_surface": float(C_end[-1]),
                "T_center_C": float(T_end[0]), "T_surface_C": float(T_end[-1]),
                "g_at_t_end": float(np.max(C_end)) - C_THRESHOLD}
assert out["at_t_end"]["max_C"] < C_THRESHOLD,\
   f"t_end 处 maxC={out['at_t_end']['max_C']:.8f} 未低于阈值 {C_THRESHOLD}"

out["at_t_end"]["g_magnitude_ok"] = bool(-1e-4 < out["at_t_end"]["g_at_t_end"] < 0.0)

cb = np.asarray(hist["C_bar"], dtype=float)
below = np.where(cb < C_THRESHOLD)[0]
t_bar_s = float(ht[below[0]]) if below.size else None
out["wrong_criterion_C_bar_t_end_h"] = (t_bar_s / 3600.0) if t_bar_s else None
out["wrong_criterion_C_bar_underestimates_by_h"] =\
   (t_end_h - t_bar_s / 3600.0) if t_bar_s else None
print(f"[Q3] t_end 处 maxC={out['at_t_end']['max_C']:.6f}
    Cbar={out['at_t_end']['C_bar']:.6f}"
     f"；若误用 Cbar 判据 t_end={out['wrong_criterion_C_bar_t_end_h']} h", flush=True)


out["duration_days_cross_check"] = {
   "computed_days": t_end_h / 24.0, "problem_statement_low": DURATION_DAYS_LOW,
   "problem_statement_high": DURATION_DAYS_HIGH,
   "inside_stated_range": bool(DURATION_DAYS_LOW <= t_end_h / 24.0 <=
       DURATION_DAYS_HIGH)}

out["expectation_window_h"] = [30.7, 80.6]
out["inside_expectation_window"] = bool(30.7 < t_end_h < 80.6)
assert 30.0 < t_end_h < 90.0, f"t_end={t_end_h:.4f} h 越出 §13 预期区间"
out["checklist_P3C2_window_h"] = [24.0, 72.0]
out["inside_checklist_window"] = bool(24.0 <= t_end_h <= 72.0)


out["air_extrapolation"] = {
   "measured_until_s": AIR_T_END_S, "hold_T_air_C": AIR_EXTRAP_T_C,
   "hold_C_air": AIR_EXTRAP_C,
   "extrapolated_fraction_pct": (1.0 - AIR_T_END_S / t_end_s) * 100.0}
print(f"[Q3] 外推段占比 {out['air_extrapolation']['extrapolated_fraction_pct']:.4f}%",
     flush=True)


dcb = -np.gradient(cb, ht)
out["drying_rate"] = {
   "max_rate_per_s": float(np.max(dcb)), "t_at_max_rate_h": float(ht[np.argmax(dcb)] /
       3600.0),
   "rate_at_1h_per_s": float(np.interp(3600.0, ht, dcb)),
```

```
    "rate_at_t_end_per_s": float(np.interp(t_end_s, ht, dcb)),
    "rate_at_12h_per_s": float(np.interp(12.0 * 3600.0, ht, dcb)),
    "C_bar_note": "体积平均 (2/R^2)int C r dr，用 §5.1 体积权，与守恒诊断同口径"}


# 收敛性
print("[Q3] t_end 网格收敛 N=100/200/400/800 ...", flush=True)
gv = []
for Nc in N_CONVERGENCE:
    rc = solve_drying("app3", T_MAX_CAP_S, stop=("first_passage", C_THRESHOLD),
                dt_solve=DT_SOLVE_Q34, dt_out=3600.0, N=Nc, air_mode="linear",
                C_init=C0, T_init=T0_C, record_fields=False)
    gv.append(float(rc["t_end"]) / 3600.0)
    print(f" N={Nc}: t_end={gv[-1]:.6f} h", flush=True)
print("[Q3] t_end 步长收敛 dt=8/4/2/1 ...", flush=True)
dv = []
for dtc in DT_CONVERGENCE_Q34:
    rc = solve_drying("app3", T_MAX_CAP_S, stop=("first_passage", C_THRESHOLD),
                dt_solve=dtc, dt_out=3600.0, N=N_CELLS, air_mode="linear",
                C_init=C0, T_init=T0_C, record_fields=False)
    dv.append(float(rc["t_end"]) / 3600.0)
    print(f" dt={dtc}: t_end={dv[-1]:.6f} h", flush=True)
out["convergence"] = {"N_list": list(N_CONVERGENCE), "dt_list":
    list(DT_CONVERGENCE_Q34),
                "grid_t_end_h": convergence_ratios(gv),
                "dt_t_end_h": convergence_ratios(dv)}
for kk in ("grid_t_end_h", "dt_t_end_h"):
    rc_ = abs(out["convergence"][kk]["rel_change"][-1])
    out["convergence"][kk]["last_rel_change_pct"] = rc_ * 100.0
    assert rc_ < 5e-3, f"{kk} 末档相对变化 {rc_ * 100:.4f}% 超 0.5%"
print(f"[Q3] 收敛：网格末档
    {out['convergence']['grid_t_end_h']['last_rel_change_pct']:.4f}%，"
    f"步长末档 {out['convergence']['dt_t_end_h']['last_rel_change_pct']:.4f}%",
        flush=True)


# 末行单独写入 t_end 状态。
col_cm = output_columns_cm(21, 0.1)
C_field = extract_output_nodes(res["C_out"], OUT_STRIDE_SPACE)
t_out = res["t_out"]
keep = t_out <= t_end_s + Q3_DT_OUT
dfC = make_field_df(t_out[keep], col_cm, C_field[keep])
xlsx = USER_DATA / "result3.xlsx"
write_xlsx(xlsx, {"水分浓度": dfC})
out["result_file"] = str(xlsx.name)
out["result_shape"] = [int(dfC.shape[0]), int(dfC.shape[1])]
out["result_last_time_s"] = float(t_out[keep][-1])
```

```
    print(f"[Q3] 写出 {xlsx.name} shape={dfC.shape} 末行 t={out['result_last_time_s']:.0f}s",
          flush=True)


    tab_ci = [col_cm.index(c) for c in (0.0, 0.5, 1.0, 1.5, 2.0)]
    tab_t_h = list(np.arange(0.0, t_end_h, Q3_TABLE_DT_H))
    rows = []
    for th in tab_t_h:
        ts = th * 3600.0
        if ts <= 0.0:
            vals = [float(C0)] * len(tab_ci)
        else:
            j = int(np.argmin(np.abs(t_out - ts)))
            assert abs(t_out[j] - ts) < 1e-6, f"表格时刻 {ts} 不在输出层"
            vals = [float(C_field[j][c]) for c in tab_ci]
        rows.append({"t_h": float(th), "values": vals})



    rows.append({"t_h": t_end_h, "values": [float(C_end[c * OUT_STRIDE_SPACE])
                                            for c in tab_ci], "is_t_end": True})
    out["paper_table_radii_cm"] = [0.0, 0.5, 1.0, 1.5, 2.0]
    out["paper_table_rows"] = rows
    out["paper_table_last_row_max_C"] = float(np.max(rows[-1]["values"]))
    assert out["paper_table_last_row_max_C"] < C_THRESHOLD, "末行（t_end）maxC 未达标"

    _gap = abs(out["paper_table_last_row_max_C"] - out["at_t_end"]["max_C"])
    assert _gap < 1e-6, (f"末行 maxC={out['paper_table_last_row_max_C']:.8f} 与 "
                         f"at_t_end maxC={out['at_t_end']['max_C']:.8f} 差 {_gap:.2e}，"
                         "末行取的不是 t_end 处的场")

    chk = pd.read_excel(xlsx, sheet_name="水分浓度")
    j6 = int(np.argmin(np.abs(t_out[keep] - 6.0 * 3600.0)))
    worst = max(abs(float(chk.iloc[j6, c + 1]) - round(float(C_field[keep][j6][c]), DECIMALS))
                for c in tab_ci)
    out["table_vs_file_max_abs_diff"] = worst
    assert worst == 0.0, f"表格与文件不一致 {worst}"

    out["wall_time_s"] = time.time() - t0_wall
    FIGDIR.mkdir(exist_ok=True)
    with open(FIGDIR / f"{TAG}_results.json", "w", encoding="utf-8") as fh:
        json.dump(out, fh, ensure_ascii=False, indent=2, default=float)
    print(f"[Q3] -> figures/{TAG}_results.json ({out['wall_time_s']:.1f} s)", flush=True)
    return out

```

```

if __name__ == "__main__":
    run()
```

1.7 问题四

```

from __future__ import annotations

import json
import time

import numpy as np
import pandas as pd

from params import (AIR_EXTRAP_C, AIR_EXTRAP_T_C, AIR_T_END_S, C0, C_THRESHOLD, DECIMALS,
                DT_CONVERGENCE_Q34, DT_SOLVE_Q34, FIGDIR, HM_MASS, KELVIN_OFFSET,
                N_CELLS, N_CONVERGENCE, OUT_STRIDE_SPACE, Q4_DT_OUT, Q4_TABLE_DT_H,
                R0, RADIUS_T_END_S, SURFACE_COL_NAME, T0_C, T_MAX_CAP_S, USER_DATA,
                INDEX_HEADER)
from props import D_app3, D_app4, props_app4
from utils import (convergence_ratios, extract_output_nodes, grid_weights, load_radius,
               output_columns_cm, radius_interpolant, solve_drying, volume_average,
               write_xlsx)

TAG = "problem_4"


def run():
    t0_wall = time.time()
    out = {"problem": 4, "property_set": "appendix4", "coupling": "two_way",
           "geometry": "ALE moving boundary x=r/R(t), R from 附件2 (PCHIP)"}


    r0_, c0_, k0_ = props_app4(C0)
    out["props_at_C0"] = {"rho": float(r0_), "cp": float(c0_), "k": float(k0_)}
    assert abs(float(c0_) - 3394.3662) < 1e-3, f"cp4(2.55) 应 3394.3662，得
        {float(c0_):.4f}"
    assert abs(float(k0_) - 0.2636620) < 1e-6, f"k4(2.55) 应 0.2636620，得 {float(k0_):.7f}"
    out["capability_checklist_deviation"] = {
        "P4-C2(c)_expects_cp": 3395.8, "corrected_cp": float(c0_),
        "P4-C2(c)_expects_k": 0.2634, "corrected_k": float(k0_),
        "note": "CAPABILITY_CHECKLIST 携带 §0 修正前的上游值；第二项自变量应为 C/(C+1)=51/71"}
    TK50 = 50.0 + KELVIN_OFFSET
    D4 = float(D_app4(C0, TK50)); D3 = float(D_app3(C0, TK50))
```

```
    out["D_app4_at_C0_50C"] = D4
    out["D_ratio_app4_over_app3"] = D4 / D3
    assert 0.15 <= D4 / D3 <= 0.25, f"P4-C2(d) D4/D3={D4 / D3:.5f} 越出 [0.15,0.25]"
    print(f"[Q4] props@C0 cp={float(c0_):.4f} k={float(k0_):.7f}; D4/D3={D4 / D3:.5f}",
        flush=True)


    # 半径数据进入 ALE 网格。
    R_of, Rdot_of, meta = radius_interpolant()
    assert abs(float(R_of(0.0)) - 0.02) < 1e-9, "R(0) 应为 2.000 cm"
    assert abs(float(R_of(RADIUS_T_END_S)) - 0.01198) < 1e-9, "R(259200) 应为 1.198 cm"
    tq = np.linspace(0.0, RADIUS_T_END_S, 200001)
    Rq = np.asarray(R_of(tq), dtype=float)
    assert np.all(np.diff(Rq) <= 1e-15), "PCHIP 插值不单调不增"
    assert Rq.min() >= 0.01198 - 1e-12 and Rq.max() <= 0.02 + 1e-12, "PCHIP 冲出实测界"
    Rdotq = np.asarray(Rdot_of(tq), dtype=float)
    Rdot_max = float(np.max(np.abs(Rdotq)))
    Pe_shr = Rdot_max * R0 / D4
    out["radius"] = {**meta, "ptp_cm": meta["ptp_cm"],
                "monotone_nonincreasing": True,
                "min_cm": float(Rq.min() * 100.0), "max_cm": float(Rq.max() * 100.0),
                "Rdot_max_abs_m_s": Rdot_max,
                "t_at_Rdot_max_s": float(tq[int(np.argmax(np.abs(Rdotq)))]),
                "diffusion_velocity_D_over_R0_m_s": D4 / R0,
                "Pe_shrink": Pe_shr,
                "area_ratio_R0_over_Rend_sq": (0.02 / 0.01198) ** 2,
                "shrink_pct": (0.02 - 0.01198) / 0.02 * 100.0}
    assert meta["ptp_cm"] > 0.5, "P4-C1(a) 收缩幅度须 >0.5 cm"
    assert Pe_shr > 1.0, f"Pe_shr={Pe_shr:.4f} 应 >1（收缩不是小扰动）"
    print(f"[Q4] Pe_shr={Pe_shr:.4f} |Rdot|max={Rdot_max:.6e} m/s 面积比="
         f"{out['radius']['area_ratio_R0_over_Rend_sq']:.4f}", flush=True)


    # 主解
    print(f"[Q4] ALE 主求解 N={N_CELLS} dt={DT_SOLVE_Q34}", flush=True)
    res = solve_drying("app4", T_MAX_CAP_S, stop=("first_passage", C_THRESHOLD),
                  geometry=("ale", (R_of, Rdot_of)), dt_solve=DT_SOLVE_Q34,
                  dt_out=Q4_DT_OUT, N=N_CELLS, air_mode="linear", C_init=C0,
                  T_init=T0_C, record_fields=True, rebound_check=True, progress=20000)
    assert res["t_end"] is not None,\
       f"上限内未达标，末态 maxC={float(np.max(res['C_final'])):.6f}"
    t_end_s = float(res["t_end"]); t_end_h = t_end_s / 3600.0
    print(f"[Q4] t_end = {t_end_s:.4f} s = {t_end_h:.4f} h = {t_end_h / 24:.4f} d "
         f"({res['n_steps']} 步 {time.time() - t0_wall:.1f} s)", flush=True)

    C_end = res["C_at_t_end"]; T_end = res["T_at_t_end"]
    R_end_m = float(R_of(t_end_s))
```

```
    _, _, V_end, _ = grid_weights(R_end_m, N_CELLS)
    out.update({"t_end_s": t_end_s, "t_end_h": round(t_end_h, DECIMALS),
             "t_end_h_full": t_end_h, "t_end_days": t_end_h / 24.0,
             "R_at_t_end_cm": R_end_m * 100.0,
             "bracket_s": [float(res["bracket"][0]), float(res["bracket"][1])],
             "no_rebound_to_1.05t_end": bool(res["no_rebound"]),
             "argmax_center_strict": bool(res["argmax_center_always"]),
             "argmax_center_within_tol": bool(res["argmax_center_within_tol"]),
             "C_hump_excess_max": res["C_hump_excess_max"],
             "final_profile_max_diff": float(np.max(np.diff(res["C_final"]))),
             "motion_terms_cancel_resid_max": res["motion_resid_max"],
             "v_mesh_active_step_fraction": res["v_mesh_active_steps"] /
                 max(res["n_steps"], 1),
             "n_steps": res["n_steps"], "dt_solve_s": DT_SOLVE_Q34,
             "dt_out_s": Q4_DT_OUT, "N_cells": N_CELLS,
             "surface_form": res["surface_form"],
             "water_resid_max": res["water_resid_max"],
             "n_picard_max": res["n_picard_max"],
             "D_field_ratio_max": res["D_field_ratio_max"],
             "at_t_end": {"max_C": float(np.max(C_end)),
                      "C_bar": volume_average(C_end, V_end),
                      "C_center": float(C_end[0]), "C_surface": float(C_end[-1]),
                      "T_center_C": float(T_end[0]),
                      "g_at_t_end": float(np.max(C_end)) - C_THRESHOLD}})
    assert res["argmax_center_within_tol"],\
       f"出现真实内部驼峰 {res['C_hump_excess_max']:.3e}：须如实报告"
    assert res["no_rebound"], "1.05*t_end 内 g 回升"
    assert out["at_t_end"]["max_C"] < C_THRESHOLD, "t_end 处 maxC 未达标"

    assert res["motion_resid_max"] < 1e-12,\
       f"运动项抵消残差 {res['motion_resid_max']:.3e} 未达 1e-12"




    # 检查网格运动项是否与实测收缩时序一致。
    span_integrated = res["n_steps"] * DT_SOLVE_Q34
    t_meas, R_meas = load_radius()
    seg_move = np.abs(np.diff(R_meas)) > 1e-15
    seg_span = np.clip(np.minimum(t_meas[1:], span_integrated) - t_meas[:-1], 0.0, None)

    denom = span_integrated
    frac_expected = float(seg_span[seg_move].sum() / denom)
    out["v_mesh_span_integrated_s"] = span_integrated
    frac_got = out["v_mesh_active_step_fraction"]
    out["v_mesh_active_fraction_expected_from_data"] = frac_expected
```

```
    out["v_mesh_active_fraction_abs_diff"] = abs(frac_got - frac_expected)
    assert abs(frac_got - frac_expected) < 1e-3,\
       f"P4-C1(b)：网格运动活跃占比 {frac_got:.6f} 与实测非平台时长占比 "\
       f"{frac_expected:.6f} 不符，收缩未按实测时序进入求解"

    R_at_end = float(R_of(t_end_s))
    out["R_at_t_end_cm"] = R_at_end * 100.0
    out["R_shrink_during_solve_cm"] = (R0 - R_at_end) * 100.0
    assert R0 - R_at_end > 5e-3, f"求解期内半径仅缩 {(R0 - R_at_end) * 100:.4f} cm"
    print(f"[Q4] 运动项抵消残差={res['motion_resid_max']:.3e} 活跃步占比="
         f"{frac_got:.6f}（实测非平台占比 {frac_expected:.6f}）"
         f" R(t_end)={R_at_end * 100:.4f} cm", flush=True)



    # 冻结半径作为对照。
    print("[Q4] P4-C1(c) 冻结半径对照 R≡2 cm（上限放宽到 3e6 s）...", flush=True)
    CAP_FROZEN_S = 3.0e6
    res_fix = solve_drying("app4", CAP_FROZEN_S, stop=("first_passage", C_THRESHOLD),
                    t_cap=CAP_FROZEN_S, geometry=("fixed", R0),
                    dt_solve=DT_SOLVE_Q34, dt_out=3600.0,
                    N=N_CELLS, air_mode="linear", C_init=C0, T_init=T0_C,
                    record_fields=False, rebound_check=False, progress=100000)
    assert res_fix["t_end"] is not None,\
       f"冻结半径算例在 {CAP_FROZEN_S:.0e} s 内未达标，末态 maxC="\
       f"{float(np.max(res_fix['C_final'])):.6f}"
    t_fix_h = float(res_fix["t_end"]) / 3600.0
    rel_shrink = abs(t_end_h - t_fix_h) / t_fix_h
    out["P4C1c_frozen_radius"] = {
       "t_end_frozen_R_h": t_fix_h, "t_end_ale_h": t_end_h,
       "rel_change": rel_shrink, "rel_change_pct": rel_shrink * 100.0,
       "shrink_shortens_drying": bool(t_end_h < t_fix_h),
       "cap_used_s": CAP_FROZEN_S,
       "note": "冻结半径下 t_end 超出 §6.3 的 120 h 时窗，说明收缩不是小扰动"
             "而是干燥能在 5 天内完成的必要条件"}
    assert rel_shrink > 0.03,\
       f"P4-C1(c) 失败：收缩只改变 t_end {rel_shrink * 100:.4f}%（<3%），收缩未进入求解"
    print(f"[Q4] 冻结 R: t_end={t_fix_h:.4f} h vs ALE {t_end_h:.4f} h "
         f"→ 相对变化 {rel_shrink * 100:.4f}%", flush=True)



    try:
       with open(FIGDIR / "problem_3_results.json", "r", encoding="utf-8") as fh:
          t3_h = float(json.load(fh)["t_end_h_full"])
    except Exception:
       t3_h = None
```

```
out["vs_problem3"] = {
    "t_end_q3_h": t3_h, "t_end_q4_h": t_end_h,
    "ratio_q4_over_q3": (t_end_h / t3_h) if t3_h else None,
    "diff_h": (t_end_h - t3_h) if t3_h else None,
    "competing_mechanisms": {
        "D4_over_D3_at_C0_50C": D4 / D3,
        "geometric_enhancement_area_ratio": out["radius"]["area_ratio_R0_over_Rend_sq"]}}
if t3_h:
    print(f"[Q4] vs 问题三：{t_end_h:.4f} h vs {t3_h:.4f} h "
          f"(比 {t_end_h / t3_h:.4f})", flush=True)


out["P4C2e_no_inversion"] = {
    "radius_plateau_start_h": meta["plateau_start_s"] / 3600.0,
    "t_end_h": t_end_h,
    "gap_from_plateau_h": abs(t_end_h - meta["plateau_start_s"] / 3600.0),
    "note": "附件2 首次达 1.198 cm 的 67.0 h 与干燥终点无因果关系；"
            "t_end 由水分方程积分产生，附件2 只经 1/R(t)^2 进入几何因子"}
out["radius_extrapolation"] = {
    "radius_data_covers_until_s": RADIUS_T_END_S,
    "t_end_beyond_radius_data": bool(t_end_s > RADIUS_T_END_S),
    "extrapolated_fraction_pct": max(0.0, (1.0 - RADIUS_T_END_S / t_end_s) * 100.0),
    "mode": "hold R=1.198 cm (禁止趋势外推：末段已进入平台，外推会给出负半径方向)"}
out["air_extrapolation"] = {
    "measured_until_s": AIR_T_END_S, "hold_T_air_C": AIR_EXTRAP_T_C,
    "hold_C_air": AIR_EXTRAP_C,
    "extrapolated_fraction_pct": (1.0 - AIR_T_END_S / t_end_s) * 100.0}


# 无收缩时，两种实现应退化为同一结果。
print("[Q4] 拉格朗日对照：R≡2 cm 下 ALE 分支 vs 固定域分支 ...", flush=True)
res_ale_frozen = solve_drying(
    "app4", 7200.0, stop=("time", 7200.0),
    geometry=("ale", (lambda tt: np.full_like(np.asarray(tt, dtype=float), R0),
                      lambda tt: np.zeros_like(np.asarray(tt, dtype=float)))),
    dt_solve=DT_SOLVE_Q34, dt_out=7200.0, N=N_CELLS, air_mode="linear",
    C_init=C0, T_init=T0_C, record_fields=False)
res_fixed_short = solve_drying("app4", 7200.0, stop=("time", 7200.0),
                               geometry=("fixed", R0), dt_solve=DT_SOLVE_Q34,
                               dt_out=7200.0, N=N_CELLS, air_mode="linear",
                               C_init=C0, T_init=T0_C, record_fields=False)
d_impl = float(np.max(np.abs(res_ale_frozen["C_final"] - res_fixed_short["C_final"])))
out["no_shrink_two_implementations_max_dC"] = d_impl
assert d_impl < 1e-8, f"无收缩时两实现不一致 dC={d_impl:.3e}"
print(f"[Q4] 无收缩两实现一致 dC={d_impl:.3e}", flush=True)
```

```
rho_d0 = float(props_app4(C0)[0]) / (1.0 + C0)
R_pred_end = R0 * np.sqrt(rho_d0 / (float(props_app4(C_THRESHOLD)[0]) / (1.0 + C_THRESHOLD)))
out["dry_mass_consistency"] = {
   "rho_dry_at_C0_kg_m3": rho_d0,
   "rho_dry_at_Cthr_kg_m3": float(props_app4(C_THRESHOLD)[0]) / (1.0 + C_THRESHOLD),
   "R_predicted_end_cm": float(R_pred_end * 100.0),
   "R_measured_end_cm": 1.198,
   "rel_gap_pct": float(abs(R_pred_end * 100.0 - 1.198) / 1.198 * 100.0),
   "note": "拉格朗日模型由干物质守恒自行预测 R(t)，与实测差异是题面数据内在不一致"
        "（DATA_FACTS F1），非实现错误；ALE 主模型中 rho_d 被完全消去，不受影响"}
print(f"[Q4] 干物质诊断：R_pred={R_pred_end * 100:.4f} cm vs 实测 1.198 cm "
     f"(差 {out['dry_mass_consistency']['rel_gap_pct']:.4f}%)", flush=True)


print("[Q4] t_end 网格收敛 N=100/200/400/800 ...", flush=True)
gv = []
for Nc in N_CONVERGENCE:
   rc = solve_drying("app4", T_MAX_CAP_S, stop=("first_passage", C_THRESHOLD),
                geometry=("ale", (R_of, Rdot_of)), dt_solve=DT_SOLVE_Q34,
                dt_out=3600.0, N=Nc, air_mode="linear", C_init=C0, T_init=T0_C,
                record_fields=False)
   gv.append(float(rc["t_end"]) / 3600.0)
   print(f" N={Nc}: t_end={gv[-1]:.6f} h", flush=True)
print("[Q4] t_end 步长收敛 dt=8/4/2/1 ...", flush=True)
dv = []
for dtc in DT_CONVERGENCE_Q34:
   rc = solve_drying("app4", T_MAX_CAP_S, stop=("first_passage", C_THRESHOLD),
                geometry=("ale", (R_of, Rdot_of)), dt_solve=dtc, dt_out=3600.0,
                N=N_CELLS, air_mode="linear", C_init=C0, T_init=T0_C,
                record_fields=False)
   dv.append(float(rc["t_end"]) / 3600.0)
   print(f" dt={dtc}: t_end={dv[-1]:.6f} h", flush=True)
out["convergence"] = {"N_list": list(N_CONVERGENCE), "dt_list":
    list(DT_CONVERGENCE_Q34),
                "grid_t_end_h": convergence_ratios(gv),
                "dt_t_end_h": convergence_ratios(dv)}
for kk in ("grid_t_end_h", "dt_t_end_h"):
   rc_ = abs(out["convergence"][kk]["rel_change"][-1])
   out["convergence"][kk]["last_rel_change_pct"] = rc_ * 100.0
   assert rc_ < 5e-3, f"{kk} 末档相对变化 {rc_ * 100:.4f}% 超 0.5%"
print(f"[Q4] 收敛：网格末档
    {out['convergence']['grid_t_end_h']['last_rel_change_pct']:.4f}%，"
     f"步长末档 {out['convergence']['dt_t_end_h']['last_rel_change_pct']:.4f}%",
         flush=True)


```

```
    # 固定半径列从 ALE 坐标插值到物理半径。
    C_field_x = res["C_out"]
    t_out = res["t_out"]
    R_out = res["R_out"]
    keep = t_out <= t_end_s + Q4_DT_OUT
    t_keep, C_keep, R_keep = t_out[keep], C_field_x[keep], R_out[keep]
    col_cm = output_columns_cm(20, 0.1)
    n_rows = t_keep.shape[0]
    data = np.full((n_rows, len(col_cm)), np.nan)
    x_nodes = np.linspace(0.0, 1.0, N_CELLS + 1)
    for i in range(n_rows):
       Rc = float(R_keep[i]) * 100.0
       for j, rc_cm in enumerate(col_cm):
          if rc_cm > Rc + 1e-12:
             continue
          xj = (rc_cm / 100.0) / float(R_keep[i])
          kj = int(round(xj * N_CELLS))
          assert abs(kj / N_CELLS - xj) < 5e-3, "固定 r 列未落在 ALE 节点附近"
          data[i, j] = C_keep[i][min(kj, N_CELLS)]
    surf = np.array([C_keep[i][-1] for i in range(n_rows)], dtype=float)
    dfC = pd.DataFrame(data, columns=[str(c) for c in col_cm])
    dfC[SURFACE_COL_NAME] = surf
    dfC.insert(0, INDEX_HEADER, t_keep.astype(float))
    xlsx = USER_DATA / "result4.xlsx"
    write_xlsx(xlsx, {"水分浓度": dfC})
    n_blank_last = int(np.sum(np.isnan(data[-1])))
    out["result_file"] = str(xlsx.name)
    out["result_shape"] = [int(dfC.shape[0]), int(dfC.shape[1])]
    out["result_last_time_s"] = float(t_keep[-1])
    out["result_last_column_name"] = SURFACE_COL_NAME
    out["blank_cells_total"] = int(np.sum(np.isnan(data)))
    out["blank_cols_in_last_row"] = n_blank_last
    out["R_at_last_row_cm"] = float(R_keep[-1]) * 100.0
    print(f"[Q4] 写出 {xlsx.name} shape={dfC.shape} 末列='{SURFACE_COL_NAME}' "
         f"末行留空 {n_blank_last} 列 (R={out['R_at_last_row_cm']:.4f} cm)", flush=True)


    tab_r_cm = [0.0, 0.5, 1.0, 1.5]
    rows = []
    for th in np.arange(0.0, t_end_h, Q4_TABLE_DT_H):
       ts = th * 3600.0
       Rc_cm = float(R_of(ts)) * 100.0
       if ts <= 0.0:
          vals = [float(C0)] * len(tab_r_cm); surf_v = float(C0)
       else:
          j = int(np.argmin(np.abs(t_keep - ts)))
          vals = []
```

```
        for rc_cm in tab_r_cm:
            if rc_cm > Rc_cm + 1e-12:
                vals.append(None)
            else:
                kj = int(round((rc_cm / 100.0) / float(R_keep[j]) * N_CELLS))
                vals.append(float(C_keep[j][min(kj, N_CELLS)]))
        surf_v = float(C_keep[j][-1])
      rows.append({"t_h": float(th), "R_cm": Rc_cm, "values": vals, "surface": surf_v})
   R_te_cm = R_end_m * 100.0
   vals_te = []
   for rc_cm in tab_r_cm:
      if rc_cm > R_te_cm + 1e-12:
         vals_te.append(None)
      else:
         kj = int(round((rc_cm / 100.0) / R_end_m * N_CELLS))
         vals_te.append(float(C_end[min(kj, N_CELLS)]))
   rows.append({"t_h": t_end_h, "R_cm": R_te_cm, "values": vals_te,
             "surface": float(C_end[-1]), "is_t_end": True})
   out["paper_table_radii_cm"] = tab_r_cm
   out["paper_table_last_column"] = SURFACE_COL_NAME
   out["paper_table_rows"] = rows
   present = [v for v in vals_te + [float(C_end[-1])] if v is not None]
   out["paper_table_last_row_max_C"] = float(max(present))
   assert out["paper_table_last_row_max_C"] < C_THRESHOLD, "末行（t_end）maxC 未达标"

   t_R15 = float(tq[int(np.argmax(Rq <= 0.015))])
   out["r_1p5cm_column_blank_from_h"] = t_R15 / 3600.0

   out["wall_time_s"] = time.time() - t0_wall
   FIGDIR.mkdir(exist_ok=True)
   with open(FIGDIR / f"{TAG}_results.json", "w", encoding="utf-8") as fh:
      json.dump(out, fh, ensure_ascii=False, indent=2, default=float)
   print(f"[Q4] -> figures/{TAG}_results.json ({out['wall_time_s']:.1f} s)", flush=True)
   return out


if __name__ == "__main__":
   run()
```

1.8 问题四全局不确定性分析

```

from __future__ import annotations

import argparse
```

```
import csv
import json
import math
import time

import numpy as np

from params import (
   C0,
   C_THRESHOLD,
   FIGDIR,
   H_CONV,
   HM_MASS,
   R0,
   T0_C,
)
from props import PROPERTY_SETS, to_kelvin
from utils import (
   D_face,
   air_driving,
   assemble_solve,
   k_face,
   radius_interpolant,
)


OUTDIR = FIGDIR / "enhanced_q4"
OUTDIR.mkdir(parents=True, exist_ok=True)

FEATURES = (
   "T_plateau_C",
   "C_plateau",
   "hm_factor",
   "eta_nonuniform",
   "gamma_timing",
)
BOUNDS = {
   "T_plateau_C": (49.0, 51.0),
   "C_plateau": (0.045, 0.055),
   "hm_factor": (0.5, 1.5),
   "eta_nonuniform": (-0.6, 0.6),
   "gamma_timing": (0.5, 2.0),
}

DEFAULTS = {
   "seed": 20260912,
   "samples": 128,
```

```
    "explain": 48,
    "background": 24,
    "N": 48,
    "dt": 120.0,
    "cap": 3.0e6,
}


# 结果写 JSON 前转成基础类型。
def _jsonable(value):
    if isinstance(value, dict):
        return {str(k): _jsonable(v) for k, v in value.items()}
    if isinstance(value, (list, tuple)):
        return [_jsonable(v) for v in value]
    if isinstance(value, np.ndarray):
        return value.tolist()
    if isinstance(value, (np.floating, np.integer, np.bool_)):
        return value.item()
    return value


def write_json(name, value):
    path = OUTDIR / name
    path.write_text(
        json.dumps(_jsonable(value), ensure_ascii=False, indent=2),
        encoding="utf-8",
    )
    return path


def sample_inputs(rng, n):
    u = rng.random((n, len(FEATURES)))
    x = np.empty_like(u)
    for j, name in enumerate(FEATURES):
        lo, hi = BOUNDS[name]
        x[:, j] = lo + (hi - lo) * u[:, j]
    return x


def feature_dict(x):
    return {name: float(x[j]) for j, name in enumerate(FEATURES)}


def geometry(t, radius_of, eta, gamma, n, radius_end):
    # eta 控制径向不均匀，gamma 控制出现时机。
    R = float(radius_of(t))
    x = np.linspace(0.0, 1.0, n + 1)
```

```
    q = max(0.0, min(1.0, 1.0 - R / R0))
    q_end = max(1.0e-14, 1.0 - radius_end / R0)
    amp = (q / q_end) ** gamma if q > 0.0 else 0.0
    phi = x + eta * amp * x * (1.0 - x)
    jac = 1.0 + eta * amp * (1.0 - 2.0 * x)
    if np.min(jac) <= 0.0:
        raise ValueError("nonuniform mapping is not monotone")
    r = R * phi
    r[-1] = R
    faces = 0.5 * (r[:-1] + r[1:])
    V = np.empty(n + 1, dtype=float)
    V[0] = faces[0] ** 2 / 2.0
    V[1:n] = (faces[1:] ** 2 - faces[:-1] ** 2) / 2.0
    V[n] = (R**2 - faces[-1] ** 2) / 2.0
    return r, V, faces, R, jac


def make_solver(x, *, n, dt, cap_s):
    params = feature_dict(x)
    radius_of, _, radius_meta = radius_interpolant()
    radius_end = float(radius_meta["R_end_m"])
    T_air, C_air = air_driving(
        params["T_plateau_C"], params["C_plateau"], mode="linear"
    )
    pset = PROPERTY_SETS["app4"]
    props_fn, D_fn = pset["props"], pset["D"]
    hm = HM_MASS * params["hm_factor"]
    eta = params["eta_nonuniform"]
    gamma = params["gamma_timing"]

    def step(C_old, T_old, t_old, h):
        t_new = t_old + h
        r, V, faces, R, _ = geometry(
            t_new, radius_of, eta, gamma, n, radius_end
        )
        dr_face = np.diff(r)
        Ta = float(T_air(t_new))
        Ca = float(C_air(t_new))
        C_iter, T_iter = C_old.copy(), T_old.copy()
        for _ in range(6):
            T_face = to_kelvin(0.5 * (T_iter[:-1] + T_iter[1:]))
            D_face_now = D_face(C_iter, pset, T_face, mode="kirchhoff")
            D_surface = float(D_fn(C_iter[-1], to_kelvin(T_iter[-1])))
            rho, cp, conductivity = props_fn(C_iter)
            thermal_capacity = rho * cp
            conductivity_face = k_face(conductivity)
            C_new = assemble_solve(
```

```
            C_old,
            np.ones_like(C_old),
            V,
            faces,
            D_face_now / dr_face,
            h,
            hm,
            Ca,
            R,
        )
        T_new = assemble_solve(
            T_old,
            thermal_capacity,
            V,
            faces,
            conductivity_face / dr_face,
            h,
            H_CONV,
            Ta,
            R,
        )
        if (
            np.max(np.abs(C_new - C_iter)) < 1.0e-9
            and np.max(np.abs(T_new - T_iter)) < 1.0e-7
        ):
            C_iter, T_iter = C_new, T_new
            break
        C_iter, T_iter = C_new, T_new
    return C_iter, T_iter, R

def solve():
    C = np.full(n + 1, C0, dtype=float)
    T = np.full(n + 1, T0_C, dtype=float)
    t = 0.0
    previous_g = C0 - C_THRESHOLD
    first_bracket = None
    while t < cap_s - 1.0e-12:
        h = min(dt, cap_s - t)
        C_before, T_before, t_before = C, T, t
        C, T, R = step(C_before, T_before, t, h)
        t += h
        g = float(np.max(C)) - C_THRESHOLD
        if previous_g >= 0.0 > g:
            first_bracket = (t_before, t)
            break
        previous_g = g
    if first_bracket is None:
```

```
            return None

        lo, hi = 0.0, first_bracket[1] - first_bracket[0]
        sub = max(dt / 8.0, 1.0)
        while hi - lo > 2.0:
            mid = 0.5 * (lo + hi)
            C_mid, T_mid = C_before.copy(), T_before.copy()
            tt = first_bracket[0]
            while tt < first_bracket[0] + mid - 1.0e-12:
                hh = min(sub, first_bracket[0] + mid - tt)
                C_mid, T_mid, _ = step(C_mid, T_mid, tt, hh)
                tt += hh
            if float(np.max(C_mid)) < C_THRESHOLD:
                hi = mid
            else:
                lo = mid
        t_end = first_bracket[0] + 0.5 * (lo + hi)
        return {
            "t_end_s": float(t_end),
            "t_end_h": float(t_end / 3600.0),
            "R_at_t_end_cm": float(radius_of(t_end) * 100.0),
            "eta": float(eta),
            "gamma": float(gamma),
            "N": int(n),
            "dt_solve_s": float(dt),
            "hm_factor": float(params["hm_factor"]),
        }

    return solve


def evaluate(x, **solver_options):
    result = make_solver(x, **solver_options)()
    row = feature_dict(x)
    if result is None:
        row.update({"status": "unreached", "t_end_h": None})
    else:
        row.update({"status": "reached", **result})
    return row


def geometry_tests(n):
    # 先检查坐标映射再跑随机样本。
    radius_of, _, meta = radius_interpolant()
    tests = []
    for eta in (-0.6, 0.0, 0.6):
        for gamma in (0.5, 1.0, 2.0):
```

```
        jac_min = 1.0
        boundary_error = 0.0
        for t in np.linspace(0.0, min(RADIUS_TEST_CAP, DEFAULTS["cap"]), 121):
            r, _, _, R, jac = geometry(
                t, radius_of, eta, gamma, n, float(meta["R_end_m"])
            )
            jac_min = min(jac_min, float(np.min(jac)))
            boundary_error = max(boundary_error, abs(float(r[-1] - R)))
        tests.append(
            {
                "eta": eta,
                "gamma": gamma,
                "min_mapping_jacobian": jac_min,
                "max_boundary_error_m": boundary_error,
                "monotone": bool(jac_min > 0.0),
            }
        )
    return {"radius_meta": meta, "tests": tests}


RADIUS_TEST_CAP = 259200.0


def run_monte_carlo(settings):
    # Monte Carlo 只对能达到阈值的样本统计。
    rng = np.random.default_rng(settings["seed"])
    X = sample_inputs(rng, settings["samples"])
    rows = []
    start = time.time()
    for i, x in enumerate(X):
        row = evaluate(
            x,
            n=settings["N"],
            dt=settings["dt"],
            cap_s=settings["cap"],
        )
        row["sample_id"] = i
        rows.append(row)
        if (i + 1) % max(1, settings["samples"] // 8) == 0:
            print(f"[global-q4] Monte Carlo {i + 1}/{settings['samples']}", flush=True)

    reached = [row for row in rows if row["t_end_h"] is not None]
    y = np.asarray([row["t_end_h"] for row in reached], dtype=float)
    if y.size == 0:
        raise RuntimeError("Monte Carlo 样本全部未在上限内达标")
    result = {
        "method": "seeded uniform Monte Carlo",
```

```
        "seed": settings["seed"],
        "n_samples": settings["samples"],
        "n_reached": int(y.size),
        "features": list(FEATURES),
        "bounds": BOUNDS,
        "solver": {
            "N": settings["N"],
            "dt_solve_s": settings["dt"],
            "t_cap_s": settings["cap"],
        },
        "summary": {
            "mean_h": float(np.mean(y)),
            "std_h": float(np.std(y, ddof=1)) if y.size > 1 else 0.0,
            "q05_h": float(np.quantile(y, 0.05)),
            "q50_h": float(np.quantile(y, 0.50)),
            "q95_h": float(np.quantile(y, 0.95)),
            "min_h": float(np.min(y)),
            "max_h": float(np.max(y)),
        },
        "rows": rows,
        "wall_time_s": time.time() - start,
    }
    write_json("monte_carlo_results.json", result)
    _write_rows_csv("monte_carlo_results.csv", rows)
    return X, y, result


def polynomial_design(X):
    X = np.asarray(X, dtype=float)
    columns = [np.ones(X.shape[0])]
    columns.extend(X[:, j] for j in range(X.shape[1]))
    columns.extend(X[:, j] ** 2 for j in range(X.shape[1]))
    for i in range(X.shape[1]):
        for j in range(i + 1, X.shape[1]):
            columns.append(X[:, i] * X[:, j])
    return np.column_stack(columns)


def fit_surrogate(X, y, seed):
    rng = np.random.default_rng(seed)
    order = rng.permutation(len(y))
    split = max(8, int(0.8 * len(y)))
    train, test = order[:split], order[split:]
    x_min = np.array([BOUNDS[name][0] for name in FEATURES])
    x_max = np.array([BOUNDS[name][1] for name in FEATURES])
    scale = np.where(x_max > x_min, x_max - x_min, 1.0)
    X01 = (X - x_min) / scale
```

```
    Z = polynomial_design(X01)
    penalty = np.eye(Z.shape[1]) * 1.0e-8
    penalty[0, 0] = 0.0
    coef = np.linalg.solve(Z[train].T @ Z[train] + penalty, Z[train].T @ y[train])
    pred_test = Z[test] @ coef if test.size else Z[train] @ coef
    truth = y[test] if test.size else y[train]
    residual = pred_test - truth
    ss_tot = float(np.sum((truth - np.mean(truth)) ** 2))
    r2 = None if ss_tot == 0.0 else 1.0 - float(np.sum(residual**2)) / ss_tot
    return {
        "coef": coef,
        "x_min": x_min,
        "scale": scale,
        "metrics": {
            "model": "second-order ridge surrogate",
            "train_n": int(train.size),
            "test_n": int(test.size),
            "test_r2": r2,
            "test_rmse_h": float(np.sqrt(np.mean(residual**2))),
            "test_mae_h": float(np.mean(np.abs(residual))),
        },
    }


def surrogate_predict(model, X):
    X01 = (np.asarray(X, dtype=float) - model["x_min"]) / model["scale"]
    return polynomial_design(X01) @ model["coef"]


def exact_interventional_shap(model, X_explain, X_background):
    # 对二阶代理模型枚举全部特征子集。
    p = len(FEATURES)
    factorial = math.factorial
    phi = np.zeros((len(X_explain), p), dtype=float)
    base = float(np.mean(surrogate_predict(model, X_background)))
    for row_id, x in enumerate(X_explain):
        values = {}
        for mask in range(1 << p):
            samples = X_background.copy()
            for j in range(p):
                if mask & (1 << j):
                    samples[:, j] = x[j]
            values[mask] = float(np.mean(surrogate_predict(model, samples)))
        for j in range(p):
            for mask in range(1 << p):
                if mask & (1 << j):
                    continue
```

```
            size = int(mask.bit_count())
            weight = factorial(size) * factorial(p - size - 1) / factorial(p)
            phi[row_id, j] += weight * (
               values[mask | (1 << j)] - values[mask]
            )
   return phi, base


def run_shap(X, y, settings):
   model = fit_surrogate(X, y, settings["seed"] + 17)
   rng = np.random.default_rng(settings["seed"] + 55)
   n_exp = min(settings["explain"], len(y))
   n_bg = min(settings["background"], len(y))
   exp_idx = rng.choice(len(y), size=n_exp, replace=False)
   bg_idx = rng.choice(len(y), size=n_bg, replace=False)
   phi, base = exact_interventional_shap(model, X[exp_idx], X[bg_idx])
   pred = surrogate_predict(model, X[exp_idx])
   efficiency = pred - (base + np.sum(phi, axis=1))
   abs_mean = np.mean(np.abs(phi), axis=0)
   signed_mean = np.mean(phi, axis=0)
   order = np.argsort(-abs_mean)
   importance = [
      {
         "feature": FEATURES[int(j)],
         "mean_abs_shap_h": float(abs_mean[j]),
         "mean_signed_shap_h": float(signed_mean[j]),
         "rank": rank,
      }
      for rank, j in enumerate(order, 1)
   ]
   result = {
      "method": "exact interventional Shapley on second-order ridge surrogate",
      "interpretation": "归因对象是代理模型，不是原始 PDE 的因果效应",
      "features": list(FEATURES),
      "baseline_expected_t_end_h": base,
      "surrogate_metrics": model["metrics"],
      "n_explain": int(n_exp),
      "n_background": int(n_bg),
      "coalitions": 2 ** len(FEATURES),
      "global_importance": importance,
      "efficiency_residual": {
         "mean_abs_h": float(np.mean(np.abs(efficiency))),
         "max_abs_h": float(np.max(np.abs(efficiency))),
      },
      "rows": [
         {
            "sample_id": int(exp_idx[i]),
```

```
            "t_end_h": float(y[exp_idx[i]]),
            "shap_h": {
                FEATURES[j]: float(phi[i, j]) for j in range(len(FEATURES))
            },
        }
        for i in range(n_exp)
    ],
}
write_json("shap_results.json", result)
_write_rows_csv(
    "shap_results.csv",
    [
        {
            "sample_id": row["sample_id"],
            "t_end_h": row["t_end_h"],
            **row["shap_h"],
        }
        for row in result["rows"]
    ],
)
return result


def _write_rows_csv(name, rows):
    path = OUTDIR / name
    if not rows:
        path.write_text("", encoding="utf-8")
        return path
    with path.open("w", newline="", encoding="utf-8-sig") as fh:
        writer = csv.DictWriter(fh, fieldnames=list(rows[0]))
        writer.writeheader()
        writer.writerows(rows)
    return path


def try_plots(mc, shap):
    # 图形是附加输出，缺少 matplotlib 时不影响计算。
    try:
        import matplotlib

        matplotlib.use("Agg")
        import matplotlib.pyplot as plt
    except ImportError as exc:
        write_json("plot_status.json", {"available": False, "reason": str(exc)})
        return

    rows = [row for row in mc["rows"] if row["t_end_h"] is not None]
```

```
    y = np.asarray([row["t_end_h"] for row in rows])
    fig, ax = plt.subplots(figsize=(7.2, 4.2), dpi=180)
    ax.hist(y, bins=12, color="#4472C4", edgecolor="white")
    ax.axvline(np.mean(y), color="#C00000", lw=1.8, label=f"mean={np.mean(y):.2f} h")
    ax.axvline(np.quantile(y, 0.05), color="#70AD47", ls="--", label="5%")
    ax.axvline(np.quantile(y, 0.95), color="#70AD47", ls="--", label="95%")
    ax.set_xlabel("Q4 first-passage time (h)")
    ax.set_ylabel("Count")
    ax.legend(frameon=False)
    fig.tight_layout()
    fig.savefig(OUTDIR / "fig_mc_t_end_distribution.png", bbox_inches="tight")
    plt.close(fig)

    imp = shap["global_importance"]
    names = [row["feature"] for row in reversed(imp)]
    values = [row["mean_abs_shap_h"] for row in reversed(imp)]
    fig, ax = plt.subplots(figsize=(7.2, 4.2), dpi=180)
    ax.barh(names, values, color="#ED7D31")
    ax.set_xlabel("Mean absolute Shapley contribution (h)")
    fig.tight_layout()
    fig.savefig(OUTDIR / "fig_shap_global_importance.png", bbox_inches="tight")
    plt.close(fig)
    write_json("plot_status.json", {"available": True})


def main(argv=None):
    parser = argparse.ArgumentParser(description="Q4 global uncertainty analysis")
    parser.add_argument("--quick", action="store_true", help="少量样本快速检查")
    parser.add_argument("--samples", type=int, default=None)
    parser.add_argument("--seed", type=int, default=DEFAULTS["seed"])
    parser.add_argument("--no-plots", action="store_true")
    args = parser.parse_args(argv)

    settings = dict(DEFAULTS)
    settings["seed"] = args.seed
    if args.samples is not None:
        settings["samples"] = max(4, args.samples)
    if args.quick:
        settings.update({"samples": min(settings["samples"], 16),
                         "explain": 8, "background": 8,
                         "N": 24, "dt": 240.0})

    start = time.time()
    geometry_result = geometry_tests(settings["N"])
    write_json(
        "nonuniform_geometry_tests.json",
        {
```

```
        "mapping": "r=R(t)[x+eta*a(t)*x*(1-x)]",
        "eta_zero_expected_exact_reduction": True,
        "all_pass": all(
            item["monotone"] and item["max_boundary_error_m"] < 1.0e-14
            for item in geometry_result["tests"]
        ),
        **geometry_result,
    },
  )
  X, y, mc = run_monte_carlo(settings)
  shap = run_shap(X, y, settings)
  if not args.no_plots:
     try_plots(mc, shap)
  summary = {
     "files": [
        "nonuniform_geometry_tests.json",
        "monte_carlo_results.json",
        "monte_carlo_results.csv",
        "shap_results.json",
        "shap_results.csv",
     ],
     "settings": settings,
     "wall_time_s": time.time() - start,
     "mc_summary": mc["summary"],
     "shap_ranking": shap["global_importance"],
  }
  write_json("enhanced_q4_summary.json", summary)
  print(json.dumps(_jsonable(summary), ensure_ascii=False, indent=2), flush=True)
  return summary


if __name__ == "__main__":
  main()
```

1.9 结构性检验

```

from __future__ import annotations

import json
import numpy as np

from params import (C0, C_THRESHOLD, DECIMALS, FIGDIR, H_CONV, HM_MASS, KELVIN_OFFSET,
                K_APP2, CP_APP2, RHO_APP2, N_CELLS, R0, T0_C)
from props import (D_app2, D_app3, D_app4, Phi_app2, PROPERTY_SETS, UnitError,
```

```
                props_app2, props_app3, props_app4, to_kelvin)
from utils import (D_face, U_eff_series, air_driving, bessel_Bn, bessel_series_T,
                check_geometry_identity, grid_weights, radius_interpolant,
                solve_drying, volume_average)

RESULTS = {}


# 统一记录测试结果。
def _rec(name, passed, detail):
    RESULTS[name] = {"pass": bool(passed), **detail}
    flag = "PASS" if passed else "FAIL"
    print(f"[{flag}] {name}: {detail}", flush=True)
    if not passed:
        raise AssertionError(f"{name} 未通过：{detail}")


def S1_geometry_weights():
    # 体积权和应回到 R^2/2。
    r, dr, V, A_in = grid_weights(1.0, 4)
    tgt = {"V0": 0.0078125, "V1": 0.0625, "V4": 0.1171875, "sum": 0.5}
    got = {"V0": float(V[0]), "V1": float(V[1]), "V4": float(V[-1]), "sum": float(V.sum())}
    err = max(abs(got[k] - tgt[k]) for k in tgt)
    rel = check_geometry_identity(R0, N_CELLS)
    _rec("S1_geometry_weights", err < 1e-15 and rel < 1e-14,
        {"got": got, "max_abs_err": err, "sum_identity_rel_err_R0N200": rel})


def S2_zero_flux_constant():
    # 零通量时常场不能自行变化。
    res = solve_drying("app3", 600.0, stop=("time", 600.0), dt_solve=2.0, dt_out=600.0,
                    N=50, air_mode="const", T_plateau=T0_C, C_plateau=C0,
                    C_init=C0, T_init=T0_C, record_fields=False)
    dC = float(np.max(np.abs(res["C_final"] - C0)))
    dT = float(np.max(np.abs(res["T_final"] - T0_C)))
    _rec("S2_zero_flux_constant", dC < 1e-12 and dT < 1e-12,
        {"max_dC": dC, "max_dT": dT})


def _bessel_err(N, surface_form):
    rho, cp, k = props_app2()
    alpha = k / (rho * cp)
    Bi = H_CONV * R0 / k
    res = solve_drying("app2", 1800.0, stop=("time", 1800.0), dt_solve=0.125,
        dt_out=1800.0,
                    N=N, air_mode="const", T_plateau=50.0, C_plateau=C0,
                    C_init=C0, T_init=T0_C, record_fields=True,
```

```
                  surface_form=surface_form)
    r, dr, V, A_in = grid_weights(R0, N)
    T_ana, lam, Cn = bessel_series_T(r, 1800.0, Bi, alpha, R0, T0_C, 50.0, n_terms=60)
    return {"max_abs_err_K": float(np.max(np.abs(res["T_final"] - T_ana))),
            "T_center_num": float(res["T_final"][0]), "T_center_ana": float(T_ana[0]),
            "T_surf_num": float(res["T_final"][-1]), "T_surf_ana": float(T_ana[-1]),
            "Bi": Bi, "alpha": alpha, "lam_1to5": [float(x) for x in lam[:5]]}


def S3_bessel_limit():
    # 与圆柱 Bessel 级数比较。
    d = _bessel_err(400, "node")
    sB, sC, lam5 = bessel_Bn(Bi_of := H_CONV * R0 / props_app2()[2], 40)
    d.update({"sum_Bn": sB, "sum_Cn": sC, "Bi_check": Bi_of})
    _rec("S3_bessel_limit", d["max_abs_err_K"] < 1e-3 and abs(sB - 1.0) < 1e-5, d)


def S3b_surface_form_order():
    ev = {}
    for form in ("node", "series"):
        for N in (200, 400):
            ev[f"{form}_N{N}"] = _bessel_err(N, form)
    e_node = [ev[f"node_N{N}"]["max_abs_err_K"] for N in (200, 400)]
    e_ser = [ev[f"series_N{N}"]["max_abs_err_K"] for N in (200, 400)]
    node_flat = abs(e_node[1] - e_node[0]) / e_node[0] < 0.05
    ser_halves = 1.6 < e_ser[0] / e_ser[1] < 2.4
    _rec("S3b_surface_form_order", node_flat and ser_halves and e_node[1] < e_ser[1] / 5.0,
        {"err_node_N200_N400": e_node, "err_series_N200_N400": e_ser,
         "node_rel_change": abs(e_node[1] - e_node[0]) / e_node[0],
         "series_ratio_N200_over_N400": e_ser[0] / e_ser[1],
         "T_surf_ana": ev["node_N400"]["T_surf_ana"],
         "T_surf_node_N400": ev["node_N400"]["T_surf_num"],
         "T_surf_series_N400": ev["series_N400"]["T_surf_num"],
         "verdict": "node-centered FV must use bare h; series U_eff is the cell-centered
             form"})


def S4_Ueff_limits():
    dr = R0 / N_CELLS
    u1 = float(U_eff_series(1e30, dr / 2.0, HM_MASS))
    u2 = float(U_eff_series(5e-9, dr / 2.0, 1e30))
    e1 = abs(u1 - HM_MASS) / HM_MASS
    e2 = abs(u2 - 5e-9 / (dr / 2.0)) / (5e-9 / (dr / 2.0))
    _rec("S4_Ueff_limits", e1 < 1e-10 and e2 < 1e-10,
        {"rel_err_hm_limit": e1, "rel_err_diff_limit": e2, "U_eff_hm": u1, "U_eff_D": u2})


```

```
def S5_kirchhoff_consistency():
    # 检查 Kirchhoff 势的差商。
    Ca, Cb = 0.08, 0.05
    ps = PROPERTY_SETS["app2"]
    Carr = np.array([Ca, Cb])
    dfc = float(D_face(Carr, ps)[0])

    ss = np.linspace(1e-9, Ca, 400001)
    Phi_num = float(np.trapezoid(D_app2(ss), ss))
    Phi_cf = float(Phi_app2(Ca))
    rel_phi = abs(Phi_num - Phi_cf) / abs(Phi_cf)

    eq = float(D_face(np.array([0.3, 0.3]), ps)[0])
    rel_eq = abs(eq - float(D_app2(0.3))) / float(D_app2(0.3))
    Da, Db = float(D_app2(Ca)), float(D_app2(Cb))
    harm = 2 * Da * Db / (Da + Db)
    arith = 0.5 * (Da + Db)
    _rec("S5_kirchhoff_consistency", rel_phi < 1e-6 and rel_eq < 1e-12,
        {"rel_err_Phi_vs_quad": rel_phi, "rel_err_equal_C_degeneracy": rel_eq,
         "D_face_kirchhoff": dfc, "harmonic": harm, "arithmetic": arith,
         "kirchhoff_over_harmonic": dfc / harm, "kirchhoff_over_arith": dfc / arith})


def S6_ale_rigid_scaling():
    R_of, Rdot_of, meta = radius_interpolant()
    res = solve_drying("app4", 7200.0, stop=("time", 7200.0), dt_solve=2.0, dt_out=7200.0,
                   geometry=("ale", (R_of, Rdot_of)), N=50, air_mode="const",
                   T_plateau=T0_C, C_plateau=C0, hm=0.0 + 1e-300, h=1e-300,
                   C_init=C0, T_init=T0_C, record_fields=False)
    dC = float(np.max(np.abs(res["C_final"] - C0)))
    frac_active = res["v_mesh_active_steps"] / max(res["n_steps"], 1)
    _rec("S6_ale_rigid_scaling",
        dC < 1e-10 and res["motion_resid_max"] < 1e-12 and frac_active > 0.5,
        {"max_dC": dC, "motion_cancel_resid_max": res["motion_resid_max"],
         "v_mesh_active_fraction": frac_active, "radius_meta": meta})


def S7_first_passage():
    # 同时测首达和回吸。



    R_small = 0.002
    kw = dict(stop=("first_passage", C_THRESHOLD), dt_out=60.0, N=100,
            geometry=("fixed", R_small), air_mode="const", T_plateau=50.0,
            C_plateau=0.05, hm=8e-5, C_init=0.5, T_init=50.0, record_fields=False)
    a = solve_drying("app3", 20000.0, dt_solve=4.0, **kw)
```

```
    b = solve_drying("app3", 20000.0, dt_solve=1.0, **kw)
    assert a["t_end"] is not None and b["t_end"] is not None, "S7 算例未达标，测试设计失效"
    rel = abs(a["t_end"] - b["t_end"]) / b["t_end"]

    import utils as U
    T_of, _ = U.air_driving(50.0, 0.05, mode="const")

    t_jump = 1.2 * a["t_end"]

    def C_air_rebound(tq):
        tq = np.asarray(tq, dtype=float)
        return np.where(tq < t_jump, 0.02, 0.60)

    step = U.make_stepper("app3", T_of, C_air_rebound, geometry=("fixed", R_small),
                          N=100, hm=8e-5)
    C, T, t = np.full(101, 0.5), np.full(101, 50.0), 0.0
    seen_below, seen_rebound = False, False
    while t < 3.0 * a["t_end"]:
        C, T, _ = step(C, T, t, 4.0)
        t += 4.0
        g = float(np.max(C)) - C_THRESHOLD
        if g < 0:
            seen_below = True
        elif seen_below:
            seen_rebound = True
            break
    _rec("S7_first_passage", rel < 5e-3 and a["bracket"] is not None and seen_rebound,
         {"t_end_dt4_s": a["t_end"], "t_end_dt1_s": b["t_end"], "rel_diff": rel,
          "bracket_len_s": a["bracket"][1] - a["bracket"][0],
          "rebound_case_detected": seen_rebound, "no_rebound_main": a["no_rebound"]})


def S8_unit_misuse_probe():
    # 摄氏度误传必须被拦截。
    raised = {}
    for name, fn in (("D_app3", D_app3), ("D_app4", D_app4)):
        try:
            fn(C0, 50.0)
            raised[name] = False
        except UnitError:
            raised[name] = True

    ok3 = float(D_app3(C0, to_kelvin(50.0)))
    ok4 = float(D_app4(C0, to_kelvin(50.0)))
    silent = float(2.4e-3 * np.exp(-0.45 / C0) * np.exp(-3850.0 / 50.0))
    _rec("S8_unit_misuse_probe", all(raised.values()) and 1e-9 < ok3 < 1e-7,
         {"raised": raised, "D_app3_K": ok3, "D_app4_K": ok4,
```

```
        "what_silent_bug_would_give": silent, "ratio_app4_app3": ok4 / ok3})


def main():
    print("=== 结构性测试 S1-S8（真实算例前置门）===", flush=True)
    S1_geometry_weights()
    S2_zero_flux_constant()
    S3_bessel_limit()
    S3b_surface_form_order()
    S4_Ueff_limits()
    S5_kirchhoff_consistency()
    S6_ale_rigid_scaling()
    S7_first_passage()
    S8_unit_misuse_probe()
    FIGDIR.mkdir(exist_ok=True)
    with open(FIGDIR / "structural_tests.json", "w", encoding="utf-8") as fh:
        json.dump(RESULTS, fh, ensure_ascii=False, indent=2, default=float)
    print(f"\n全部 {len(RESULTS)} 项结构性测试通过 -> figures/structural_tests.json",
        flush=True)


if __name__ == "__main__":
    main()
```

1.10 灵敏度分析

```

from __future__ import annotations

import json
import time

import numpy as np

from params import (AIR_EXTRAP_C, AIR_EXTRAP_T_C, C0, C_THRESHOLD, DT_SOLVE_Q34, FIGDIR,
                H_CONV, HM_MASS, N_CELLS, R0, SENS_C_AIR, SENS_DT_OUT_S,
                SENS_DT_SOLVE_S, SENS_H_FACTORS, SENS_HM_FACTORS, SENS_T_AIR_C,
                T0_C, T_MAX_CAP_S)
from props import D_app3, props_app2
from utils import air_driving, radius_interpolant, solve_drying, volume_average,
    grid_weights

TAG = "sensitivity"
OUT = {}

```

```

def _checkpoint(stage):
   # 中断后可以从最近一节继续检查。
   p = FIGDIR.parent / "_tmp" / f"{TAG}_partial.json"
   p.parent.mkdir(exist_ok=True)
   with open(p, "w", encoding="utf-8") as fh:
      json.dump({"last_completed": stage, **OUT}, fh, ensure_ascii=False,
              indent=2, default=float)


def _t_end_h(pset, geometry, **kw):
   kw.setdefault("dt_solve", SENS_DT_SOLVE_S)
   kw.setdefault("dt_out", SENS_DT_OUT_S)
   kw.setdefault("N", N_CELLS)
   kw.setdefault("air_mode", "linear")
   res = solve_drying(pset, T_MAX_CAP_S, stop=("first_passage", C_THRESHOLD),
                geometry=geometry, C_init=C0, T_init=T0_C,
                record_fields=False, rebound_check=False, **kw)
   return (float(res["t_end"]) / 3600.0) if res["t_end"] is not None else None


def _rel(x, base):
   return None if x is None else (x - base) / base


def _h(x):
   return "unreached" if x is None else f"{x:.4f} h"


def _p(rel):
   return "   n/a" if rel is None else f"{rel * 100:+.4f}%"


def _req(rel, label):
   assert rel is not None,\
      f"§8.3 对照方案 {label} 在 {T_MAX_CAP_S / 3600:.0f} h 上限内未达阈值："\
      "无法与主方案在同一口径下比较相对差，须先登记该对照未达标"
   return rel


def baselines():
   # 所有相对变化都以这里的结果为基准。
   print("[SENS] 基准（dt=4）...", flush=True)
   R_of, Rdot_of, _ = radius_interpolant()
   b3 = _t_end_h("app3", ("fixed", R0))
   b4 = _t_end_h("app4", ("ale", (R_of, Rdot_of)))
   ref3, ref4 = 57.47449652777778, 51.08876736111111
```

```
    OUT["baseline"] = {
        "t_end_q3_h_dt4": b3, "t_end_q4_h_dt4": b4,
        "t_end_q3_h_main_dt2": ref3, "t_end_q4_h_main_dt2": ref4,
        "q3_dt_offset_pct": (b3 - ref3) / ref3 * 100.0,
        "q4_dt_offset_pct": (b4 - ref4) / ref4 * 100.0,
        "note": "扫描用 dt=4 以控制机时；步长偏移 <0.01%，远小于所有被扫参数的效应，"
                "故相对变化的排序不受影响"}
    print(f"[SENS] 基准 Q3={b3:.6f} h Q4={b4:.6f} h "
          f"(相对主结果 {OUT['baseline']['q3_dt_offset_pct']:+.4f}% / "
          f"{OUT['baseline']['q4_dt_offset_pct']:+.4f}%)", flush=True)
    return b3, b4, R_of, Rdot_of


def sec_8_1_air(b3, b4, R_of, Rdot_of):
    # 空气平台温度和浓度的 3x3 扫描。
    print("[SENS] §8.1 空气外推 3x3（Q3 与 Q4 各 9 个组合）...", flush=True)
    assert max(SENS_C_AIR) < C_THRESHOLD,\
        f"C_air 上界 {max(SENS_C_AIR)} 已达阈值 {C_THRESHOLD}，终点不可达（§8.1 有解前提）"
    cells = []
    for Ta in SENS_T_AIR_C:
        for Ca in SENS_C_AIR:
            t3 = _t_end_h("app3", ("fixed", R0), T_plateau=Ta, C_plateau=Ca)
            t4 = _t_end_h("app4", ("ale", (R_of, Rdot_of)), T_plateau=Ta, C_plateau=Ca)
            cells.append({"T_air_C": Ta, "C_air": Ca, "t_end_q3_h": t3, "t_end_q4_h": t4,
                          "rel_q3": _rel(t3, b3), "rel_q4": _rel(t4, b4)})
            print(f" T={Ta} C={Ca}: Q3={_h(t3)} ({_p(_rel(t3, b3))}) "
                  f"Q4={_h(t4)} ({_p(_rel(t4, b4))})", flush=True)

    n_unreached = sum(1 for c in cells if c["t_end_q3_h"] is None or c["t_end_q4_h"] is
        None)
    assert n_unreached == 0,\
        f"§8.1 有 {n_unreached}/{len(cells)} 个组合在 {T_MAX_CAP_S / 3600:.0f} h
            上限内未达阈值："\
        "跨度与单调性无法在同一口径下比较，须先如实登记未达标组合再重排"

    byT = {Ta: [c for c in cells if c["T_air_C"] == Ta] for Ta in SENS_T_AIR_C}
    mono_T = all(byT[SENS_T_AIR_C[0]][j]["t_end_q3_h"] >
        byT[SENS_T_AIR_C[1]][j]["t_end_q3_h"]
              > byT[SENS_T_AIR_C[2]][j]["t_end_q3_h"] for j in range(len(SENS_C_AIR)))
    byC = {Ca: [c for c in cells if c["C_air"] == Ca] for Ca in SENS_C_AIR}
    mono_C = all(byC[SENS_C_AIR[0]][j]["t_end_q3_h"] < byC[SENS_C_AIR[1]][j]["t_end_q3_h"]
              < byC[SENS_C_AIR[2]][j]["t_end_q3_h"] for j in range(len(SENS_T_AIR_C)))
    r3 = [c["rel_q3"] for c in cells]
    r4 = [c["rel_q4"] for c in cells]

    T_K = AIR_EXTRAP_T_C + 273.15
    dlnD_dT = 3850.0 / T_K ** 2
```

```
    OUT["sec_8_1_air_extrapolation"] = {
        "T_air_grid_C": list(SENS_T_AIR_C), "C_air_grid": list(SENS_C_AIR),
        "cells": cells,
        "span_rel_q3": float(max(r3) - min(r3)), "span_rel_q4": float(max(r4) - min(r4)),
        "max_abs_rel_q3": float(max(abs(x) for x in r3)),
        "max_abs_rel_q4": float(max(abs(x) for x in r4)),
        "monotone_T_up_gives_shorter_q3": bool(mono_T),
        "monotone_C_up_gives_longer_q3": bool(mono_C),
        "dlnD_dTK_per_K": dlnD_dT, "D_change_per_1K_pct": dlnD_dT * 100.0,
        "unreachable_bound_note": f"若 C_air >= {C_THRESHOLD} 则终点永不可达；"
                                  f"本扫描上界 {max(SENS_C_AIR)} 远低于该边界，"
                                  "但这一边界是问题三、四有解的前提，必须写入论文",
        "C_air_unreachable_threshold": C_THRESHOLD}
    assert mono_T, "§8.1 预期方向被证伪：T_air 升高未使 t_end 减小，须查 Arrhenius 路径"
    assert mono_C, "§8.1 预期方向被证伪：C_air 升高未使 t_end 增大，须查表面驱动力"
    print(f"[SENS] §8.1 Q3 相对变化跨度 {OUT['sec_8_1_air_extrapolation']['span_rel_q3'] *
        100:.4f}%"
          f" Q4 {OUT['sec_8_1_air_extrapolation']['span_rel_q4'] * 100:.4f}%"
          f"；单调性 T:{mono_T} C:{mono_C}", flush=True)


def sec_8_2_surface(b3, b4, R_of, Rdot_of):
    # 分别扰动传热和传质系数。
    print("[SENS] §8.2 h / h_m 各 ±50% ...", flush=True)
    rows_hm, rows_h = [], []
    for f in SENS_HM_FACTORS:
        hm = HM_MASS * f
        t3 = _t_end_h("app3", ("fixed", R0), hm=hm)
        t4 = _t_end_h("app4", ("ale", (R_of, Rdot_of)), hm=hm)
        rows_hm.append({"factor": f, "hm_m_s": hm, "t_end_q3_h": t3, "t_end_q4_h": t4,
                        "rel_q3": _rel(t3, b3), "rel_q4": _rel(t4, b4)})
        print(f" h_m x{f}: Q3={_h(t3)} ({_p(_rel(t3, b3))}) "
              f"Q4={_h(t4)} ({_p(_rel(t4, b4))})", flush=True)
    for f in SENS_H_FACTORS:
        hh = H_CONV * f
        t3 = _t_end_h("app3", ("fixed", R0), h=hh)
        t4 = _t_end_h("app4", ("ale", (R_of, Rdot_of)), h=hh)
        rows_h.append({"factor": f, "h_W_m2K": hh, "t_end_q3_h": t3, "t_end_q4_h": t4,
                       "rel_q3": _rel(t3, b3), "rel_q4": _rel(t4, b4)})
        print(f" h x{f}: Q3={_h(t3)} ({_p(_rel(t3, b3))}) "
              f"Q4={_h(t4)} ({_p(_rel(t4, b4))})", flush=True)

    n_unreached = sum(1 for r in rows_hm + rows_h
                      if r["t_end_q3_h"] is None or r["t_end_q4_h"] is None)
    assert n_unreached == 0,\
        f"§8.2 有 {n_unreached}/{len(rows_hm) + len(rows_h)} 个扰动在 "\
        f"{T_MAX_CAP_S / 3600:.0f} h 上限内未达阈值：跨度不可比，须先登记未达标项"
```

```
    hm_hi = [r for r in rows_hm if r["factor"] == max(SENS_HM_FACTORS)][0]
    pure_surface_pred = 1.0 / max(SENS_HM_FACTORS) - 1.0
    ctrl = hm_hi["rel_q3"] / pure_surface_pred

    D_thr = float(D_app3(C_THRESHOLD, AIR_EXTRAP_T_C + 273.15))
    D_C0v = float(D_app3(C0, AIR_EXTRAP_T_C + 273.15))
    Rs = 1.0 / HM_MASS
    Ri_thr, Ri_C0 = R0 / D_thr, R0 / D_C0v
    OUT["sec_8_2_surface_coefficients"] = {
        "hm_rows": rows_hm, "h_rows": rows_h,
        "hm_span_rel_q3": float(max(r["rel_q3"] for r in rows_hm)
                                - min(r["rel_q3"] for r in rows_hm)),
        "h_span_rel_q3": float(max(r["rel_q3"] for r in rows_h)
                               - min(r["rel_q3"] for r in rows_h)),
        "hm_span_rel_q4": float(max(r["rel_q4"] for r in rows_hm)
                                - min(r["rel_q4"] for r in rows_hm)),
        "h_span_rel_q4": float(max(r["rel_q4"] for r in rows_h)
                               - min(r["rel_q4"] for r in rows_h)),
        "surface_control_index_q3": ctrl,
        "pure_surface_control_prediction_rel": pure_surface_pred,
        "hm_hi_actual_rel_q3": hm_hi["rel_q3"],
        "resistance_surface_s_m": Rs,
        "resistance_internal_at_C0_s_m": Ri_C0,
        "resistance_internal_at_Cthr_s_m": Ri_thr,
        "internal_fraction_at_C0": Ri_C0 / (Ri_C0 + Rs),
        "internal_fraction_at_Cthr": Ri_thr / (Ri_thr + Rs),
        "verdict": "内部扩散电阻占主导 -> h_m 的 t_end 灵敏度远弱于纯表面控制的 1/h_m 标度"}
    assert abs(ctrl) < 1.0,\
        f"h_m 灵敏度 {hm_hi['rel_q3'] * 100:.4f}% 达到或超过纯表面控制预期 "\
        f"{pure_surface_pred * 100:.4f}%，与内部电阻占主导的结论矛盾"
    print(f"[SENS] §8.2 h_m 跨度 {OUT['sec_8_2_surface_coefficients']['hm_span_rel_q3'] *
        100:.4f}%"
          f" h 跨度 {OUT['sec_8_2_surface_coefficients']['h_span_rel_q3'] * 100:.4f}%"
          f"；表面控制指数 {ctrl:.4f}（内部电阻占比 @Cthr "
          f"{OUT['sec_8_2_surface_coefficients']['internal_fraction_at_Cthr'] * 100:.2f}%）",
          flush=True)


def sec_8_3_forms(b3, b4, R_of, Rdot_of):
    # 比较方程形式和离散选项。
    print("[SENS] §8.3 形式选择对照 ...", flush=True)
    items = {}
    OUT["sec_8_3_form_choices"] = items


    t3 = _t_end_h("app3", ("fixed", R0), mass_form="rho_d_C")
```

```
    t4 = _t_end_h("app4", ("ale", (R_of, Rdot_of)), mass_form="rho_d_C")
    items["mass_form_rho_d_C"] = {
       "main": "C", "alt": "rho_d_C", "t_end_q3_h": t3, "t_end_q4_h": t4,
       "rel_q3": _rel(t3, b3), "rel_q4": _rel(t4, b4),
       "criterion": "问题一应严格一致（rho_d 常数）；问题三、四的差异即 rho_d 变化的影响"}
    print(f" mass_form=rho_d_C: Q3 {_p(_rel(t3, b3))} Q4 {_p(_rel(t4, b4))}", flush=True)
```

```
    rho2 = float(props_app2()[0])
    kwu = dict(N=N_CELLS, air_mode="linear", C_init=C0, T_init=T0_C, record_fields=False)

    def _two_forms(span, dt, **extra):
       a_ = solve_drying("app2", span, stop=("time", span), dt_solve=dt, dt_out=span,
                         **kwu, **extra)
       b_ = solve_drying("app2", span, stop=("time", span), dt_solve=dt, dt_out=span,
                         mass_form="rho_d_C", **kwu, **extra)
       rd_ = rho2 / (1.0 + a_["C_final"])
       return (float(np.max(np.abs(a_["C_final"] - b_["C_final"]))),
               float(rd_.max() / rd_.min() - 1.0))


    dC_unif, sp_unif = _two_forms(0.25, 0.25, picard_fixed=1)

    ladder = []
    for span in (60.0, 300.0, 1800.0):
       d_, s_ = _two_forms(span, 0.25)
       ladder.append({"span_s": span, "max_dC": d_, "rho_d_spread": s_})
       print(f" mass_form Q1 t={span:6.1f}s: dC={d_:.6e} rho_d 跨度={s_:.6e}", flush=True)
    dC1 = ladder[-1]["max_dC"]
    items["mass_form_q1_identity"] = {
       "claim_under_test": "§1 假设(4) / LOGIC_CONTRACT A4：附录2 rho 常数 => rho_d 常数 "
                           "=> 两形式严格等价，Q1 敏感度恰为 0",
       "verdict": "falsified",
       "why": "rho_d = rho/(1+C)（§10 参数表定义）。附录2 的 rho=820 为常数，但 C 沿径向"
            "不均匀，故 rho_d 在 C in [0.15,2.55] 上取 230.9859~713.0435 kg/m^3（3.0870
                倍），"
            "并非常数。等价性的真正条件是 rho_d 空间均匀，而非 rho 为常数。",
       "true_condition_uniform_rho_d": {
          "max_dC": dC_unif, "rho_d_spread": sp_unif,
          "setup": "首步 dt=0.25 s + picard_fixed=1：系数取在均匀初值 C=C0 上，rho_d 逐点相等",
          "criterion_lt_1e-12": bool(dC_unif < 1e-12)},
       "nonuniformity_ladder": ladder,
       "max_dC_at_1800s": dC1,
       "rho_d_at_C0": rho2 / (1.0 + C0), "rho_d_at_Cthr": rho2 / (1.0 + C_THRESHOLD),
```

```
    "rho_d_ratio_Cthr_over_C0": (1.0 + C0) / (1.0 + C_THRESHOLD),
    "note": "Q1 的 8.675e-02 不是实现误差，而是假设(4) 在问题一口径下的真实效应量；"
          "报告与逻辑契约中『Q1 敏感度恰为 0』一项据此改为『仅在 rho_d 均匀时为 0』"}

assert dC_unif < 1e-12,\
    f"rho_d 空间均匀时两形式应逐点相同，实得 dC={dC_unif:.3e}：离散层面的缩放不变性被破坏"

assert all(ladder[i]["max_dC"] < ladder[i + 1]["max_dC"]
        and ladder[i]["rho_d_spread"] < ladder[i + 1]["rho_d_spread"]
        for i in range(len(ladder) - 1)),\
    f"两形式差未随 rho_d 非均匀度单调增长：{ladder}，差的来源需重新归因"
assert dC_unif < ladder[0]["max_dC"],\
    "均匀 rho_d 极限的差未小于最短非均匀情形，等价条件的判定失效"
print(f" mass_form Q1 等价条件：均匀 rho_d 下 dC={dC_unif:.3e}（跨度 {sp_unif:.3e}）；"
     f"1800 s 时 dC={dC1:.3e} —— 『Q1 恰为 0』证伪", flush=True)


kwp = dict(stop=("time", 1800.0), dt_solve=0.25, dt_out=1800.0, N=N_CELLS,
        C_init=C0, T_init=T0_C, record_fields=False)
la = solve_drying("app2", 1800.0, air_mode="linear", **kwp)
pa = solve_drying("app2", 1800.0, air_mode="pchip", **kwp)
dT_air = float(np.max(np.abs(la["T_final"] - pa["T_final"])))
t3p = _t_end_h("app3", ("fixed", R0), air_mode="pchip")
items["air_interp_pchip"] = {
    "main": "linear", "alt": "pchip", "max_dT_1800s_K": dT_air,
    "t_end_q3_h": t3p, "rel_q3": _rel(t3p, b3),
    "criterion_dT_lt_0p01K": bool(dT_air < 0.01)}
assert dT_air < 0.01, f"附件1 插值方式使 T(r,1800) 差 {dT_air:.4e} K > 0.01
    K：驱动数据分辨率不足"
print(f" air_interp=pchip: dT(1800s)={dT_air:.4e} K, Q3 t_end {_rel(t3p, b3) *
    100:+.4f}%",
     flush=True)


t3a = _t_end_h("app3", ("fixed", R0), d_face_mode="arithmetic")
t4a = _t_end_h("app4", ("ale", (R_of, Rdot_of)), d_face_mode="arithmetic")
items["d_face_arithmetic"] = {
    "main": "kirchhoff", "alt": "arithmetic", "t_end_q3_h": t3a, "t_end_q4_h": t4a,
    "rel_q3": _rel(t3a, b3), "rel_q4": _rel(t4a, b4),
    "criterion": "差异用于说明 §5.2 Kirchhoff 的必要性（退化扩散下平均方式非中性）"}
print(f" d_face=arithmetic: Q3 {_p(_rel(t3a, b3))} Q4 {_p(_rel(t4a, b4))}", flush=True)


t3f = _t_end_h("app3", ("fixed", R0), picard_fixed=2)
items["picard_fixed_2"] = {"main": "tol 1e-10", "alt": "fixed 2 iters",
                    "t_end_q3_h": t3f, "rel_q3": _rel(t3f, b3),
                    "criterion_rel_lt_1e-4": bool(abs(_req(_rel(t3f, b3),
```

```
                                                    "picard_fixed_2")) < 1e-4)}
    assert abs(_req(_rel(t3f, b3), "picard_fixed_2")) < 1e-4,\
        f"PX-C3: 固定 2 轮使 t_end 变 {_rel(t3f, b3):.3e}（>1e-4），疑把迭代当推进"
    print(f" picard_fixed=2: Q3 {_rel(t3f, b3) * 100:+.6f}%", flush=True)


    Rl_of, Rldot_of, meta_lin = radius_interpolant(mode="linear")
    t4l = _t_end_h("app4", ("ale", (Rl_of, Rldot_of)))
    items["radius_interp_linear"] = {
        "main": "pchip", "alt": "linear", "t_end_q4_h": t4l, "rel_q4": _rel(t4l, b4),
        "criterion_rel_lt_0p5pct": bool(abs(_req(_rel(t4l, b4),
                                        "radius_interp_linear")) < 5e-3),
        "cubic_spline_forbidden": "三次样条在平台衔接处过冲出实测下界 1.198 cm，禁用",
        "linear_meta": meta_lin}
    assert abs(_req(_rel(t4l, b4), "radius_interp_linear")) < 5e-3,\
        f"附件2 插值方式使 t_end 差 {_rel(t4l, b4) * 100:.4f}%（>0.5%），半径数据分辨率不足"
    print(f" radius_interp=linear: Q4 {_rel(t4l, b4) * 100:+.4f}%", flush=True)


    t3s = _t_end_h("app3", ("fixed", R0), surface_form="series")
    t4s = _t_end_h("app4", ("ale", (R_of, Rdot_of)), surface_form="series")
    items["surface_form_series"] = {
        "main": "node (bare h)", "alt": "series U_eff (§4.2 原式)",
        "t_end_q3_h": t3s, "t_end_q4_h": t4s,
        "rel_q3": _rel(t3s, b3), "rel_q4": _rel(t4s, b4),
        "why_main_is_node": "节点中心布局下末节点即表面，串联 (dr/2)/D 重复计入半格内阻；"
                    "S3b 实测 series 误差 ~O(dr)（N=200/400: 1.99e-2/1.02e-2 K），"
                    "node 误差与 dr 无关（4.94e-4/4.91e-4 K）"}
    print(f" surface_form=series: Q3 {_p(_rel(t3s, b3))} Q4 {_p(_rel(t4s, b4))}",
          flush=True)

    OUT["sec_8_3_form_choices"] = items


def sec_8_4_tornado():
    # 按实际跨度排序。
    a1 = OUT["sec_8_1_air_extrapolation"]
    a2 = OUT["sec_8_2_surface_coefficients"]
    a3 = OUT["sec_8_3_form_choices"]

    def _span(rows, key):
        vals = [r[key] for r in rows if r.get(key) is not None]
        return float(max(vals) - min(vals)) if vals else None


    cells = a1["cells"]
    T_mid, C_mid = SENS_T_AIR_C[1], SENS_C_AIR[1]
```

```
T_only = [c for c in cells if c["C_air"] == C_mid]
C_only = [c for c in cells if c["T_air_C"] == T_mid]

def _abs(x):
    return None if x is None else abs(x)

entries = []
for name, q3, q4, note in (
      ("air_T_infty_pm1K", _span(T_only, "rel_q3"), _span(T_only, "rel_q4"),
       "§8.1 T_air 49/50/51 C（C_air 固定 0.05）"),
      ("air_C_infty_pm10pct", _span(C_only, "rel_q3"), _span(C_only, "rel_q4"),
       "§8.1 C_air 0.045/0.05/0.055（T_air 固定 50 C）"),
      ("air_joint_3x3", a1["span_rel_q3"], a1["span_rel_q4"],
       "§8.1 两者联合 3x3 全跨度"),
      ("h_m_pm50pct", a2["hm_span_rel_q3"], a2["hm_span_rel_q4"], "§8.2 传质系数"),
      ("h_pm50pct", a2["h_span_rel_q3"], a2["h_span_rel_q4"], "§8.2 传热系数"),
      ("mass_form_rho_d_C", _abs(a3["mass_form_rho_d_C"]["rel_q3"]),
       _abs(a3["mass_form_rho_d_C"]["rel_q4"]), "§8.3 水分方程形式"),
      ("d_face_arithmetic", _abs(a3["d_face_arithmetic"]["rel_q3"]),
       _abs(a3["d_face_arithmetic"]["rel_q4"]), "§8.3 界面 D 取算术平均"),
      ("surface_form_series", _abs(a3["surface_form_series"]["rel_q3"]),
       _abs(a3["surface_form_series"]["rel_q4"]), "§8.3 表面 Robin 串联形式"),
      ("air_interp_pchip", _abs(a3["air_interp_pchip"]["rel_q3"]), None,
       "§8.3 附件1 插值改 PCHIP"),
      ("radius_interp_linear", None, _abs(a3["radius_interp_linear"]["rel_q4"]),
       "§8.3 附件2 插值改线性"),
      ("picard_fixed_2", _abs(a3["picard_fixed_2"]["rel_q3"]), None,
       "§8.3 Picard 固定 2 轮")):
    entries.append({"factor": name, "span_rel_q3": q3, "span_rel_q4": q4,
                    "span_pct_q3": None if q3 is None else q3 * 100.0,
                    "span_pct_q4": None if q4 is None else q4 * 100.0,
                    "description": note})
rank3 = sorted([e for e in entries if e["span_rel_q3"] is not None],
               key=lambda e: -e["span_rel_q3"])
rank4 = sorted([e for e in entries if e["span_rel_q4"] is not None],
               key=lambda e: -e["span_rel_q4"])
OUT["sec_8_4_tornado"] = {
    "entries": entries,
    "ranking_q3": [e["factor"] for e in rank3],
    "ranking_q4": [e["factor"] for e in rank4],
    "top_factor_q3": rank3[0]["factor"], "top_span_pct_q3": rank3[0]["span_pct_q3"],
    "top_factor_q4": rank4[0]["factor"], "top_span_pct_q4": rank4[0]["span_pct_q4"],
    "metric": "t_end 相对变化幅度（跨度），不作显著/不显著判断",
    "note": "排序由本次计算产生；§8.4 的先验猜测是 h_m 或空气外推，实际结果见 ranking_*"}
print(f"[SENS] §8.4 Q3 首位 {rank3[0]['factor']} ({rank3[0]['span_pct_q3']:.4f}%)；"
      f"Q4 首位 {rank4[0]['factor']} ({rank4[0]['span_pct_q4']:.4f}%)", flush=True)
for e in rank3[:5]:
```

```
        print(f" Q3 {e['factor']:<24s} {e['span_pct_q3']:8.4f}%", flush=True)
    for e in rank4[:5]:
        print(f" Q4 {e['factor']:<24s} {e['span_pct_q4']:8.4f}%", flush=True)


def run():
    t0 = time.time()
    b3, b4, R_of, Rdot_of = baselines()
    _checkpoint("baseline")
    sec_8_1_air(b3, b4, R_of, Rdot_of)
    _checkpoint("sec_8_1")
    sec_8_2_surface(b3, b4, R_of, Rdot_of)
    _checkpoint("sec_8_2")
    sec_8_3_forms(b3, b4, R_of, Rdot_of)
    _checkpoint("sec_8_3")
    sec_8_4_tornado()
    OUT["wall_time_s"] = time.time() - t0
    OUT["dt_solve_s"] = SENS_DT_SOLVE_S
    OUT["N_cells"] = N_CELLS
    FIGDIR.mkdir(exist_ok=True)
    with open(FIGDIR / f"{TAG}_results.json", "w", encoding="utf-8") as fh:
        json.dump(OUT, fh, ensure_ascii=False, indent=2, default=float)
    print(f"[SENS] -> figures/{TAG}_results.json ({OUT['wall_time_s']:.1f} s)", flush=True)
    return OUT


if __name__ == "__main__":
    run()
```

1.11 程序入口

```

from __future__ import annotations

import argparse
import json
import os
import sys
import time


_HERE = os.path.dirname(os.path.abspath(__file__))
# 直接运行脚本时也能找到同目录模块。
if _HERE not in sys.path:
    sys.path.insert(0, _HERE)
```

```python

import numpy as np

from params import C_THRESHOLD, FIGDIR


STAGES = [
    # 后一阶段会读取前一阶段的结果。
    ("structural_tests", "structural_tests", "结构性检验 S1–S8（解析极限与单位误用探针）"),
    ("problem_1", "problem1", "问题一：附录2 常物性解耦场，t≤1800 s"),
    ("problem_2", "problem2", "问题二：附录3 双向耦合，t≤3 h"),
    ("problem_3", "problem3", "问题三：附录3 固定几何，干燥终点首达 t_end"),
    ("problem_4", "problem4", "问题四：附录4 + ALE 移动边界，干燥终点首达 t_end"),
    ("sensitivity", "sensitivity_analysis", "灵敏度与鲁棒性 §8 + 龙卷排序"),
]


def _path(tag):
    # structural_tests 的文件名不同。
    if tag == "structural_tests":
        return FIGDIR / "structural_tests.json"
    return FIGDIR / f"{tag}_results.json"


def _verify(tag, d):
    def finite(x):
        return x is not None and np.all(np.isfinite(np.atleast_1d(np.asarray(x, float))))

    if tag == "structural_tests":
        if not d:
            return False, "结果为空"
        bad = [k for k, v in d.items() if isinstance(v, dict) and not v.get("pass")]
        return (not bad), (f"S1–S8 全部通过（{len(d)} 项）" if not bad else f"未通过 {bad}")

    if tag == "problem_1":
        for key in ("Bi_T", "Bi_m", "alpha_m2_s", "C_bar_1800"):
            if not finite(d.get(key)):
                return False, f"{key} 缺失或非有限"
        err = d.get("duhamel_check", {}).get("max_abs_err_K")
        if not finite(err):
            return False, "Duhamel 独立解校核缺失"

        if (d.get("decoupling_swap_order_max_dT_K") != 0.0
                or d.get("decoupling_swap_order_max_dC") != 0.0):
            return False, "解耦顺序互换未逐点一致"
        return True, f"Duhamel 独立解最大偏差 {float(err):.2e} K，解耦顺序互换严格一致"

```

```
    if tag == "problem_2":
        if not finite(d.get("C_bar_10800")):
            return False, "Cbar(3h) 缺失或非有限"
        rel = d.get("PXC3_picard_invariance", {}).get("rel_diff")
        if not finite(rel) or abs(float(rel)) >= 1e-4:
            return False, f"PX-C3 Picard 不变性未通过（rel_diff={rel}）"
        if d.get("P2C2_long_vs_short_max_dC") != 0.0:
            return False, "P2-C2 长程/短程解不一致"
        return True, f"Cbar(3h)={d['C_bar_10800']:.6f}，Picard 不变性 rel={float(rel):.2e}"

    if tag in ("problem_3", "problem_4"):
        t_end = d.get("t_end_s")
        mx = d.get("at_t_end", {}).get("max_C")
        if not finite(t_end) or not finite(mx):
            return False, "t_end 或 t_end 处 max_r C 缺失"

        g = float(mx) - C_THRESHOLD
        if g > 0:
            return False, f"t_end 处 max_r C={mx:.10f} 未过阈值（g={g:+.2e}）"
        if abs(g) > 1e-5:
            return False, f"t_end 处 g={g:+.2e} 超出二分容差量级"
        if not d.get("no_rebound_to_1.05t_end"):
            return False, "1.05·t_end 内 max_r C 出现回升，首达时刻不成立"
        return True, f"t_end={float(t_end) / 3600:.4f} h, g={g:+.2e}, 无回升"

    if tag == "sensitivity":
        tor = d.get("sec_8_4_tornado", {})
        if not tor.get("ranking_q3") or not tor.get("ranking_q4"):
            return False, "龙卷排序缺失"
        return True, f"Q3 首位 {tor['top_factor_q3']}, Q4 首位 {tor['top_factor_q4']}"

    return False, "未知阶段"


def _load(tag):
    p = _path(tag)
    if not p.exists() or p.stat().st_size == 0:
        return None
    try:
        with open(p, "r", encoding="utf-8") as fh:
            return json.load(fh)
    except (json.JSONDecodeError, OSError) as exc:
        print(f" 缓存不可读（{exc}）→ 重算", flush=True)
        return None


def run_stage(tag, module, desc, force):
```

```
    print(f"\n{'=' * 72}\n[MAIN] {tag}: {desc}\n{'=' * 72}", flush=True)
    if not force:
        cached = _load(tag)
        if cached is not None:
            ok, why = _verify(tag, cached)
            if ok:
                print(f"[MAIN] 复用已验证缓存 → {why}", flush=True)
                return cached, True
            print(f"[MAIN] 缓存未通过复核（{why}）→ 重算", flush=True)

    mod = __import__(module)
    entry = getattr(mod, "run", None) or getattr(mod, "main")
    t0 = time.time()
    entry()
    d = _load(tag)
    if d is None:
        raise RuntimeError(f"{tag} 运行结束但未写出 {_path(tag).name}")
    ok, why = _verify(tag, d)
    if not ok:
        raise RuntimeError(f"{tag} 新结果未通过复核：{why}")
    print(f"[MAIN] {tag} 完成（{time.time() - t0:.1f} s）→ {why}", flush=True)
    return d, False


def summarize(results):
    q1, q2 = results["problem_1"], results["problem_2"]
    q3, q4 = results["problem_3"], results["problem_4"]
    t3, t4 = float(q3["t_end_s"]), float(q4["t_end_s"])
    mech = q4["vs_problem3"]["competing_mechanisms"]
    return {
        "problem_1": {"t_span_s": 1800.0,
                  "T_center_1800s_C": q1["t1800_temperature_C"]["0.0"],
                  "T_surface_1800s_C": q1["t1800_temperature_C"]["2.0"],
                  "C_center_1800s": q1["t1800_moisture_kgkg"]["0.0"],
                  "C_surface_1800s": q1["t1800_moisture_kgkg"]["2.0"],
                  "C_bar_1800s": q1["C_bar_1800"],
                  "Bi_T": q1["Bi_T"], "Bi_m": q1["Bi_m"],
                  "Le": q1["Le_alpha_over_D"]},
        "problem_2": {"t_span_s": 10800.0,
                  "T_center_3h_C": q2["t10800_temperature_C"]["0.0"],
                  "C_center_3h": q2["t10800_moisture_kgkg"]["0.0"],
                  "C_surface_3h": q2["t10800_moisture_kgkg"]["2.0"],
                  "C_bar_3h": q2["C_bar_10800"]},
        "problem_3": {"t_end_s": t3, "t_end_h": t3 / 3600.0, "t_end_d": t3 / 86400.0,
                  "max_C_at_t_end": q3["at_t_end"]["max_C"],
                  "C_bar_at_t_end": q3["at_t_end"]["C_bar"],
                  "wrong_criterion_C_bar_t_end_h": q3["wrong_criterion_C_bar_t_end_h"],
```

```
                "geometry": "fixed R=2 cm", "property_set": "appendix3"},
      "problem_4": {"t_end_s": t4, "t_end_h": t4 / 3600.0, "t_end_d": t4 / 86400.0,
                "max_C_at_t_end": q4["at_t_end"]["max_C"],
                "C_bar_at_t_end": q4["at_t_end"]["C_bar"],
                "R_final_cm": q4["R_at_t_end_cm"],
                "frozen_radius_t_end_h":
                   q4["P4C1c_frozen_radius"]["t_end_frozen_R_h"],
                "geometry": "ALE moving boundary (附件2)", "property_set": "appendix4"},
      "q4_vs_q3": {"t_end_ratio": t4 / t3,
                "delta_h": (t4 - t3) / 3600.0,
                "D4_over_D3_at_C0_50C": mech["D4_over_D3_at_C0_50C"],
                "geometric_enhancement_area_ratio":
                   mech["geometric_enhancement_area_ratio"],
                "verdict": "收缩带来的几何增强胜过附录4 更小的扩散系数"
                        if t4 < t3 else "附录4 更小的扩散系数占主导"},
      "threshold": C_THRESHOLD,
      "criterion": "干燥终点判据为 max_r C(r,t) < 0.15（全域上确界），非体积平均",
   }


def main(argv=None):
   ap = argparse.ArgumentParser(description="药材烘干四问主调度")
   ap.add_argument("--force", action="store_true", help="忽略缓存，全部重算")
   ap.add_argument("--only", nargs="*", default=None,
               help="只跑指定阶段（tag），其余阶段仍从缓存读取以完成汇总")
   ap.add_argument("--analysis-global", action="store_true",
               help="只运行问题四非均匀收缩、Monte Carlo 与 Shapley 分析")
   ap.add_argument("--quick", action="store_true",
               help="用于 --analysis-global 的快速检查模式")
   args = ap.parse_args(argv)

   if args.analysis_global:
      from global_q4_analysis import main as run_global
      return run_global(["--quick"] if args.quick else [])

   t0 = time.time()
   results, reused = {}, []
   for tag, module, desc in STAGES:
      if args.only and tag not in args.only:
         cached = _load(tag)
         if cached is None:
            raise SystemExit(f"--only 跳过了 {tag} 但其结果文件不存在，无法汇总")
         ok, why = _verify(tag, cached)
         if not ok:
            raise SystemExit(f"--only 跳过的 {tag} 缓存未通过复核：{why}")
         results[tag], _ = cached, True
         reused.append(tag)
```

```
            print(f"[MAIN] {tag}: --only 跳过，复用缓存（{why}）", flush=True)
            continue
        d, was_reused = run_stage(tag, module, desc, args.force)
        results[tag] = d
        if was_reused:
            reused.append(tag)

    summary = summarize(results)
    all_out = {
        "meta": {"stages": [s[0] for s in STAGES], "reused_from_cache": reused,
                 "wall_time_s": time.time() - t0,
                 "python": sys.version.split()[0], "numpy": np.__version__},
        "summary": summary,
        "problem_1": results["problem_1"], "problem_2": results["problem_2"],
        "problem_3": results["problem_3"], "problem_4": results["problem_4"],
        "structural_tests": results["structural_tests"],
        "sensitivity": results["sensitivity"],
    }
    FIGDIR.mkdir(exist_ok=True)
    with open(FIGDIR / "all_results.json", "w", encoding="utf-8") as fh:
        json.dump(all_out, fh, ensure_ascii=False, indent=2, default=float)

    print(f"\n{'=' * 72}\n[MAIN] 汇总", flush=True)
    print(f" 问题一 T(0,1800s) = {summary['problem_1']['T_center_1800s_C']:.4f} °C, "
          f"Cbar = {summary['problem_1']['C_bar_1800s']:.6f}", flush=True)
    print(f" 问题二 Cbar(3h) = {summary['problem_2']['C_bar_3h']:.6f}", flush=True)
    print(f" 问题三 t_end = {summary['problem_3']['t_end_h']:.4f} h "
          f"({summary['problem_3']['t_end_d']:.4f} d)", flush=True)
    print(f" 问题四 t_end = {summary['problem_4']['t_end_h']:.4f} h "
          f"({summary['problem_4']['t_end_d']:.4f} d), R_end = "
          f"{summary['problem_4']['R_final_cm']:.4f} cm", flush=True)
    print(f" Q4/Q3 = {summary['q4_vs_q3']['t_end_ratio']:.5f}"
          f"（{summary['q4_vs_q3']['delta_h']:+.4f} h）", flush=True)
    print(f"[MAIN] -> figures/all_results.json（总 {all_out['meta']['wall_time_s']:.1f} s, "
          f"复用 {len(reused)}/{len(STAGES)} 阶段）", flush=True)
    return all_out


if __name__ == "__main__":
    main()
```